LocalBuddy — Backend Architecture & Design

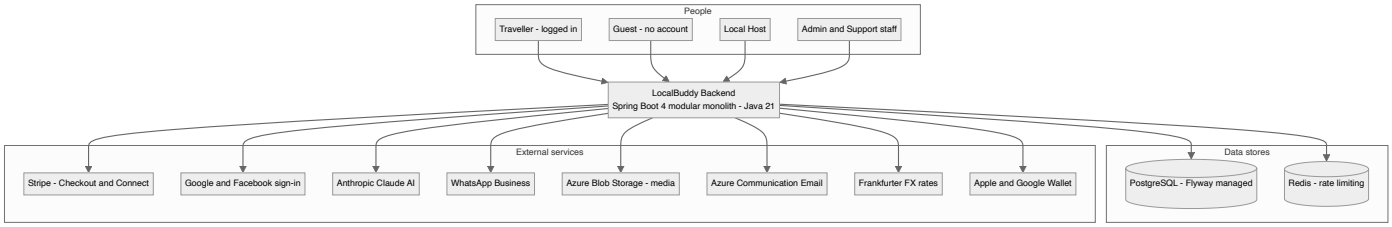
Architecture documentation set · June 2026

This set documents the architecture and design of the LocalBuddy backend for a senior software-architect audience. It was produced by auditing the live codebase (538 Java source files, 53 JPA entities, 24 Flyway migrations) across eight architectural lenses, so every claim and diagram is grounded in the source, not the roadmap. Diagrams are Mermaid and render natively on GitHub, in VS Code, and at mermaid.live.

Architecture in one paragraph

LocalBuddy is a **modular monolith**: a single Spring Boot 4 / Java 21 fat-JAR organised into ~45 feature packages under `com.localbuddy`, each following a `controller → service → repository → @Entity` layering with shared cross-cutting concerns (`auth`, `config`, `common`, `audit`, `ratelimit`). State lives in **PostgreSQL** (schema owned by JPA entities, applied by **Flyway**, validated by Hibernate at boot); **Redis** backs rate-limiting only. Authentication is **stateless JWT** (plus Google/Facebook social login) with role-based authorization and a first-class **guest (no-account)** identity. The commercial core is a **money engine** — a pure- `BigDecimal` pricing calculator that splits a guest service fee from host commission with VAT and configurable discount cost-bearer, wired to **Stripe** hosted Checkout + webhooks for payment and **Stripe Connect** for host payouts, with refunds + proportional clawback and VAT invoicing, all holding a cash-conservation invariant. Background work (notifications, reminders, payout runs, booking expiry) runs **in-process** via nine `@Scheduled` pollers over a transactional **outbox**. It is deployed as a plain JAR to an **Azure Web App** via a manually-triggered GitHub Actions workflow.

System context



The documents

#	Document	What it covers
1	System Architecture — Stack, Runtime & Modules	Tech stack, runtime/process model, build & Azure deployment, modular-monolith package map, layering, cross-cutting concerns.
2	Data Architecture & Domain Model	Domain model & ER diagram, the ~50-table schema by domain, JPA mapping, money/geo columns, retention.
3	Security, Identity, Consent & Data Protection	AuthN (JWT + social), role-based authZ, CORS, rate-limiting, versioned consent, GDPR export/erasure, audit, guest identity, known gaps.
4	Money — Pricing Engine, Payments, Payouts & Invoicing	Pricing engine, Stripe Checkout + webhooks, Stripe Connect payouts, refunds + clawback, invoicing, the cash-conservation invariant.
5	Booking, Availability, Concurrency & Lifecycle	Availability & capacity, pessimistic seat-reservation locking, the booking state machine and every transition, scheduled jobs.
6	External Integrations, Adapters & Resilience	Every outbound integration (Stripe, social auth, Claude, WhatsApp, Azure Blob, Frankfurter, email, wallet) as ports-and-adapters with fallbacks.
7	End-to-End Flows, Eventing & Observability	End-to-end sequence diagrams (onboarding→approval, geo check-in, no-show→refund) and the notification outbox / scheduled-job backbone.

Consolidated architectural decisions & trade-offs

A cross-cutting register of the notable decisions surfaced by the audit (see each document for context).

System Architecture — Stack, Runtime & Modules

- Monolithic fat-JAR over containers: build produces a single executable Spring Boot JAR (localbuddy-backend-0.0.1-SNAPSHOT.jar) deployed directly to Azure Web App via azure/webapps-deploy; no Dockerfile or docker-compose exists in the repo.
- Spring Boot 4.0.6 on the bleeding edge (Jakarta EE, spring-boot-starter-webmvc/-flyway naming), pinning Java 21 as the language level but compiling Kotlin stdlib to an obsolete jvmTarget 1.8 — an inconsistency, though no Kotlin sources are actually present.
- Configuration is 100% environment-variable driven from a single application.yaml with inline ${ENV:default} placeholders; there are NO profile-specific YAML files (application-prod.yaml etc.). The only Spring profile in code is dev (default), which only gates DevAdminBootstrap seeding.
- Schema is owned by JPA entities and enforced at boot via spring.jpa.hibernate.ddl-auto: validate; Flyway (baseline-on-migrate true, V1..V24) applies the DDL. A consolidated V1 baseline was authored from the entities as source of truth.
- Secrets (JWT, Stripe, Azure, WhatsApp, Google/Apple Wallet, Anthropic) are externalized to env vars with empty-string defaults; the CI workflow only injects an Azure publish profile secret, so all runtime secrets must be set as Azure App Settings out-of-band.
- Background work runs in-process via Spring @Scheduled fixedDelay pollers (9 schedulers) plus @EnableAsync; there is no external job queue or worker tier, which couples scheduled processing to every running instance and is unsafe to scale horizontally without leader election.
- HikariCP pool is deliberately tiny (max 5, min-idle 1) — sized for a single small Azure instance and a constrained Postgres connection budget.
- CI/CD is intentionally manual (workflow_dispatch only, push trigger commented out) and deploys a tests-skipped artifact (mvnw -DskipTests clean package), so the pipeline does not gate releases on the test suite.
- Modular monolith with package-per-feature (~44 packages under com.localbuddy), each owning its full vertical stack — but module boundaries are convention-only: no Spring Modulith, no ArchUnit, no package-info markers, so any service can inject any other module's repository.
- Strict 4-tier layering: @RestController -> @Service (@Transactional) -> Spring Data JpaRepository -> JPA @Entity. DTOs (request records + *Response records) keep entities off the wire; mapping is hand-written via static Response.from(...) factories (12 found) — no MapStruct.
- API surface is segmented by audience via path prefix rather than versioning: /api/public/** (unauthenticated, rate-limited), /api/admin/** (ROLE_ADMIN), /api/host/**, and authenticated /api/**. No version segment (no /v1) — versioning is an explicit gap.
- Authorization is coarse URL-based matching in SecurityConfig plus per-handler Authentication params (122 occurrences of UUID.fromString(authentication.getName())). No @PreAuthorize / method security; ownership checks live inside services.
- Cross-cutting concerns are centralized as a few beans: one @RestControllerAdvice (GlobalExceptionHandler) emitting a uniform ErrorResponse, a stateless JwtAuthenticationFilter, a Redis-backed RateLimitService that fails open, @EnableAsync + @EnableScheduling, and @ConfigurationProperties for typed config.
- Inter-module integration is in-process and synchronous (direct service/repository calls), with BookingService as the dependency hub (~13 modules). The only decoupled path is booking notifications via a Spring ApplicationEvent consumed by an @Async @TransactionalEventListener AFTER_COMMIT.
- The audit/ package is an empty .gitkeep placeholder — there is no global audit framework; auditing is local and ad hoc (e.g. pricing/RateChangeAudit only).
- Schema is owned by Flyway (24 V-migrations, ddl-auto=validate); entities mix JPA @ManyToOne associations (32 entities) with raw UUID FK columns, and there is no @MappedSuperclass base entity for common id/timestamp fields.

Data Architecture & Domain Model

- Entity-wins schema authority: ddl-auto=validate (never update); V1__baseline_schema.sql is a consolidated 1205-line baseline regenerated from the @Entity classes, with V2-V24 as forward deltas. Where old migrations and entities disagreed, the entity is authoritative (annotated ENTITY-WINS in V1), e.g. reviews.booking_id UNIQUE was dropped to allow bidirectional reviews.

- UUID surrogate PKs everywhere via GenerationType.UUID, defaulted server-side to gen_random_uuid() (pgcrypto). Exceptions are application-assigned PKs: CancellationRefundPolicy (UUID, no @GeneratedValue) and natural keys BookingReferenceSequence (LocalDate) and InvoiceSequence (year INT).
- All enums persisted as @Enumerated(EnumType.STRING) into VARCHAR columns (no Postgres enum types), trading storage for safe additive evolution; rename migrations (V14/V15) must rewrite stored string values.
- Money is uniformly NUMERIC(10,2) (NUMERIC(12,2) for payouts/ledger/invoices), rates NUMERIC(5,4); each row carries its own 3-char currency defaulting to EUR. There is no FX/multi-currency conversion engine - currency is a stored attribute, not a converted one.
- Immutable financial snapshot on payments: commission/service-fee/VAT/net/gross/place-of-supply are resolved at booking time from time-boxed scoped rules (commission_rules, service_fee_rules, vat_rates) and frozen on the Payment row, decoupling historical money from later rate changes.
- Append-only host_ledger_entries (EARNING/REVERSAL/ADJUSTMENT x PENDING/AVAILABLE/PAID/REVERSED) is the source of truth for host earnings, with a partial-unique index guaranteeing at most one EARNING per payment; payouts batch AVAILABLE entries and clawbacks are negative REVERSALs.
- Concurrency/duplicate protection via partial unique indexes rather than app locks: one active booking per (traveler|guest, slot), one active payment per booking, one EARNING per payment, one host check-in per slot, one guest check-in per booking, idempotent webhooks via UNIQUE(provider, provider_event_id).
- No generic soft-delete column; deletion is modeled as state (UserStatus.DELETED) plus explicit lifecycle/event tables: data_deletion_requests (GDPR), user_consents and per-booking consent snapshot columns, and append-only audit_logs / rate_change_audit / attendance_check_ins for retention and forensics.
- Dual identity for bookings: a row is anchored to either a logged-in User (traveler_user_id) or denormalized guest_* fields, enforced by chk_bookings_traveler_or_guest; the same OR-guest pattern repeats in waitlist and promo/referral redemptions.
- Geolocation is lightweight: experiences.latitude/longitude and check-in coordinates are plain NUMERIC(9,6) columns with no PostGIS extension; distance is computed in application code, so 'near me' sorting is not index-accelerated.

Security, Identity, Consent & Data Protection

- Stateless, non-revocable access tokens only: HS256 JWT, 15-minute default expiry (app.security.jwt.access-token-expiration-minutes), no refresh token and no server-side session or token blacklist. Logout and revocation are impossible until natural expiry.
- Authorization is split across two layers: coarse URL rules in SecurityConfig (only /api/users/** and /api/admin/** are pinned to ROLE_ADMIN; everything else is merely authenticated), and fine-grained ownership/role checks performed manually inside services (e.g. BookingService.getBookingById, LocalProfile lookups). Method-level security (@EnableMethodSecurity / @PreAuthorize) is NOT enabled anywhere.
- LOCAL (host) authorization is resource-driven, not route-driven: host operations require an approved LocalProfile for the caller rather than a role check on the URL, so the JWT role claim is largely advisory outside the ADMIN routes.
- Guest (no-account) identity = booking reference plus matching guest email, verified on every lookup (BookingService.lookupGuestBooking); guest mutations are protected only by Redis IP rate limiting, not by a secret token.
- Versioned consent enforcement: a single CURRENT_CONSENT_VERSION constant (2026-05-v1) gates traveler bookings (requireTravelerConsents) and host actions (requireLocalConsents); consents are immutable, append-only rows capturing IP and User-Agent for evidentiary value.
- GDPR erasure is a request-then-admin-approve workflow that anonymizes the users row in place (overwrites name/email/phone, nulls password) rather than hard-deleting, preserving booking/payment referential integrity; export is self-service and synchronous.
- Secrets (JWT secret, DB, Stripe, social client IDs, Azure, wallet keys) are externalized entirely to environment variables in application.yaml with no committed defaults for the sensitive ones; the seeded dev admin is confined to the dev Spring profile.
- Rate limiting fails open: any Redis/infrastructure error in RateLimitService is swallowed so the app keeps serving (no throttle), and limit breaches return HTTP 400 rather than 429.

Money — Pricing Engine, Payments, Payouts & Invoicing

- Pricing is a pure, dependency-free calculator (PricingCalculator) fed by resolved inputs (PricingEngine + resolvers), verified against golden vectors and a cash-conservation invariant in PricingCalculatorTest — separating rate resolution (DB/config) from arithmetic.
- Every monetary value is BigDecimal NUMERIC(_,2) rounded HALF_UP per line before summing; rates are NUMERIC(5,4) decimals (0.2100 = 21%). No floating point anywhere in the money path.
- The full breakdown is snapshotted onto the Payment row at checkout-prep time (commission/service-fee/VAT/payout columns) so issued invoices and payouts are immutable even if rate rules change later.
- Customer pricing invariant: customerTotal = experienceGross + serviceFee + serviceFeeVat — the service fee is added ON TOP of the (post-discount) experience gross, not carved out of it.
- Commission base is hostGross (which can exceed customerGross when the platform absorbs a promo discount), letting the host still earn on platform-borne discounts while the platform's margin absorbs them.
- Host VAT status (NL_REGISTERED / NL_NOT_REGISTERED / EU_OTHER_REGISTERED / NON_EU) drives both experience-leg VAT and commission-leg VAT treatment (STANDARD / NOT_REGISTERED / REVERSE_CHARGE / OUT_OF_SCOPE).
- Stripe is integrated as hosted Checkout Sessions (no card data touches the backend); the platform is merchant-of-record (INTERMEDIARY) and pays hosts separately via Stripe Connect Express transfers — a separate-charge-and-transfer model, not destination charges.
- Webhook idempotency is enforced by a unique (provider, providerEventId) on payment_webhook_events; signature verification is mandatory in StripeWebhookController.
- Payouts run off a double-entry-ish host ledger (host_ledger_entries) with a hold window (experience end + app.payout.hold-hours, default 72h); earnings are PENDING then AVAILABLE then PAID, with a unique index guaranteeing one EARNING per payment.
- Refund triggers a PROPORTIONAL clawback: the host earning is reversed only by the same fraction the customer is actually refunded (0% refund leaves the host whole), splitting gift-card share back to the card and cash share to Stripe.
- Commission rate is hard-capped by app.platform.max-commission-rate (default 0.50) in RateAdminService to guard against driving host payout negative once commission VAT is added.
- Invoice numbers are gap-free per-year sequences allocated under a SELECT ... FOR UPDATE row lock (InvoiceNumberService), and issuer company/BTW details are snapshotted onto each invoice.

Booking, Availability, Concurrency & Lifecycle

- Seat concurrency is controlled by pessimistic row locks (SELECT FOR UPDATE via AvailabilitySlotRepository.findByIdForUpdate, @Lock PESSIMISTIC_WRITE); there is no @Version optimistic locking and no explicit isolation override.
- Capacity correctness is layered: in-transaction validateSlot check + DB CHECK constraint booked_count <= capacity + partial-unique active-booking indexes (ux_bookings_active_traveler_slot / _guest_slot), with DataIntegrityViolationException translated to a friendly error.
- Bookings start at PENDING_PAYMENT (admin offline path starts at CONFIRMED); the REQUESTED/ACCEPTED host-handshake (acceptBooking/declineBooking) is dormant scaffolding not reachable from live creation flows.
- The seatsBlocked field decouples reserved seats from head count, enabling private whole-slot buyout (seats = capacity, flat privatePrice) to share all capacity-release logic with shared bookings.
- Payment is the source of truth for confirmation; the hold-expiry vs late-payment race is handled bidirectionally — expiry confirms a late-PAID booking, and a too-late payment is refunded by finalizePaidBooking's safety net.
- Four scheduled @Transactional sweepers reconcile lifecycle: pending-payment expiry (60s), underbooked notice (300s), auto-completion 48h post-start (300s), and check-in retention purge (daily), all bounded with findTop100/200 queries.
- Completion requires a two-sided safety checklist (host + traveller) via validateSafetyChecklistCompleted, except auto-completion which intentionally bypasses the gate 48h after start.
- Guaranteed-departure is host-driven, never automatic: UnderbookedSlotService notifies hosts of sub-minimum (default 3) slots and lets them cancel with full refunds (CANCELLED_MINIMUM_NOT_MET) before a deadline.

External Integrations, Adapters & Resilience

- Ports-and-adapters only where a swap is plausible: payments (PaymentCheckoutProvider), payouts (ConnectPayoutProvider), media (MediaStorageProvider), exchange rates (ExchangeRateProvider), email (EmailProviderService), social auth (SocialTokenVerifier). AI, WhatsApp, wallet, and calendar are concrete services with no interface — accepted because there is currently a single vendor each.
- Config-guarded, dormant-by-default integrations: every paid/credentialed adapter exposes isConfigured() (or an empty @Value default) so the app boots and runs in dev/test with zero external credentials. Stripe is the one exception — StripeClientConfig fails fast at startup if the secret key is absent.
- Graceful degradation is bespoke per integration rather than a shared resilience policy: currency serves a stale in-memory cache on provider failure; email falls back to a console logger; WhatsApp falls back to credential-free wa.me click-to-chat links; media falls back to a register-external-URL endpoint; AI moderation fails-open. No resilience4j / spring-retry / circuit breaker is present.
- Stripe webhook integrity + idempotency: signatures are verified with the Stripe SDK (Webhook.constructEvent) and replays are de-duplicated by persisting every event in payment_webhook_events keyed on (provider, provider_event_id). Webhook processing swallows handler exceptions, storing processing_error so a failed event is recorded rather than retried automatically.
- Outbound adapter failures are uniformly wrapped in the domain BadRequestException (HTTP 400) instead of surfacing as 5xx, and best-effort operations (expireCheckout, blob delete) deliberately never throw.

- Social-login extensibility via Spring collection injection: SocialAuthService builds a Map<SocialProvider, SocialTokenVerifier> from all beans, so adding a provider is just adding a bean. APPLE is enumerated but has no verifier, so it returns a graceful 'not available yet' error.
- Notifications are processed asynchronously out-of-band by a DB-polling scheduler (NotificationProcessor every 10s), decoupling the request thread from email/WhatsApp provider latency; unsupported channels (SMS) are marked SKIPPED, not failed.
- Wallet passes are generated in-process (no vendor round-trip at issue time): Apple .pkpass is PKCS#7-signed locally via BouncyCastle; Google Wallet is an RS256 JWT save-link via jjwt — both dormant until certs/keys are configured.

End-to-End Flows, Eventing & Observability

- Transactional outbox for delivery: notifications are persisted as PENDING rows in the notifications table inside the business transaction, then drained asynchronously by a polling NotificationProcessor (fixedDelay 10s). This decouples delivery from request latency and survives provider outages, at the cost of up to ~10s delivery delay.
- Idempotency is pushed entirely to the database. Every emission carries a deterministic dedupeKey with a UNIQUE constraint (notifications.dedupe_key); duplicates are swallowed via existsByDedupeKey plus a DataIntegrityViolationException catch. Schedulers can therefore run as often as they like (at-least-once emission, exactly-once row).
- Eventing is minimal and in-process: only BookingCreatedEvent exists, delivered via @TransactionalEventListener(AFTER_COMMIT) + @Async so notification fan-out runs after the booking commits and off the request thread. All other 'events' are direct synchronous service calls or scheduler polls — there is no message broker.
- Scheduling is built on plain @Scheduled fixedDelay with the default single-threaded scheduler; ~11 jobs share one thread. There is NO ShedLock / distributed lock, so the design implicitly assumes a single app instance. Horizontal scaling would cause duplicate scheduler runs (mostly safe due to dedupe keys, but payouts/expiry rely on row-level pessimistic locks, not job-level locks).
- Concurrency on attendance check-ins is handled with V23 partial unique indexes (uq_attendance_host_per_slot, uq_attendance_guest_per_booking) plus a REQUIRES_NEW nested insert (AttendanceCheckInWriter) so a racing double-submit fails only the nested tx and the caller recovers by re-reading the winning row.
- Geofence trust model is asymmetric: guests are hard-gated (must be inside the error-adjusted geofence with trustworthy GPS accuracy <= 500m), hosts are never rejected for distance (soft check-in with a warning). Check-ins are explicitly operational signal, not refund proof, and are purged after 90 days.
- No-show resolution is human-in-the-loop: either party files within 48h, an admin verifies; HOST no-show triggers a full automated refund via PaymentService and CANCELLED_BY_ADMIN, CUSTOMER no-show is informational. A separate BookingAutoCompletionService auto-completes confirmed bookings 48h after start, but defers any booking with a still-pending report.
- Error/retry posture is fail-soft, not retrying. A failed email/WhatsApp send marks the row FAILED with a failure_reason and is never retried by the worker (no backoff, no max-attempts, no dead-letter); PROCESSING rows that crash mid-flight are also never reclaimed. Operability depends on querying the notifications table by status.
- Observability is thin: Actuator exposes only health and info, logging is SLF4J/Logback to stdout at DEBUG for com.localbuddy with no correlation/trace IDs, no Micrometer metrics registry, and no distributed tracing. Some hot paths emit ad-hoc timing logs (LOGGED_IN_BOOKING_TIMING, GUEST_BOOKING_TIMING).

System Architecture — Stack, Runtime & Modules

LocalBuddy backend architecture · June 2026

Technology stack, runtime, build, deployment & configuration

> A single Spring Boot 4 / Java 21 fat-JAR monolith, built with Maven and deployed manually to an Azure Web App, configured entirely through environment variables with Postgres + Flyway for state and Redis for rate limiting.

Lens summary

LocalBuddy's backend is a **single-deployable Spring Boot 4 monolith** targeting **Java 21**, built with **Maven** (via the committed `mvnw` wrapper) into one executable fat-JAR and shipped to an **Azure Web App** through a manually-triggered GitHub Actions workflow. State lives in **PostgreSQL** (schema managed by **Flyway** + validated by Hibernate); **Redis** backs only rate-limiting. Almost all configuration — including every secret — is supplied through **environment variables** consumed by a single `application.yaml`. The deployment unit is a plain JAR, not a container: there is **no `Dockerfile`, `docker-compose`, `Procfile`, or `web.config`** anywhere in the repo.

The codebase is sizable and feature-oriented: **538 Java source files** across ~40 top-level feature packages under `com.localbuddy` (e.g. `booking`, `payout`, `payment`, `giftcard`, `promo`, `attendance`, `notification`, `wallet`, `whatsapp`, `ai`).

Technology stack inventory

Platform & language

Concern	Choice	Source
Language / level	Java 21 (`<java.version>21</java.version>`)	`pom.xml`
Framework	Spring Boot **4.0.6** (`spring-boot-starter-parent`)	`pom.xml`
Web layer	`spring-boot-starter-webmvc` (servlet MVC, not WebFlux)	`pom.xml`
Build	Maven + Maven Wrapper (`mvnw` , `.mvn/wrapper`)	repo root
Packaging	Executable fat-JAR via `spring-boot-maven-plugin`	`pom.xml` build

> **Note — Spring Boot 4 specifics:** the dependency artifact names are the Boot 4 / Jakarta-era forms (`spring-boot-starter-webmvc` , `spring-boot-starter-flyway` , and the matching `*-test` starters), which differ from the Boot 3 names (`-web` , no dedicated flyway starter). This is a deliberately modern, early-adopter stack.

Core dependencies (from `pom.xml`)

Capability	Artifact	Version
Web MVC	`spring-boot-starter-webmvc`	(Boot-managed)
Persistence	`spring-boot-starter-data-jpa` (Hibernate)	(Boot-managed)
DB migrations	`spring-boot-starter-flyway` + `flyway-database-postgresql`	(Boot-managed)
Driver	`org.postgresql:postgresql` (runtime)	(Boot-managed)
Security	`spring-boot-starter-security`	(Boot-managed)
Validation	`spring-boot-starter-validation`	(Boot-managed)
Ops / health	`spring-boot-starter-actuator`	(Boot-managed)
Cache / rate-limit store	`spring-boot-starter-data-redis`	(Boot-managed)
JWT	`io.jsonwebtoken:jjwt-api/-impl/-jackson`	0.12.6
Payments	`com.stripe:stripe-java`	32.1.0
Email	`com.azure:azure-communication-email`	1.0.23
Media storage	`com.azure:azure-storage-blob`	12.29.1
PDF (invoices/tickets)	`com.openhtmltopdf:openhtmltopdf-pdfbox`	1.0.10
Crypto (Wallet passes)	`org.bouncycastle:bcpkix-jdk18on`	1.78.1
API docs	`org.springdoc:springdoc-openapi-starter-webmvc-ui`	3.0.0
Codegen	`org.projectlombok:lombok` (optional)	(Boot-managed)
Dev loop	`spring-boot-devtools` (runtime, optional)	(Boot-managed)
Kotlin stdlib	`kotlin-stdlib-jdk8` + `kotlin-maven-plugin`	2.3.10

> **Kotlin trade-off / inconsistency:** the build wires a `kotlin-maven-plugin` (with `src/main/java` as a Kotlin sourceDir) and `kotlin-stdlib-jdk8` / `kotlin-test` , but the codebase is Java (Lombok-based). The Kotlin compiler is configured with `<jvmTarget>1.8</jvmTarget>` — far below the project's Java 21 — and the Lombok annotation processor is wired through `maven-compiler-plugin` . This dual-compiler setup is over-configured for a Java-only project and the 1.8 jvmTarget is a latent footgun if Kotlin code is ever added.

Application bootstrap (`LocalbuddyBackendApplication.java`)

A single `@SpringBootApplication` entry point. It additionally enables:

- `@EnableScheduling` — activates the in-process `@Scheduled` pollers.
- `@EnableConfigurationProperties({ StripeProperties, EmailProperties, AzureCommunicationProperties, RateLimitProperties, CheckInProperties })` .

> The file `com/localbuddy/giftcard/GiftCardApplication.java` is **not** a second Spring app — it is a `record GiftCardApplication(UUID, BigDecimal)` (a gift-card reservation value object). There is exactly **one** runtime entry point.

`config/` package

Class	Role
`AsyncConfig`	`@EnableAsync` — enables `@Async` execution (used in 1 place; default `SimpleAsyncTaskExecutor` , no custom pool defined)
`RedisConfig`	Exposes a `StringRedisTemplate` bean over the auto-configured connection factory
`JacksonConfig`	JSON serialization config
`OpenApiConfig`	springdoc `OpenAPI` bean: title "LocalBuddy API" v1.0, `bearerAuth` HTTP/JWT security scheme for Swagger UI
`DevAdminBootstrap`	`@Profile("dev")` `CommandLineRunner` that seeds/repairs an ADMIN user at startup

Cross-cutting config also lives outside this package — notably `auth/SecurityConfig` (stateless, `csrf.disable()`, `SessionCreationPolicy.STATELESS`, CORS from `app.cors.allowed-origins`, a `jwtAuthenticationFilter` added before `UsernamePasswordAuthenticationFilter`, with `/api/public/**` and `/api/auth/**` permitted).

Runtime / process model

This is an **in-process, single-tier** runtime. There is no separate worker/scheduler service — scheduled and async work share the web process.

Scheduled background processors (9 `@Scheduled` pollers)

All use `fixedDelayString` bound to `app.*` keys (all overridable via env):

Scheduler	Default delay	Config key
`notification/NotificationProcessor`	10 s	`app.notifications.processor-delay-ms`
`booking/BookingExpiryService`	60 s	`app.booking.expiry-processor-delay-ms`
`booking/UnderbookedSlotService`	300 s	`app.booking.underbooked-processor-delay-ms`
`noshow/BookingAutoCompletionService`	300 s	`app.booking.auto-complete-processor-delay-ms`
`payout/HostLedgerService`	300 s	`app.payout.ledger-release-delay-ms`
`notification/BookingReminderService`	900 s	`app.notifications.reminder-processor-delay-ms`
`reminder/ReminderService` (×2 methods)	1800 s	`app.reminders.processor-delay-ms`
`payout/PayoutScheduler`	3600 s	`app.payout.processor-delay-ms`
`attendance/AttendanceService` (retention purge)	86400 s	`app.checkin.retention-processor-delay-ms`

> **Scaling trade-off:** because these run in every JVM and there is no leader election or distributed lock visible, **horizontal scale-out would double-fire** payouts, reminders, and expiry jobs. The current design is single-instance-safe only. The tiny Hikari pool (below) reinforces that the system is sized for one small instance.

Datasource / connection pool (`application.yaml`)

- `HikariCP` pool `LocalBuddyHikariPool`: `maximum-pool-size: 5`, `minimum-idle: 1`, `connection-timeout: 30000`, `idle-timeout: 300000`, `max-lifetime: 600000`, `keepalive-time: 300000`.
- JPA: `open-in-view: false`, `ddl-auto: validate`, `show-sql: false`, JDBC `time_zone: UTC`.

Redis

- `spring.data.redis` host/port/password from `REDIS_HOST/PORT/PASSWORD` (defaults `localhost:6379`), `timeout: 2000ms`.
- Used by `ratelimit/RateLimitService` (the only consumer of `StringRedisTemplate`). The **Redis health indicator is disabled** (`management.health.redis.enabled: false`) so Redis being down does not fail the actuator health probe — a deliberate availability choice.

Actuator / health

- Exposed web endpoints: **`health`, `info`** only. `endpoint.health.show-details: never` (no internals leaked). These are the natural Azure App Service health-probe targets.

Persistence & schema lifecycle (Flyway)

- `spring.flyway.enabled: true`, `locations: classpath:db/migration`, `baseline-on-migrate: true`.
- **24 migrations** `V1__baseline_schema.sql` … `V24__payment_gift_card.sql`. `V1` is a **consolidated baseline** (~30 `create table`) authored with the JPA `@Entity` classes as the declared source of truth; `ddl-auto: validate` must pass against it at boot.
- 5 `@ConfigurationProperties` records/classes: `CheckInProperties` (`app.checkin`), `EmailProperties` (`app.email`), `AzureCommunicationProperties` (`app.azure.communication`), `StripeProperties` (`app.payments.stripe`), `RateLimitProperties` (`app.rate-limit`).

> **Memory note (operational caveat, not in repo's code):** the Supabase deployment requires Flyway pinned to the `public` schema (`currentSchema=public` + `SPRING_FLYWAY_SCHEMAS`). That pinning is **not** present in `application.yaml` and must be supplied via env on that target.

Build & release pipeline

Local / CI build

- `./mvnw -B -ntp -DskipTests clean package` → produces `target/localbuddy-backend-0.0.1-SNAPSHOT.jar` (executable Spring Boot fat-JAR; Lombok excluded from the repackaged JAR).

CI/CD — `.github/workflows/azure-webapp.yml`

Aspect	Value
Name	"Deploy to Azure Web App"
Trigger	`workflow_dispatch` only (push-to-`main` trigger is **commented out**)
Runner	`ubuntu-latest`
JDK	Temurin **21**, Maven cache enabled
Build step	`chmod +x ./mvnw && ./mvnw -B -ntp -DskipTests clean package`
Deploy step	`azure/webapps-deploy@v3`
App name	`AZURE_WEBAPP_NAME: localbuddy-backend` (env, hard-coded placeholder)
Artifact	`target/localbuddy-backend-0.0.1-SNAPSHOT.jar`
Auth	`secrets.AZURE_WEBAPP_PUBLISH_PROFILE`

> **Release trade-offs:**
>
> - **Manual-only** (`workflow_dispatch`): no automatic deploy on merge; releases are operator-initiated. The header comment states this is intentional so feature pushes don't fail before Azure is wired up.
> - **Tests are skipped** in the deploy build (`-DskipTests`): the pipeline does **not** gate releases on the test suite, so quality gating depends entirely on developer discipline / a separate (not-present) CI run.
> - **Publish-profile auth** rather than OIDC federated credentials — simpler, but a long-lived secret.
> - No build-number/versioning beyond the static `0.0.1-SNAPSHOT`; the artifact path is hard-coded in the workflow, coupling it to that version string.

Environment & secret configuration strategy

The strategy is **single-file, env-var-overridable, defaults-baked-in**. Every externalized value uses `${ENV_NAME:default}` in `application.yaml`. **Secrets default to empty strings**, so they must be injected as Azure App Settings (or local env) at runtime; the CI workflow injects only the publish profile, never runtime secrets.

Profiles

- `spring.profiles.active: ${SPRING_PROFILES_ACTIVE:dev}` — **default profile is `dev`**.

- **No** `application-<profile>.yaml` files exist. The only profile-gated bean is `DevAdminBootstrap` (`@Profile("dev")`). Production must set `SPRING_PROFILES_ACTIVE` to something other than `dev` to suppress admin seeding (otherwise an admin from `app.dev-admin.*` , default `admin@test.com` / `Password@123` , is created).

Configuration domains (all under `app.*` unless noted)

Domain	Key prefix	Notable env vars / defaults
Core infra	`spring.datasource` / `spring.data.redis`	`DB_URL` , `DB_USERNAME` , `DB_PASSWORD` (no defaults — **required**); `REDIS_HOST/PORT/PASSWORD`
Server	`server.port`	`SERVER_PORT:8080`
JWT / auth	`app.security.jwt`	`JWT_SECRET` (required), `JWT_ACCESS_TOKEN_EXPIRATION_MINUTES:15`
CORS / FE	`app.cors` , `app.frontend`	`CORS_ALLOWED_ORIGINS` , `FRONTEND_BASE_URL` (default `localhost:3000`)
Commission / fees	`app.platform` , `app.fees`	`commission-percentage:20` , `MAX_COMMISSION_RATE:0.50` , `SERVICE_FEE_PERCENTAGE:2.5`
Pricing age bands	`app.pricing.age-band`	adult/teen 1.0, child 0.5, infant 0.0 (env-overridable)
VAT	`app.vat`	`VAT_DEFAULT_COUNTRY:NL` , `VAT_STANDARD_RATE_PERCENTAGE:21` , `VAT_MERCHANT_OF_RECORD:INTERMEDIARY`
Payouts	`app.payout`	`PAYOUT_SCHEDULE:MONTHLY` , hold/min/day-of-month, processor delay
Booking lifecycle	`app.booking`	pending-payment expiry 15m, min-guests-to-confirm 3, underbooked + auto-complete windows
Notifications/reminders	`app.notifications` , `app.reminders`	reminder lead 24h, offsets `2,24,48` , max-age 72h
Geo check-in	`app.checkin`	geofence 300m, max-accuracy 500m, retention 90d
Payments (Stripe)	`app.payments.stripe`	`STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` , success/cancel URLs
Email	`app.email` , `app.azure.communication`	`EMAIL_PROVIDER:console` , `AZURE_COMMUNICATION_CONNECTION_STRING`
Media	`app.media.azure`	blob connection-string, container `experience-photos` , public-base-url
WhatsApp	`app.whatsapp`	access-token, phone-number-id, Graph API base
Wallet passes	`app.wallet.google` / `.apple`	Google issuer/class/SA key; Apple pass-type/team/cert-base64/WWDR
Rate limit	`app.rate-limit.public-api`	enabled, 20 req / 60s window (Redis-backed)
Social login	`app.social.google`	`GOOGLE_CLIENT_ID`
AI	`app.ai.anthropic`	`ANTHROPIC_API_KEY` , model `claude-sonnet-4-6` , max-tokens 1024

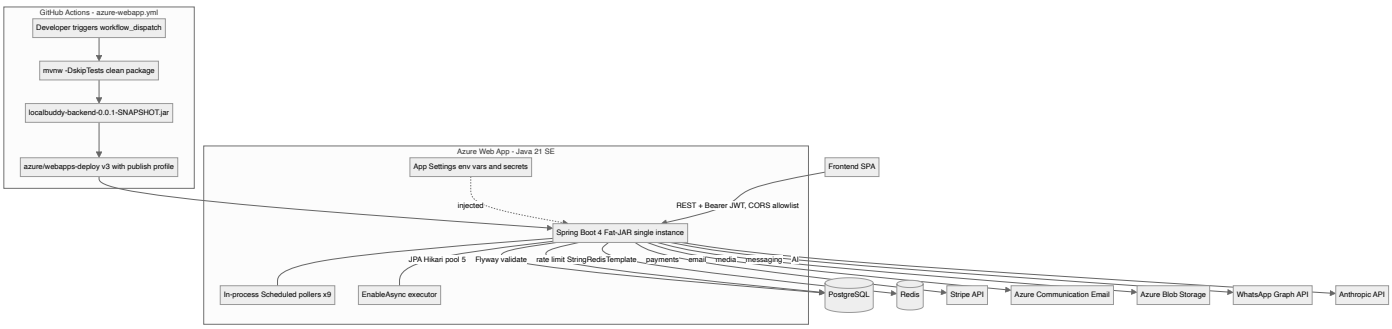
> **"Feature flags":** there are no formal flag toggles beyond `app.rate-limit.public-api.enabled` . Other features are effectively **gated by whether their secret/connection-string is configured** (e.g. empty Stripe/Azure/WhatsApp/Wallet/Anthropic values disable those integrations). `app.email.provider` (`console` default) is a strategy selector rather than a boolean flag.
>
> **Trade-offs:** the all-in-one-file + env-default approach is simple and 12-factor-friendly, but (1) baking real-looking dev defaults (admin creds, `localhost` URLs) into the shipped artifact is risky if `SPRING_PROFILES_ACTIVE` is misconfigured; (2) there is no committed `.env.example` or per-profile file, so the full required-env contract is implicit in the YAML; (3) secrets rely entirely on the Azure App Settings plane (no Key Vault reference is wired in code).

What is NOT in the code (explicit gaps)

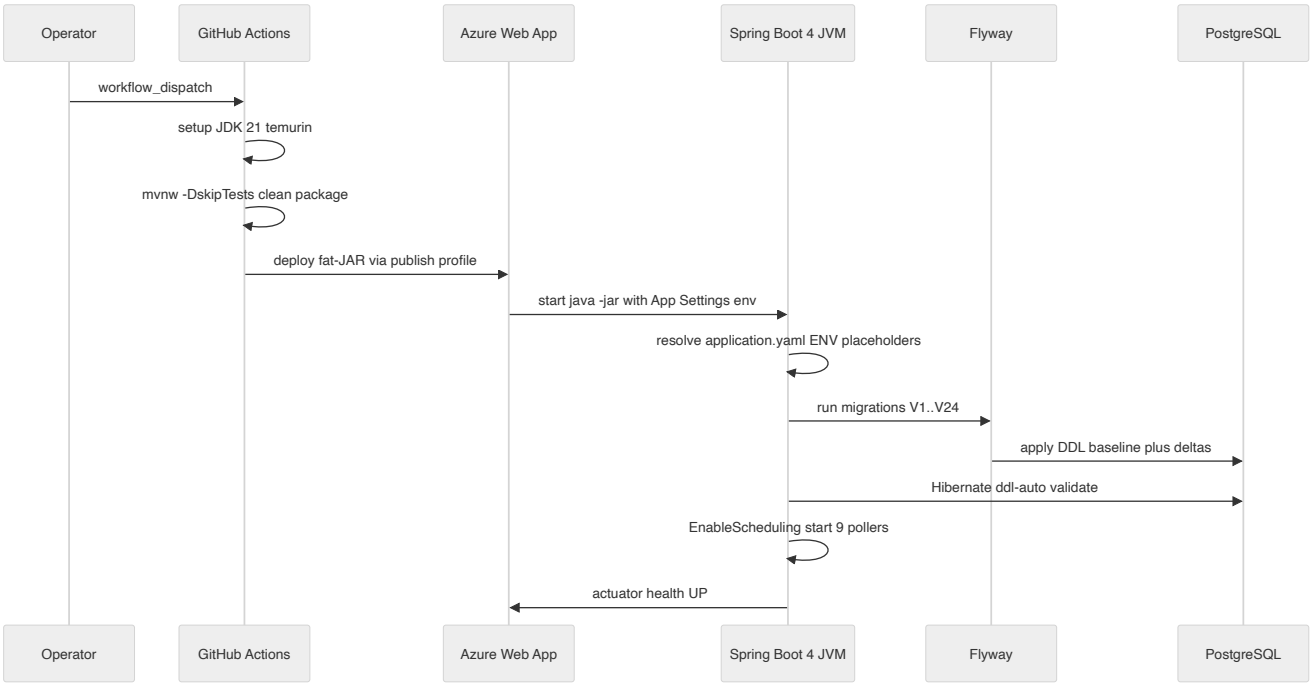
- **No containerization** — no `Dockerfile` / `docker-compose` / `Procfile` / `web.config` . Azure runs the JAR directly (Java SE stack on App Service).
- **No** `application-prod.yaml` or any profile-specific override file.
- **No infrastructure-as-code** (no Bicep/Terraform/ARM templates in repo).
- **No external job queue / message broker** — async + scheduling are in-JVM.
- **No leader-election / distributed-lock** mechanism for the schedulers.
- **No Azure Key Vault** integration in code; secrets come from plain env/App Settings.

Diagrams

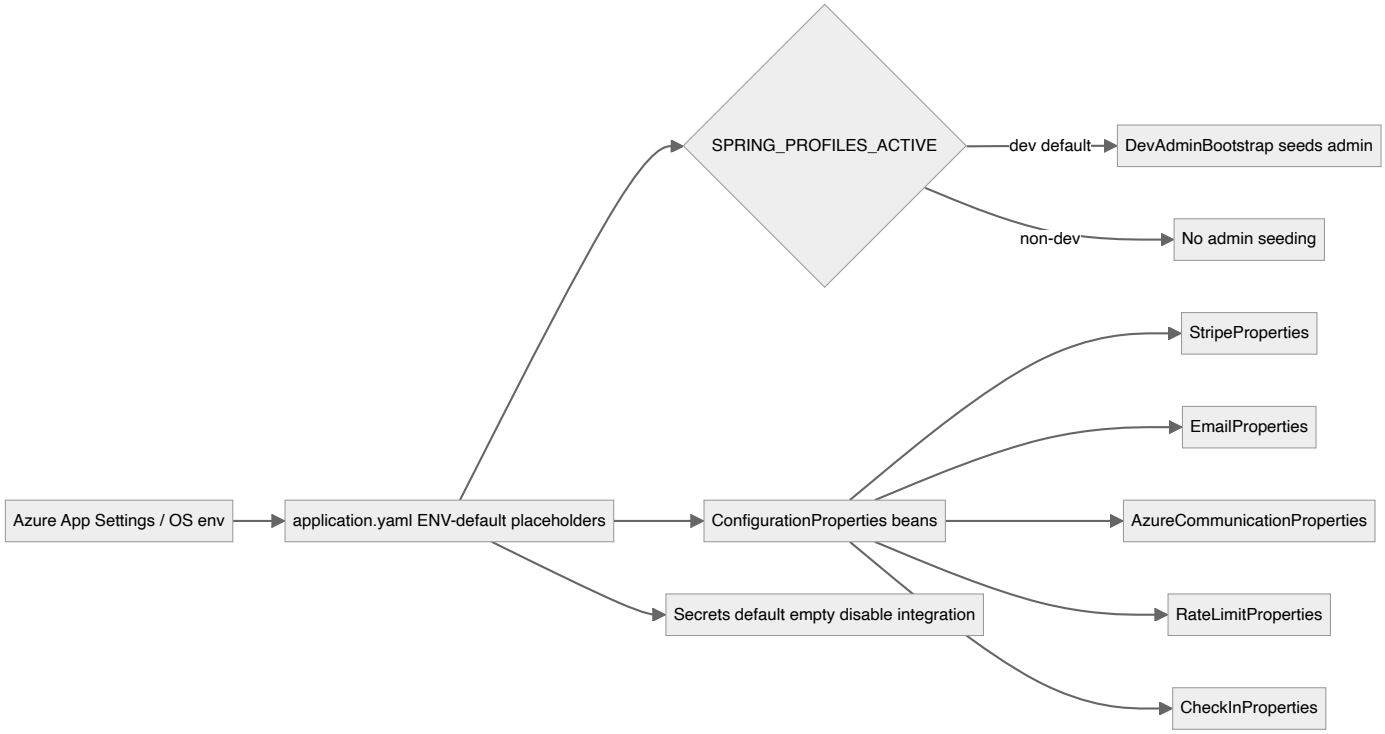
Deployment & runtime topology



Build to boot sequence



Configuration resolution model



Key architectural decisions & trade-offs

- Monolithic fat-JAR over containers: build produces a single executable Spring Boot JAR (localbuddy-backend-0.0.1-SNAPSHOT.jar) deployed directly to Azure Web App via azure/webapps-deploy; no Dockerfile or docker-compose exists in the repo.
- Spring Boot 4.0.6 on the bleeding edge (Jakarta EE, spring-boot-starter-webmvc/-flyway naming), pinning Java 21 as the language level but compiling Kotlin stdlib to an obsolete jvmTarget 1.8 — an inconsistency, though no Kotlin sources are actually present.
- Configuration is 100% environment-variable driven from a single application.yaml with inline ${ENV:default} placeholders; there are NO profile-specific YAML files (application-prod.yaml etc.). The only Spring profile in code is dev (default), which only gates DevAdminBootstrap seeding.
- Schema is owned by JPA entities and enforced at boot via spring.jpa.hibernate.ddl-auto: validate; Flyway (baseline-on-migrate true, V1..V24) applies the DDL. A consolidated V1 baseline was authored from the entities as source of truth.
- Secrets (JWT, Stripe, Azure, WhatsApp, Google/Apple Wallet, Anthropic) are externalized to env vars with empty-string defaults; the CI workflow only injects an Azure publish profile secret, so all runtime secrets must be set as Azure App Settings out-of-band.
- Background work runs in-process via Spring @Scheduled fixedDelay pollers (9 schedulers) plus @EnableAsync; there is no external job queue or worker tier, which couples scheduled processing to every running instance and is unsafe to scale horizontally without leader election.
- HikariCP pool is deliberately tiny (max 5, min-idle 1) — sized for a single small Azure instance and a constrained Postgres connection budget.
- CI/CD is intentionally manual (workflow_dispatch only, push trigger commented out) and deploys a tests-skipped artifact (mvnw -DskipTests clean package), so the pipeline does not gate releases on the test suite.

Modular structure, layering & cross-cutting concerns

> A package-per-feature modular monolith (~44 domain packages, 87 controllers) on a strict controller to service to repository to entity layering, with conventions enforced only by discipline — no module-boundary tooling, URL-based authz, manual DTO mapping, and a thin set of global cross-cutting beans.

Overview

LocalBuddy is a **modular monolith**: a single Spring Boot 3 / Java 21 deployable (`LocalbuddyBackendApplication`) whose code is decomposed into roughly **44 feature packages** directly under `com.localbuddy` . Each package is a self-contained vertical slice that owns its controllers, services, repositories, JPA entities, enums and DTOs. There is **no separate `web` , `service` , `dao` or `domain` top-level layer** — layering is expressed *within* each feature package by class role and suffix convention, not by package. The result is high feature cohesion at the cost of module boundaries that are enforced only by developer discipline.

Counts grounded in the code: 87 `@RestController` classes, 53 `@Entity` classes, 24 Flyway migrations (`V1` – `V24`), 60 classes carrying `@Transactional` , and 9 classes with `@Scheduled` methods.

Module / package map

The packages group naturally into domains even though the code keeps them flat:

Domain area	Packages
Identity & access	`auth` , `user` , `consent`
Catalog & supply	`experience` , `localprofile` , `availability` , `media` , `calendar`
Booking lifecycle	`booking` , `waitlist` , `attendance` , `noshow` , `reminder` , `notification`
Money	`payment` , `payout` , `pricing` , `promo` , `referral` , `giftcard` , `invoice` , `currency` , `deals`
Trust & safety	`safety` , `tripsafety` , `trustsafety` , `review` , `gdpr`
Comms & growth	`messaging` , `whatsapp` , `newsletter` , `announcement` , `wishlist` , `contact` , `ai` , `wallet`
Admin surfaces	`admin` , `adminops`
Cross-cutting / infra	`common` , `config` , `ratelimit` , `audit` (empty placeholder)

A few naming overlaps are worth flagging to an architect: **safety is split across three packages** — `safety` (booking safety checklists + safety reports), `tripsafety` (SOS / emergency contacts / trip-check events) and `trustsafety` (account restrictions + a *second* `SafetyReport` / `CreateSafetyReportRequest` / `SafetyReportType` triad). The duplicated DTO/enum names across `safety` and `trustsafety` are a latent source of confusion and a candidate for consolidation. Similarly `admin` (per-domain admin controllers) and `adminops` (bulk seeding/ops) overlap conceptually.

The `common` and `config` packages are deliberately thin. `common` holds only `HealthController` , `GeoUtil` , and the `exception` subpackage; `config` holds five `@Configuration` classes. There is **no shared base entity** (`@MappedSuperclass` is absent), so id and timestamp fields are repeated per entity, and **no `package-info.java` boundary markers**.

Layering convention

Every feature follows the same four-tier shape, identifiable purely by class-name suffix:

Tier	Marker	Convention
Controller	`*Controller` + `@RestController`	Thin; binds HTTP, validates with `@Valid` , extracts the caller via an `Authentication` parameter, delegates to one service, returns `ResponseEntity<*Response>`
Service	`*Service` + `@Service`	Business logic + transaction boundary (`@Transactional`); orchestrates repositories and other services; throws `BadRequestException` / `ResourceNotFoundException`
Repository	`*Repository extends JpaRepository`	Spring Data; derived + `@Query` methods
Entity	`@Entity` (Lombok-annotated)	Maps to a Flyway-managed table
DTO	request `record` s + `*Response` `record` s	Keep entities off the wire

DTO mapping is hand-written, not generated: response records expose static factory methods (the `public static XResponse from(...)` pattern, 12 found) and there is **no MapStruct dependency**. Entities are never returned directly from controllers. Validation is Jakarta Bean Validation on request DTOs (`spring-boot-starter-validation`), with a custom `@StrongPassword` constraint (`StrongPasswordValidator`) in `auth` .

`spring.jpa.open-in-view: false` is set, which is the correct choice for this style — it forces all lazy-loading to happen inside the service transaction and surfaces `LazyInitializationException` early rather than silently keeping sessions open through view rendering.

```
HTTP -> JwtAuthenticationFilter -> Controller(@Valid, Authentication) -> Service(@Transactional) -> Repository -> Entity/DB
                                                                                                  |-> other module Services / Repositories (in-process)
                                                                                                  |-> publishEvent -> @Async @TransactionalEventListener (notifications)
```

Inter-module dependency direction

Modules integrate **in-process and synchronously** by directly injecting other modules' services *and repositories*. This is the most architecturally significant property of the codebase. The most-injected repositories across module boundaries are:

Repository	Cross-module injections
`user.UserRepository`	26
`booking.BookingRepository`	13
`localprofile.LocalProfileRepository`	11
`experience.ExperienceRepository`	6
`availability.AvailabilitySlotRepository`	5

`BookingService` is the **dependency hub** of the system: it imports types/beans from ~13 other modules — `availability` , `consent` , `experience` , `localprofile` , `messaging` , `notification` , `payment` , `promo` , `referral` , `safety` , `trustsafety` , `user` , `waitlist` . Because nothing prevents a module from reaching directly into another module's persistence layer, the encapsulation boundary is the package only by convention. With **no Spring Modulith, no ArchUnit, and no `package-info` allowed-dependency declarations**, there is no compile-time or test-time guard against cyclic or layering-violating dependencies. This is the principal trade-off of the current design: fast intra-process calls and simple transactions, but erosion-prone module seams.

The one place the system **decouples** modules is booking notifications: `BookingService` publishes a `BookingCreatedEvent` via `ApplicationEventPublisher` , and `BookingNotificationEventListener` consumes it with `@Async` `@TransactionalEventListener(phase = AFTER_COMMIT)` — so notification fan-out runs off the request thread and only after the booking transaction commits. This event-after-commit pattern is used narrowly (booking creation), not as a general integration backbone.

Cross-cutting concerns

Cross-cutting concerns are implemented as a small, centralized set of beans rather than via AOP aspects:

- **Global error handling** — `common.exception.GlobalExceptionHandler` is the single `@RestControllerAdvice` . It maps `BadRequestException` →400, `ResourceNotFoundException` →404, `MethodArgumentNotValidException` / `ConstraintViolationException` →400 (concatenating field errors), `HttpMessageNotReadableException` →400 ("Invalid request body"), `NoResourceFoundException` / `NoHandlerFoundException` →404, and a catch-all `Exception` →500 ("An unexpected error occurred"). All responses share the uniform `ErrorResponse` record `(timestamp, status, error, message, path)` . Note the catch-all **swallows the real exception message** (good for not leaking internals, but pairs with no correlation id, so 500s are hard to trace).
- **Authentication** — `auth.JwtAuthenticationFilter` (a `OncePerRequestFilter` registered before `UsernamePasswordAuthenticationFilter`) parses the Bearer JWT, loads the `User` , and sets a `UsernamePasswordAuthenticationToken` whose principal name is the user UUID and whose single authority is `ROLE_<UserRole>` . JWT signing/parsing is in `JwtService` (jjwt). Sessions are `STATELESS` ;

CSRF/formLogin/httpBasic disabled.

- **Authorization** — coarse, URL-based in `SecurityConfig`: `permitAll` for `/api/health`, actuator health/info, swagger, `/api/auth/**`, `/api/public/**`, `/api/experience-categories/**`, `/api/cities/**`; `hasRole("ADMIN")` for `/api/users/**` and `/api/admin/**`; everything else `authenticated()`. There is **no method security** (`@PreAuthorize` / `@EnableMethodSecurity` absent); finer ownership/role checks live inside services. Handlers obtain the caller via an `Authentication` parameter and `UUID.fromString(authentication.getName())` (122 occurrences) — a repeated idiom with **no shared `@CurrentUser` resolver/helper**, which is a small DRY/abstraction gap.
- **Rate limiting** — `ratelimit.RateLimitService` uses a Redis `StringRedisTemplate` INCR + TTL window keyed `rate-limit:public:<key>`, configured by `RateLimitProperties` (`app.rate-limit.public-api.*`, default 20 req / 60 s). It is invoked explicitly by public controllers (not a filter) and **fails open** on any Redis error. `ClientIpResolver` derives the client key.
- **Async & scheduling** — `config.AsyncConfig` (`@EnableAsync`) and `@EnableScheduling` on the application class. Nine `@Scheduled` services drive the temporal backbone: `BookingExpiryService`, `UnderbookedSlotService`, `BookingAutoCompletionService`, `AttendanceService` (retention), `BookingReminderService`, `NotificationProcessor` (outbox-style dispatch), `HostLedgerService`, `PayoutScheduler`, `ReminderService`. **No custom `TaskExecutor` / `TaskScheduler` bean is defined**, so async and scheduling run on Spring's default pools — a scaling caveat in a single instance.
- **Typed configuration** — `@EnableConfigurationProperties` registers `StripeProperties`, `EmailProperties`, `AzureCommunicationProperties`, `RateLimitProperties`, `CheckInProperties`; the rest of `application.yaml` (extensive `app.*` tree for pricing, VAT, payout, reminders, checkin, wallet, AI, etc.) is read via `@Value` / `@ConfigurationProperties` per module.
- **Serialization** — `JacksonConfig` exposes a single shared `ObjectMapper` (`@ConditionalOnMissingBean`, `findAndRegisterModules()`), explicitly added because some beans inject `ObjectMapper` directly and the web starter doesn't register one.
- **External-integration adapter pattern** — providers are abstracted behind interfaces with swappable implementations selected by config, e.g. `MediaStorageProvider` → `AzureBlobStorageProvider`, `EmailProviderService` → `AzureEmailProviderService` / `ConsoleEmailProviderService` (`app.email.provider`), `PaymentCheckoutProvider` → `StripePaymentCheckoutProvider`, `ConnectPayoutProvider` → `StripeConnectPayoutProvider`, `ExchangeRateProvider` → `FrankfurterExchangeRateProvider`, `SocialTokenVerifier` → `Google/FacebookTokenVerifier`. This is the one place where clean ports-and-adapters discipline is consistently applied.

Audit — gap

The dedicated `audit/` package contains only a `.gitkeep` — **there is no cross-cutting audit framework** (no `@EntityListeners`, no global audit advice). Auditing exists only locally and inconsistently, e.g. `pricing.RateChangeAudit` + `RateChangeAuditRepository` for rate-admin changes. For a marketplace handling money and GDPR, the absence of a uniform audit trail is a notable architectural gap.

API & versioning conventions

- **Audience-by-prefix, not version-by-path.** The API is segmented by caller type using path prefixes: `/api/public/**` (open, rate-limited; e.g. `PublicExperienceController`, `PublicGiftCardController`, `PublicGuestBookingController`, plus the Stripe webhook at `/api/public/payments/webhooks/stripe`), `/api/admin/**` (28 controllers, ROLE_ADMIN), `/api/host/**`, and authenticated `/api/**`. Guest (unauthenticated buyer) flows are mirrored as `Public*` controllers alongside their authenticated counterparts (e.g. `BookingController` vs `PublicGuestBookingController`).
- **No version segment** (`/v1`, header versioning, or media-type versioning) exists. With an OpenAPI spec advertised at version "1.0" (`OpenApiConfig`) but no URL versioning, breaking changes have no migration path — an explicit trade-off/gap.
- OpenAPI/Swagger is wired via `springdoc-openapi-starter-webmvc-ui` with a global `bearerAuth` security scheme.

Persistence & migration conventions

Schema is **owned by Flyway** (`src/main/resources/db/migration`, `V1` – `V24`, `baseline-on-migrate: true`) and Hibernate runs with `ddl-auto: validate`, so the DB is the source of truth and entities must match. Entity relationship modeling is **mixed**: 32 entities use JPA `@ManyToOne(fetch = LAZY)` + `@JoinColumn` associations (e.g. `Booking` maps `traveler_user_id`, `local_profile_id`, `experience_id`, `availability_slot_id`, `promo_code_id`, `referral_code_id`), while other entities store **raw `UUID` FK columns** without a mapped association — a deliberate loose-coupling lever between modules, but one that makes the object graph inconsistent and bypasses referential navigation. Per the memory note, future Supabase migrations must pin Flyway to the `public` schema.

Build, CI & deployment shape

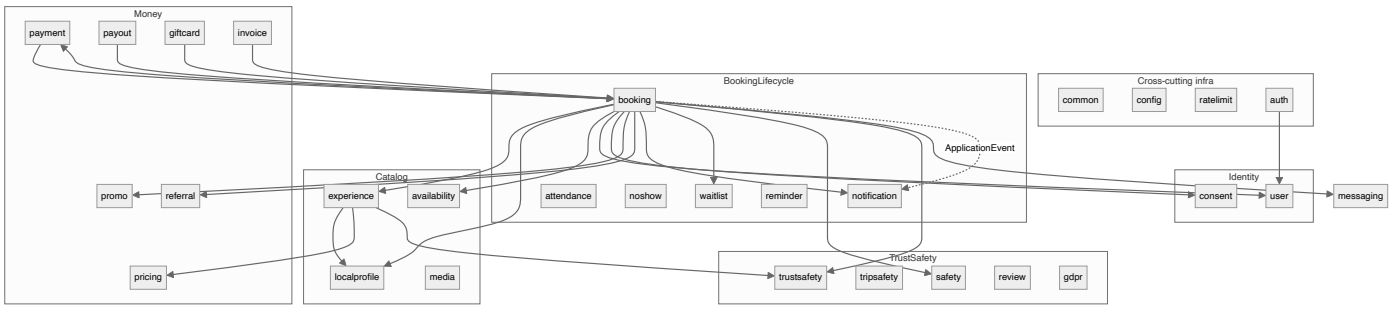
Single Maven module (`spring-boot-starter-parent`), Java 21, Lombok, key starters: `data-jpa`, `data-redis`, `security`, `validation`, `webmvc`, `actuator`, plus `flyway-database-postgresql`, `jjwt-*`, `stripe-java`, `azure-storage-blob`, `azure-communication-email`, `springdoc`. CI is a **single workflow** `.github/workflows/azure-webapp.yml` (`workflow_dispatch` only — push trigger commented out) that builds with `-DskipTests` and deploys the jar to Azure Web App via publish profile. There is **no CI test/lint/arch-check gate**, which reinforces that module boundaries and layering are unverified by automation. Tests exist (`src/test/java/com/localbuddy/{booking,pricing,payout,promo,giftcard,invoice,attendance,finance,...}`) but run only locally.

Key trade-offs summary

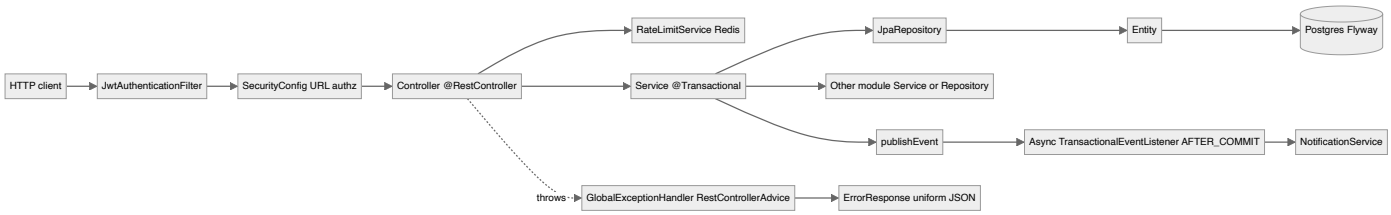
Decision	Benefit	Cost / risk
Modular monolith, package-per-feature	High cohesion, simple deploy, simple transactions	Boundaries unenforced; `BookingService` hub coupling
Direct cross-module service+repo injection	Fast, transactional, no serialization	Encapsulation erosion; refactors ripple widely
No Spring Modulith / ArchUnit / package-info	Zero ceremony	No guard against cyclic/illegal deps
URL-based authz + in-service ownership checks	Simple filter chain	Scattered checks, repeated `authentication.getName()` idiom, IDOR risk (see SUPPORT booking-read note)
Manual DTO mapping (no MapStruct)	Explicit, debuggable	Boilerplate, drift risk
Prefix-based API segmentation, no versioning	Simple routing	No breaking-change migration path
Empty `audit` package	—	No uniform audit trail for money/GDPR
Default async/scheduler pools	Less config	Single-instance scaling ceiling

Diagrams

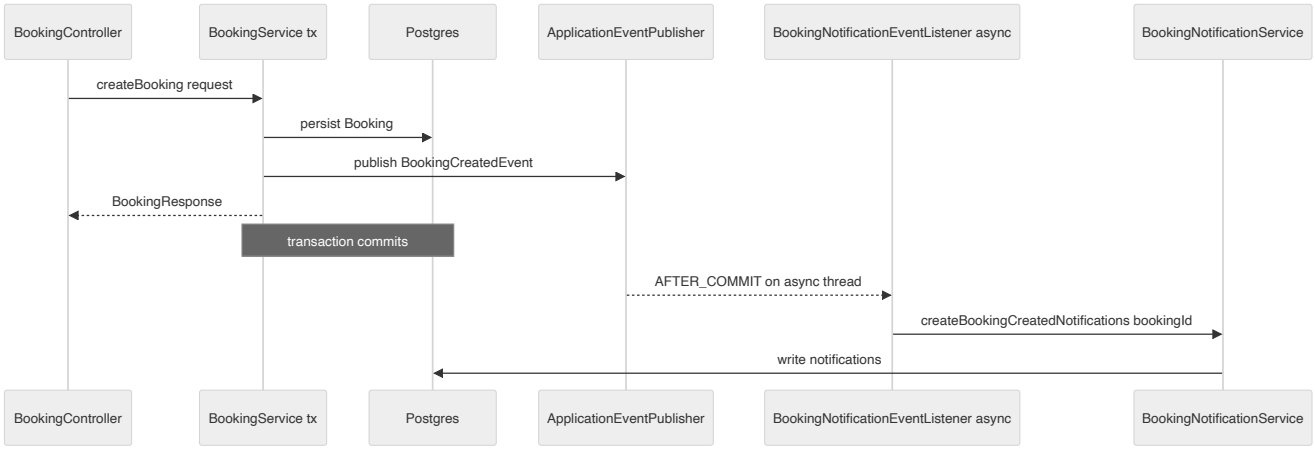
Module map and dependency directions (modular monolith)



Layering and request flow with cross-cutting concerns



Booking-created notification: event-after-commit decoupling



Key architectural decisions & trade-offs

- Modular monolith with package-per-feature (~44 packages under com.localbuddy), each owning its full vertical stack — but module boundaries are convention-only: no Spring Modulith, no ArchUnit, no package-info markers, so any service can inject any other module's repository.
- Strict 4-tier layering: @RestController -> @Service (@Transactional) -> Spring Data JpaRepository -> JPA @Entity. DTOs (request records + *Response records) keep entities off the wire; mapping is hand-written via static Response.from(...) factories (12 found) — no MapStruct.
- API surface is segmented by audience via path prefix rather than versioning: /api/public/** (unauthenticated, rate-limited), /api/admin/** (ROLE_ADMIN), /api/host/**, and authenticated /api/**. No version segment (no /v1) — versioning is an explicit gap.
- Authorization is coarse URL-based matching in SecurityConfig plus per-handler Authentication params (122 occurrences of UUID.fromString(authentication.getName())). No @PreAuthorize / method security; ownership checks live inside services.
- Cross-cutting concerns are centralized as a few beans: one @RestControllerAdvice (GlobalExceptionHandler) emitting a uniform ErrorResponse, a stateless JwtAuthenticationFilter, a Redis-backed RateLimitService that fails open, @EnableAsync + @EnableScheduling, and @ConfigurationProperties for typed config.
- Inter-module integration is in-process and synchronous (direct service/repository calls), with BookingService as the dependency hub (~13 modules). The only decoupled path is booking notifications via a Spring ApplicationEvent consumed by an @Async @TransactionalEventListener AFTER_COMMIT.
- The audit/ package is an empty .gitkeep placeholder — there is no global audit framework; auditing is local and ad hoc (e.g. pricing/RateChangeAudit only).
- Schema is owned by Flyway (24 V-migrations, ddl-auto=validate); entities mix JPA @ManyToOne associations (32 entities) with raw UUID FK columns, and there is no @MappedSuperclass base entity for common id/timestamp fields.

← back to the architecture index

Data Architecture & Domain Model

LocalBuddy backend architecture · June 2026

Data architecture, domain model & persistence

> A code-first JPA domain (53 @Entity classes, ~50 tables) over PostgreSQL with a Flyway-managed schema validated by Hibernate, UUID surrogate keys, string-stored enums, an immutable per-transaction financial snapshot + append-only host ledger, and consent/event tables instead of true soft-delete.

Overview

LocalBuddy uses a **code-first JPA domain model** as the single source of truth for its relational schema. There are **53 `@Entity` classes** under `com.localbuddy.**` mapping to **~50 physical tables** (a handful of entities share helper/sequence tables and there are pure join tables). Persistence is PostgreSQL via the `postgresql` JDBC driver; schema lifecycle is owned by **Flyway** (`flyway-database-postgresql`), and Hibernate runs in **validate-only** mode.

Key persistence configuration (`src/main/resources/application.yaml`):

Setting	Value	Consequence
`spring.jpa.hibernate.ddl-auto`	`validate`	Hibernate never mutates the schema; it asserts the entities match the Flyway-built schema at boot.
`spring.jpa.open-in-view`	`false`	No OSIV; lazy associations must be fetched inside the service/transaction boundary.
`spring.jpa.properties.hibernate.jdbc.time_zone`	`UTC`	All `Instant` / `TIMESTAMPTZ` round-trips normalized to UTC.
`spring.flyway.enabled` / `locations`	`true` / `classpath:db/migration`	Migrations `V1` .. `V24` applied in order.
`spring.flyway.baseline-on-migrate`	`true`	Allows adopting an existing DB at baseline.

Schema authority model — "entity wins"

The migration set is unusual and worth calling out for an architect: **`V1__baseline_schema.sql` is a consolidated 1,205-line baseline** that *reproduces the final merged schema regenerated from the `@Entity` classes* (it references the historical `V1..V32` only for non-entity artifacts like `audit_logs` , CHECK constraints, FK on-delete behavior, and seed data). `V2` .. `V24` are then **forward deltas layered on top of that baseline**.

The baseline's stated acceptance criterion is that `ddl-auto: validate` passes against it, and **where an entity and the old migrations disagreed, the entity wins** (annotated inline as `ENTITY-WINS:`). Concrete examples found in `V1` :

- `reviews.booking_id` was `UNIQUE` in the old `V10` ; the entity maps it as a plain `@ManyToOne` , so uniqueness was **intentionally dropped** to allow bidirectional reviews (traveler→host and host→traveler rows per booking).
- `experience_category_links` , `conversations` , `messages` , and `deals` have **no historical migration at all** — they were created purely from entity mappings.
- `local_profiles.bio` / `profile_photo_url` were created nullable historically but are `NOT NULL` on the entity, so the baseline enforces `NOT NULL` .

> Trade-off: this gives a clean single-file baseline and guarantees entity/schema parity, but it means the migration history is **not a faithful replay** of how production evolved — `V1` is a snapshot, and reviewers must read the entities, not just the SQL, to know intent.

JPA mapping strategy

Conventions are highly uniform across entities:

- **Identity:** `@Id @GeneratedValue(strategy = GenerationType.UUID)` with `@Column(name="id", updatable=false, nullable=false)` , defaulted in SQL to `gen_random_uuid()` (the `pgcrypto` extension is created first in `V1`).
  - **Application-assigned / natural keys (exceptions):** `CancellationRefundPolicy` (UUID, *no* `@GeneratedValue` — seeded with fixed UUIDs), `BookingReferenceSequence` (PK `booking_date LocalDate`), and `InvoiceSequence` (PK `year INTEGER`). These two sequence tables back per-day booking references and per-year invoice numbering respectively.
- **Lombok** `@Getter/@Setter/@NoArgsConstructor` on every entity; no business logic in entities beyond `@PrePersist` / `@PreUpdate` lifecycle hooks that stamp `createdAt` / `updatedAt` (`Instant`) and apply enum/collection defaults defensively.
- **Associations are `FetchType.LAZY` by default** (consistent with `open-in-view=false`). `@ManyToOne` joins use explicit `@JoinColumn` . `LocalProfile` → `User` is a `@OneToOne(LAZY)` on a `UNIQUE` FK.
- **Many-to-many via `@JoinTable` :** `LocalProfile.experienceCities` / `experienceCategories` (`local_profile_experience_cities` , `local_profile_experience_categories`) and `Experience.categories` (`experience_category_links`).
- **JSON columns:** `LocalProfile.experienceLanguages` is `@JdbcTypeCode(SqlTypes.JSON)` mapped to `jsonb` . Other JSONB usage is in audit tables (`audit_logs.old_value_json/new_value_json` , `rate_change_audit.old_value/new_value`).
- **Timestamps:** entity fields are `Instant` → `TIMESTAMPTZ` . The exception is `BookingReferenceSequence` , which uses `LocalDateTime` → plain `TIMESTAMP` (noted explicitly in `V1`).
- **Enums:** universally `@Enumerated(EnumType.STRING)` into `VARCHAR` columns (commonly `length=40`). **No native Postgres enum types are used.**

Enums-as-columns catalogue (selected)

All enums are stored as strings, so adding values is additive but renames require data migrations (see `V14` / `V15` , which rewrote `TRAVELER` → `LOGGED_IN_USER` across `users.role` , `bookings.status` / `booking_source` , and `cancellation_refund_policies.cancelled_by`).

Enum (Java)	Column	Values
`UserRole`	`users.role`	LOGGED_IN_USER, LOCAL, ADMIN, SUPPORT
`UserStatus`	`users.status`	ACTIVE, PENDING_VERIFICATION, SUSPENDED, DELETED
`BookingStatus`	`bookings.status`	REQUESTED, ACCEPTED, PENDING_PAYMENT, CONFIRMED, DECLINED, CANCELLED_BY_LOGGED_IN_USER, CANCELLED_BY_LOCAL, CANCELLED_BY_ADMIN, CANCELLED_MINIMUM_NOT_MET, COMPLETED, EXPIRED
`BookingSource`	`bookings.booking_source`	LOGGED_IN_USER, GUEST_USER, ADMIN
`AttendanceOutcome`	`bookings.attendance_outcome`	NONE, HOST_NO_SHOW, CUSTOMER_NO_SHOW
`GuestShowStatus`	`bookings.guest_show_status`	PENDING, SHOWED, NO_SHOW
`ExperienceStatus`	`experiences.status`	DRAFT, SUBMITTED, APPROVED, REJECTED, PAUSED, BLOCKED
`BookingMode`	`experiences.booking_mode`	SHARED, PRIVATE_ALLOWED, PRIVATE_ONLY
`ExternalListingType`	`experiences.external_listing_type`	NONE, OWN_WEBSITE_SOCIAL, AGGREGATOR_PLATFORM
`PriceInputMode`	`experiences.price_input_mode`	GROSS, NET
`AvailabilityStatus`	`availability_slots.status`	AVAILABLE, BLOCKED, CANCELLED
`PaymentStatus`	`payments.payment_status`	PENDING, PROCESSING, PAID, FAILED, CANCELLED, REFUNDED, PARTIALLY_REFUNDED, REFUND_PENDING, REFUND_FAILED
`PaymentProvider`	`payments.provider` / `payouts.provider`	STRIPE, PAYPAL, ADYEN, MANUAL
`LocalApprovalStatus`	`local_profiles.approval_status`	DRAFT, SUBMITTED, CHANGES_REQUESTED, APPROVED, REJECTED, BLOCKED
`LocalVerificationStatus`	`local_profiles.verification_status`	NOT_STARTED, ID_PENDING, ID_VERIFIED, MANUALLY_APPROVED, REJECTED
`PayoutStatus`	`payouts.status`	PENDING, PROCESSING, PAID, FAILED
`LedgerEntryType` / `LedgerEntryStatus`	`host_ledger_entries.entry_type` / `.status`	EARNING, REVERSAL, ADJUSTMENT / PENDING, AVAILABLE, PAID, REVERSED
`GiftCardStatus`	`gift_cards.status`	PENDING_PAYMENT, ACTIVE, DEPLETED, CANCELLED, EXPIRED
`ReviewDirection` / `ReviewStatus`	`reviews.direction` / `.status`	TRAVELER_TO_HOST, HOST_TO_TRAVELER / VISIBLE, HIDDEN
`PromoDiscountType` / `DiscountBearer`	`promo_codes.discount_type` / `.discount_bearer`	PERCENTAGE, FIXED_AMOUNT / HOST, PLATFORM, SPLIT
`ScopeType`	`commission_rules.scope_type`, `service_fee_rules.scope_type`	PLATFORM, CITY, CATEGORY, HOST, EXPERIENCE
`CommissionVatTreatment`	`payments.commission_vat_treatment`	STANDARD, NOT_REGISTERED, REVERSE_CHARGE, OUT_OF_SCOPE
`InvoiceType` / `RecipientType`	`invoices.invoice_type` / `.recipient_type`	COMMISSION, SERVICE_FEE_RECEIPT, PAYOUT_STATEMENT / HOST, CUSTOMER

Money, multi-currency, VAT and the financial snapshot

Money columns are **`NUMERIC(10,2)`** in transactional tables (`experiences.price_amount`, `bookings.total_amount`, `payments.amount`, `gift_cards.balance`) and **`NUMERIC(12,2)`** in aggregation tables (`payouts.amount`, `host_ledger_entries.amount`, `invoices.*`, `invoice_lines.*`). All **rates are `NUMERIC(5,4)`** (so `0.2100` = 21%) with `CHECK (rate >= 0 AND rate <= 1)`. Every money-bearing table carries its own `currency VARCHAR(3) DEFAULT 'EUR'`.

There is **no FX/conversion engine** — currency is a *stored attribute*, not a converted one; the platform is effectively single-currency (EUR) today, and a real multi-currency rollout would need conversion logic that does not yet exist in code.

The **financial engine** (`V9__financial_engine_foundation.sql`) introduces configurable, time-boxed, scoped rules:

- `commission_rules` and `service_fee_rules`: `scope_type` ∈ {PLATFORM, CITY, CATEGORY, HOST, EXPERIENCE}, with resolution precedence **EXPERIENCE > HOST > CATEGORY > CITY > PLATFORM** and `[effective_from, effective_to)` windowing. Defaults seeded: 20% commission, 2.5% service fee.
- `vat_rates`: by `country` × `category_id` × date, `rate_kind` ∈ {STANDARD, REDUCED, ZERO}. Seeded NL 21% standard + 9% reduced.
- `rate_change_audit`: JSONB before/after of any rate change.
- `company_settings`: single active issuer row (BTW/KvK/IBAN) for invoices.

The resolved breakdown is then **frozen onto the `payments` row** as an immutable snapshot (`commission_rate/amount`, `commission_vat_rate/amount/treatment`, `service_fee_amount/_vat_amount`, `experience_gross/net_amount`, `experience_vat_rate/amount`, `place_of_supply_country`, `host_payout_amount`). This decouples historical financial records from later rate changes — a deliberate and correct accounting choice. Host-side tax/DAC7 fields live on `local_profiles` (`vat_registered`, `vat_number`, `tax_country`, `tax_identification_number`, `business_registration_number`, `date_of_birth`, optional `commission_rate` override).

`payments` enforces money integrity via CHECKs: `amount >= 0`, fees non-negative, and `platform_fee_amount + local_payout_amount <= amount`.

Host earnings ledger & payouts

`host_ledger_entries` (`V10`) is an **append-only ledger** and the source of truth for host money: an `EARNING` is created when a booking is paid, stays `PENDING` until `available_at` (experience end + hold window, default 72h), becomes `AVAILABLE`, then `PAID` once batched into a `Payout` (via `payout_id`). Refunds after payout create negative `REVERSAL` (clawback) rows. A **partial unique index** **`ux_ledger_earning_per_payment`** guarantees at most one EARNING per payment. `Payout` + `PayoutItem` record disbursement so a host portion is never paid twice.
`invoices` / `invoice_lines` / `invoice_sequences` produce sequential per-year VAT documents (COMMISSION, SERVICE_FEE_RECEIPT, PAYOUT_STATEMENT) with snapshotted issuer details for immutability.

Concurrency, idempotency & integrity via partial unique indexes

Rather than application locking, correctness leans on Postgres **partial/filtered unique indexes** and CHECK constraints:

Guard	Index/constraint	Table
One active booking per traveler per slot	`ux_bookings_active_traveler_slot WHERE status IN (REQUESTED,ACCEPTED,PENDING_PAYMENT,CONFIRMED)`	`bookings`
One active booking per guest email per slot	`ux_bookings_active_guest_slot WHERE traveler_user_id IS NULL AND status IN (...)`	`bookings`
One active payment per booking	`ux_payments_booking_active WHERE payment_status IN (PENDING,PROCESSING,PAID)`	`payments`
Webhook idempotency	`UNIQUE(provider, provider_event_id)` + per-id partial uniques on session/intent/charge	`payment_webhook_events`, `payments`
One EARNING per payment	`ux_ledger_earning_per_payment WHERE entry_type='EARNING'`	`host_ledger_entries`
One host check-in per slot / one guest check-in per booking	`uq_attendance_host_per_slot`, `uq_attendance_guest_per_booking`	`attendance_check_ins`
One promo/referral redemption per booking	`uk_promo_redemptions_booking_id`, `uk_referral_redemptions_booking_id` (partial)	redemptions
Capacity integrity	`chk_availability_booked_count_capacity (booked_count <= capacity)`	`availability_slots`
Identity-or-guest	`chk_bookings_traveler_or_guest`, `chk_waitlist_user_or_guest`	`bookings`, `slot_waitlist_entries`

Booking/availability tables also carry **composite indexes** tuned for the host/traveler dashboards (e.g. `idx_bookings_local_profile_status_requested (local_profile_id, status, requested_at DESC)`, `idx_availability_experience_status_start_time`).

Geo columns

Geolocation is intentionally minimal: **no PostGIS**. `experiences.latitude/longitude` (`V20`) and `attendance_check_ins.latitude/longitude/accuracy_meters/distance_meters/within_geofence` (`V23`) are plain `NUMERIC(9,6)` / `DOUBLE PRECISION` columns. Distance and geofence evaluation happen in application code; "near me" sorting and the geofenced check-in gate are therefore not index-accelerated and won't scale to large geo datasets without revisiting this.

Soft-delete, consent & retention

There is **no generic `deleted_at` /soft-delete column or `@SQLDelete` / `@Where` filtering**. Instead:

- **Deletion as state + request:** `UserStatus.DELETED` plus the `data_deletion_requests` GDPR table (`DataDeletionStatus` REQUESTED/PROCESSED/REJECTED), FK `ON DELETE CASCADE` to `users`.
- **Consent capture:** `user_consents` (unique per `user_id, consent_type, version`) and a denormalized consent snapshot directly on `bookings` (`guest_consent_version/_accepted_at/_ip_address/_user_agent`, plus `guest_terms_accepted` / `safety_accepted` / `liability_accepted`) so guest consent is preserved per booking.
- **Forensic/retention tables (append-only):** `audit_logs` (no entity; actor + entity_type/id + JSONB old/new + IP/UA), `rate_change_audit`, and `attendance_check_ins` (server-clock timestamps).
- **FK cleanup is mixed:** strong ownership uses `ON DELETE CASCADE` (e.g. `local_profiles → users`, `experiences → local_profiles`, `availability_slots → experiences`); financial/reference links use the default `RESTRICT` (e.g. `bookings → experiences`, `payments → bookings`) so money rows cannot be orphaned; some optional links use `ON DELETE SET NULL` (`gift_cards.purchaser_user_id`, `payments.gift_card_id`).

> Note: bookings reference users/experiences with `RESTRICT`, so a true hard-delete of a user with bookings is blocked — anonymization (not deletion) is the implied GDPR path, but the actual anonymization routine is in service code, not the schema.

Table catalogue (~50 tables) grouped by domain

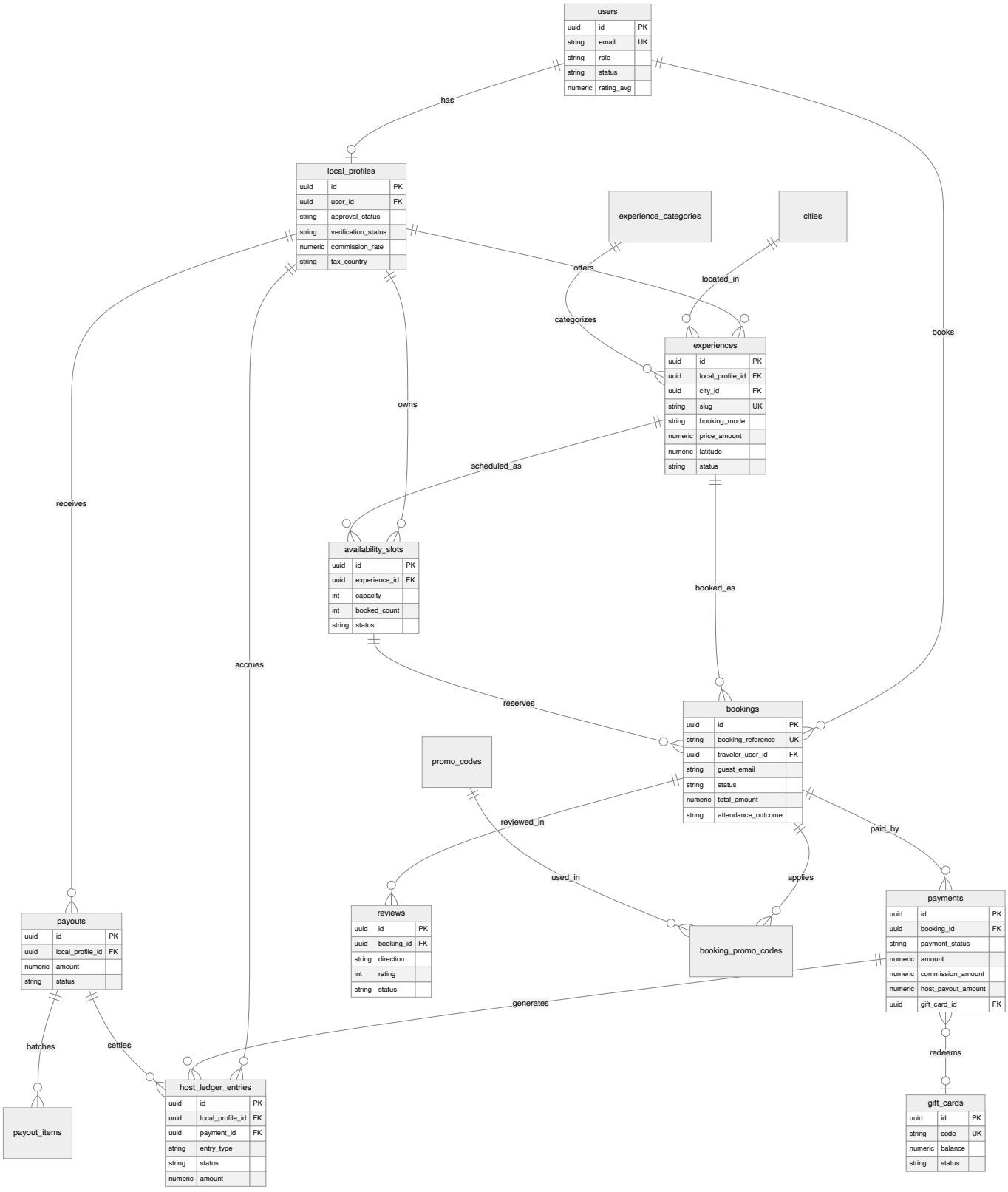
Domain	Tables
Identity & access	`users`, `audit_logs` (no entity)
Reference / lookup	`cities`, `experience_categories`
Host profiles	`local_profiles`, `local_profile_experience_cities`, `local_profile_experience_categories`
Experiences	`experiences`, `experience_category_links`, `experience_photos`
Availability	`availability_slots`, `slot_waitlist_entries`
Bookings	`bookings`, `booking_reference_sequences`, `cancellation_refund_policies`, `booking_promo_codes`, `booking_safety_checklists`
Promotions & referrals	`promo_codes`, `promo_code_redemptions`, `referral_codes`, `referral_redemptions`, `deals`
Payments	`payments`, `payment_webhook_events`
Pricing / tax config	`commission_rules`, `service_fee_rules`, `vat_rates`, `rate_change_audit`, `company_settings`
Payouts & ledger	`payouts`, `payout_items`, `host_ledger_entries`
Invoicing	`invoices`, `invoice_lines`, `invoice_sequences`
Gift cards	`gift_cards`, `gift_card_redemptions`
Reviews	`reviews`
Messaging	`conversations`, `conversation_participants`, `messages`
Notifications & comms	`notifications`, `notification_preferences`, `newsletter_subscriptions`, `announcements`, `host_follows`
Wishlist	`wishlist_items`
Consent / GDPR	`user_consents`, `data_deletion_requests`
Safety (legacy)	`safety_reports`, `booking_safety_checklists`
Trust & safety	`trust_safety_reports`, `user_account_restrictions`
Trip safety	`trip_safety_events`, `emergency_contacts`
No-show & attendance	`no_show_reports`, `attendance_check_ins`

Key files

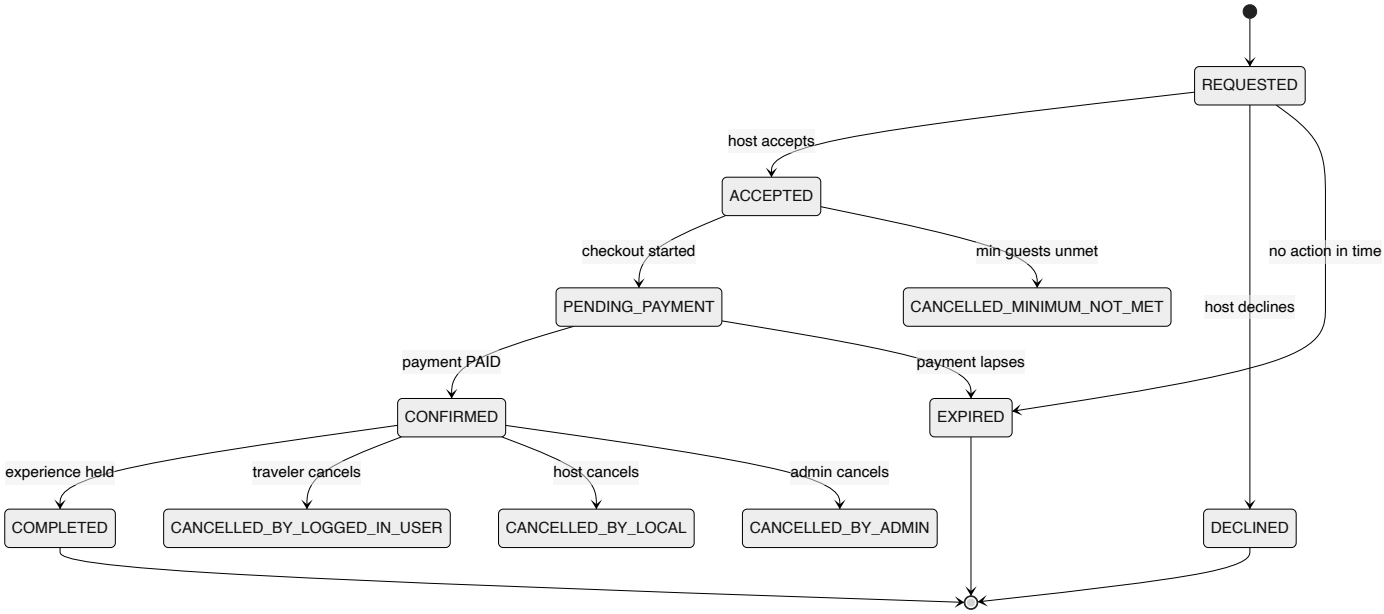
- Entities: `src/main/java/com/localbuddy/**` (53 `@Entity` classes; core: `user/User.java`, `localprofile/LocalProfile.java`, `experience/Experience.java`, `availability/AvailabilitySlot.java`, `booking/Booking.java`, `payment/Payment.java`, `payout/Payout.java`, `payout/HostLedgerEntry.java`, `review/Review.java`, `giftcard/GiftCard.java`).
- Migrations: `src/main/resources/db/migration/V1__baseline_schema.sql` (consolidated baseline) plus `V2` .. `V24` deltas; notably `V9__financial_engine_foundation.sql`, `V10__create_host_ledger.sql`, `V11__create_invoices.sql`, `V23__attendance_checkins.sql`, `V24__payment_gift_card.sql`.
- Config: `src/main/resources/application.yaml` (`jpa.hibernate.ddl-auto=validate`, Flyway), `pom.xml` (postgresql, flyway-database-postgresql).

Diagrams

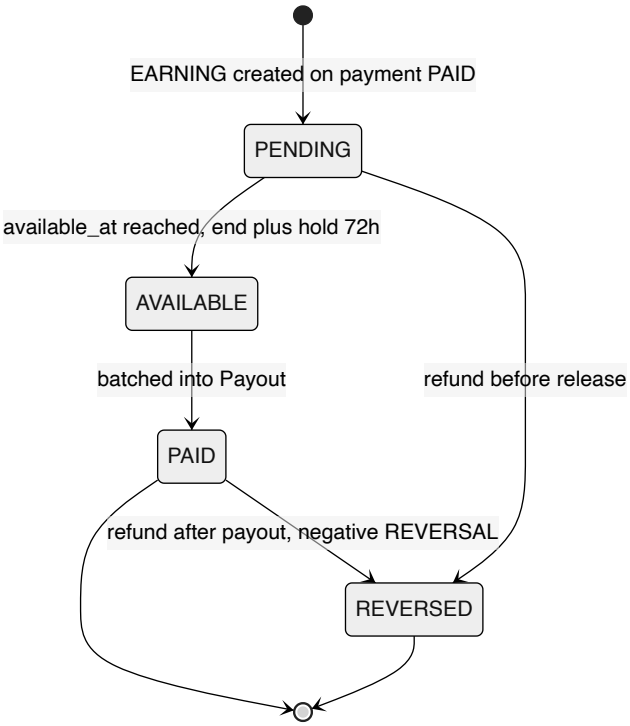
Core domain ER model



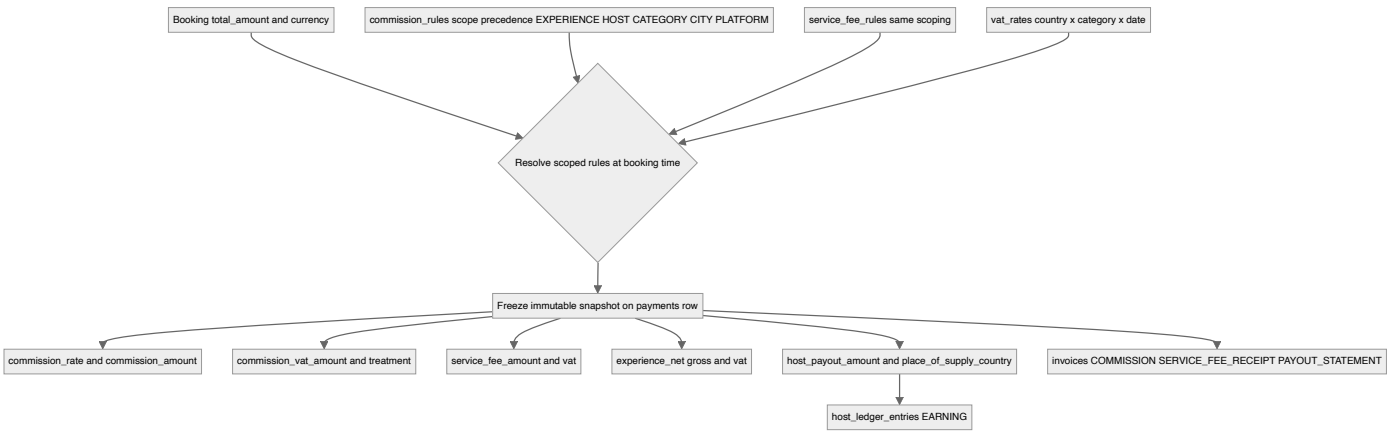
Booking lifecycle (BookingStatus)



Host earnings ledger flow



Financial snapshot resolution at booking



Key architectural decisions & trade-offs

- Entity-wins schema authority: ddl-auto=validate (never update); V1__baseline_schema.sql is a consolidated 1205-line baseline regenerated from the @Entity classes, with V2-V24 as forward deltas. Where old migrations and entities disagreed, the entity is authoritative (annotated ENTITY-WINS in V1), e.g. reviews.booking_id UNIQUE was dropped to allow bidirectional reviews.
- UUID surrogate PKs everywhere via GenerationType.UUID, defaulted server-side to gen_random_uuid() (pgcrypto). Exceptions are application-assigned PKs: CancellationRefundPolicy (UUID, no @GeneratedValue) and natural keys BookingReferenceSequence (LocalDate) and InvoiceSequence (year INT).
- All enums persisted as @Enumerated(EnumType.STRING) into VARCHAR columns (no Postgres enum types), trading storage for safe additive evolution; rename migrations (V14/V15) must rewrite stored string values.
- Money is uniformly NUMERIC(10,2) (NUMERIC(12,2) for payouts/ledger/invoices), rates NUMERIC(5,4); each row carries its own 3-char currency defaulting to EUR. There is no FX/multi-currency conversion engine - currency is a stored attribute, not a converted one.
- Immutable financial snapshot on payments: commission/service-fee/VAT/net/gross/place-of-supply are resolved at booking time from time-boxed scoped rules (commission_rules, service_fee_rules, vat_rates) and frozen on the Payment row, decoupling historical money from later rate changes.
- Append-only host_ledger_entries (EARNING/REVERSAL/ADJUSTMENT x PENDING/AVAILABLE/PAID/REVERSED) is the source of truth for host earnings, with a partial-unique index guaranteeing at most one EARNING per payment; payouts batch AVAILABLE entries and clawbacks are negative REVERSALs.
- Concurrency/duplicate protection via partial unique indexes rather than app locks: one active booking per (traveler|guest, slot), one active payment per booking, one EARNING per payment, one host check-in per slot, one guest check-in per booking, idempotent webhooks via UNIQUE(provider, provider_event_id).
- No generic soft-delete column; deletion is modeled as state (UserStatus.DELETED) plus explicit lifecycle/event tables: data_deletion_requests (GDPR), user_consents and per-booking consent snapshot columns, and append-only audit_logs / rate_change_audit / attendance_check_ins for retention and forensics.
- Dual identity for bookings: a row is anchored to either a logged-in User (traveler_user_id) or denormalized guest_* fields, enforced by chk_bookings_traveler_or_guest; the same OR-guest pattern repeats in waitlist and promo/referral redemptions.
- Geolocation is lightweight: experiences.latitude/longitude and check-in coordinates are plain NUMERIC(9,6) columns with no PostGIS extension; distance is computed in application code, so 'near me' sorting is not index-accelerated.

← back to the architecture index

Security, Identity, Consent & Data Protection

LocalBuddy backend architecture · June 2026

Security, identity, access control, consent & data protection

> Stateless JWT auth with social-token login, a four-role model enforced by a thin URL ruleset plus in-service ownership checks, Redis IP rate limiting on guest endpoints, versioned consent gating for traveler/host actions, and self-service GDPR export plus admin-driven anonymization — with notable gaps around refresh tokens, method-level authZ, auth-endpoint throttling, and host-data erasure.

Scope and module map

This lens covers authentication, authorization, secrets handling, abuse protection, consent capture/enforcement, GDPR data-subject rights, and audit. The relevant code lives under `src/main/java/com/localbuddy/` :

Concern	Package / key classes
Web security wiring	`auth/SecurityConfig`
JWT issuance & parsing	`auth/JwtService` , `auth/JwtAuthenticationFilter`
Password & social auth	`auth/AuthService` , `auth/SocialAuthService` , `auth/GoogleTokenVerifier` , `auth/FacebookTokenVerifier` , `auth/SocialTokenVerifier` , `auth/StrongPasswordValidator`
Role model	`user/UserRole` , `user/UserStatus` , `user/User`
Rate limiting	`ratelimit/RateLimitService` , `ratelimit/ClientIpResolver` , `ratelimit/RateLimitProperties`
Consent	`consent/ConsentService` , `consent/ConsentType` , `consent/UserConsent` , `consent/ConsentController`
GDPR	`gdpr/GdprService` , `gdpr/AccountGdprController` , `gdpr/AdminGdprController` , `gdpr/DataDeletionRequest` , `gdpr/DataDeletionStatus`
Audit	`pricing/RateChangeAudit` (rate changes only); `audit/` package is **empty** (only `.gitkeep`)
Webhook trust	`payment/StripeWebhookController`
Dev bootstrap	`config/DevAdminBootstrap`

> Note: the `audit/` package exists but contains no code. The only persisted audit trail in the codebase is `pricing/RateChangeAudit` (table `rate_change_audit` , capturing commission/service-fee/VAT rate edits with `changed_by_user_id` and JSONB old/new values). There is **no** general security/access audit log (no record of logins, role changes, GDPR processing actor, or admin actions beyond rate edits).

Authentication (authN)

Token-based, stateless. `SecurityConfig` disables CSRF, form login, and HTTP Basic, sets `SessionCreationPolicy.STATELESS` , and inserts `JwtAuthenticationFilter` before `UsernamePasswordAuthenticationFilter` .

JWT design (`JwtService`).

- Algorithm: HMAC-SHA via `Keys.hmacShaKeyFor(jwtSecret.getBytes(UTF_8))` (jjwt 0.12.6). The signing strength depends on the length of `JWT_SECRET` ; a short secret silently weakens to a smaller HS key.
- Claims: `subject` = user UUID, plus custom `email` and `role` claims, `iat` , `exp` .
- Expiry: `app.security.jwt.access-token-expiration-minutes` , default **15 minutes**.
- **No refresh token, no issuer/audience claims, no `jti` /revocation list.** Access tokens are the only credential; there is no logout or server-side invalidation path.

Filter behavior (`JwtAuthenticationFilter`). Reads the `Authorization: Bearer` header, extracts the user id, then **re-loads the `User` from the database on every request** (`userRepository.findById`). This is a deliberate trade-off: it costs a DB hit per call but means a suspended/deleted user loses access immediately (their role/status is always fresh) rather than waiting for token expiry. Crucially, the granted authority is derived from the **DB role**, not the token's `role` claim — so a tampered role claim is irrelevant, and a privilege change takes effect on the next request. On any `JwtException` / `IllegalArgumentException` the context is cleared and the request proceeds anonymously (to be rejected later by the authorize rules).

One subtlety: the filter does **not** check `UserStatus` — a `SUSPENDED` or `DELETED` user still authenticates if they hold a valid unexpired token. Status is only enforced at login time (`AuthService.login` , `SocialAuthService.socialLogin` reject `SUSPENDED` / `DELETED`), so suspension does not revoke an already-issued token until it expires.

Password auth (`AuthService`). Emails are normalized (`trim().toLowerCase()`). Passwords are BCrypt-hashed (`BCryptPasswordEncoder`). `StrongPasswordValidator` enforces 8–100 chars, at least one upper/lower/digit/special, and no whitespace. Login uses a generic "Invalid email or password" message for both unknown-email and bad-password cases (no user enumeration via login). Public signup forbids requesting `ADMIN` / `SUPPORT` roles.

Social login (`SocialAuthService` + verifiers). Pluggable via `SocialTokenVerifier` beans keyed by `SocialProvider` (`GOOGLE` , `FACEBOOK`).

- `GoogleTokenVerifier` calls Google's `tokeninfo` endpoint, optionally checks `aud` against `app.social.google.client-id` **only if that id is configured** (so with no client id set, audience is unverified), and requires `email_verified == true` .
- `FacebookTokenVerifier` calls the Graph `/me` endpoint and requires a non-blank email; it performs **no app-id/audience check at all**.
- A verified social user is matched by email; if none exists, a `LOGGED_IN_USER` is auto-provisioned with `status=ACTIVE` , `emailVerified=true` , and **no password hash**. Account linking is purely email-based, which means a social login can attach to a pre-existing password account that shares the email.

Authorization (authZ)

Authorization is two-layered and intentionally thin at the URL level.

Layer 1 — URL rules (`SecurityConfig.authorizeHttpRequests`).

Matcher	Rule
`/api/health` , `/actuator/health` , `/actuator/info` , swagger/ `/v3/api-docs/**`	permitAll
`/api/auth/**` , `/api/public/**` , `/api/experience-categories/**` , `/api/cities/**`	permitAll
`/api/users/**`	`hasRole("ADMIN")`
`/api/admin/**`	`hasRole("ADMIN")`
anything else	`authenticated()`

Only **ADMIN** is enforced by route. There is no URL rule for `LOCAL` (host) or `SUPPORT` ; every other authenticated endpoint is reachable by any logged-in user at the URL layer.

Layer 2 — in-service checks. Finer authorization is done manually inside services:

- **Host (LOCAL) operations** are gated by requiring an (approved) `LocalProfile` for the caller, e.g. `ExperienceService.createMyExperience` → `getApprovedLocalProfileByUserId(userId)` . This is *resource-ownership* authorization: a non-host simply has no profile and is rejected, so the role check is implicit.
- **Booking reads** (`BookingService.getBookingById`) branch explicitly on `UserRole` : `ADMIN` sees any booking; `LOGGED_IN_USER` only their own; `LOCAL` only bookings tied to their profile; and **`SUPPORT` only bookings for a conversation they participate in** (`conversationRepository.existsBookingConversationParticipant`). The inline comment documents this as an IDOR/GDPR mitigation.

> **Discrepancy with internal notes.** `docs/OPEN_FINDINGS.md` and the auto-memory list the "SUPPORT can read ANY booking" IDOR as still open `[verified 06-26]` . The current `getBookingById` source already scopes SUPPORT to conversation participants — so for this code path the finding appears **fixed**; the docs are stale and should be reconciled.

Key authZ gap: no method security. There is no `@EnableMethodSecurity` and zero `@PreAuthorize` / `@Secured` / `hasRole` usage outside `SecurityConfig` . Authorization correctness therefore depends entirely on (a) every sensitive endpoint living under `/api/admin/**` or `/api/users/**` , and (b) each service remembering to perform its own ownership check. A new controller placed outside those prefixes that forgets

its in-service check is an unguarded-privilege-escalation risk by construction.

Guest (no-account) identity model

Guests transact without an account. Identity is the tuple **booking reference + guest email**:

- `BookingReferenceGenerator` produces `yyyyMMdd` + a 4-digit daily sequence (row-locked `booking_reference_sequence`), i.e. predictable and enumerable references like `202606270042`.
- `BookingService.lookupGuestBooking` requires the reference **and** a matching `guestEmail` (case-insensitive), and rejects anything whose `bookingSource != GUEST_USER`. Because the reference alone is guessable, the email match is the real authenticator — anyone who knows a guest's email and can guess the daily sequence could attempt lookups, throttled only by IP rate limiting.
- All public guest endpoints (`/api/public/guest-bookings`, `/api/public/payments/...`, guest check-in, no-show, contact, newsletter) gate on `RateLimitService.checkPublicApiLimit(...)` keyed by resolved client IP.

Secrets and key handling

- All sensitive values are externalized to env vars in `application.yaml` with **no committed defaults** for the truly secret ones: `JWT_SECRET`, `DB_URL/USERNAME/PASSWORD`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `REDIS_PASSWORD`, Azure connection strings, wallet certs/keys, `ANTHROPIC_API_KEY`.
- The dev admin (`config/DevAdminBootstrap`) is `@Profile("dev")` only and seeds `app.dev-admin.*` (defaulting to `admin@test.com` / `Password@123`). These defaults are dev-only, but the default values are present in `application.yaml`, so a misconfigured production profile (`SPRING_PROFILES_ACTIVE=dev`) would seed a known-credential admin — operationally important.
- CI (`.github/workflows/azure-webapp.yml`) uses only `secrets.AZURE_WEBAPP_PUBLISH_PROFILE`; runtime secrets are expected to be set as Azure App Service settings, not in the repo.
- Actuator is locked down: only `health,info` exposed and `management.endpoint.health.show-details: never`.

CORS

`SecurityConfig.corsConfigurationSource` reads `app.cors.allowed-origins` (default `http://localhost:3000`), splits on commas, allows methods `GET,POST,PUT,PATCH,DELETE,OPTIONS`, **all headers**, and `allowCredentials=true`. Because credentials are allowed, the origin list must be a concrete allow-list (it is — no wildcard), but it **defaults to localhost** and must be set to real production origins before launch. No `exposedHeaders` are configured.

Rate limiting

`RateLimitService.checkPublicApiLimit(key)` implements a fixed-window counter in Redis (`rate-limit:public:<key>`, `INCR` + `EXPIRE` on first hit), governed by `app.rate-limit.public-api.{enabled,maxRequests,windowSeconds}` (defaults: enabled, **20 requests / 60 s**). Notable properties:

- **Client identity is spoofable.** `ClientIpResolver` trusts the first `X-Forwarded-For` value, then `X-Real-IP`, then `getRemoteAddr()` — no trusted-proxy validation, so an attacker can rotate the header to evade limits.
- **Fails open.** Any non-`BadRequestException` runtime error (e.g. Redis down) is swallowed and the request proceeds unthrottled. Spring's Redis health check is also disabled (`management.health.redis.enabled: false`), so an outage is silent.
- **Wrong status code.** A breach throws `BadRequestException` → **HTTP 400**, not 429.
- **Coverage gaps.** Only specific guest controllers call it. The **auth endpoints (`/api/auth/login`, `/signup`, `/social`) are not rate limited at all**, leaving password brute-force / credential-stuffing unthrottled. Public read/enumeration endpoints (experiences, promo validate, gift-card balance) are also not throttled.

Versioned consent enforcement

`ConsentService` holds a single source-of-truth version constant `CURRENT_CONSENT_VERSION = "2026-05-v1"` and two required sets:

Set	Required `ConsentType`s
`REQUIRED_TRAVELER_CONSENTS`	`TERMS_OF_SERVICE`, `PRIVACY_POLICY`
`REQUIRED_LOCAL_CONSENTS`	`TERMS_OF_SERVICE`, `PRIVACY_POLICY`, `COMMUNITY_GUIDELINES`

`ConsentType` also defines `SAFETY_GUIDELINES` and `LIABILITY_ACKNOWLEDGEMENT`, which are not currently in any required set.

- **Capture.** `acceptConsent` is idempotent per `(user, type, version)` (DB unique constraint `uk_user_consents_user_type_version`) and records `acceptedAt`, `ipAddress`, and `userAgent` for evidentiary purposes. Rows are append-only/immutable in practice (versions create new rows; the entity's `@PreUpdate` only touches `updatedAt`).
- **Enforcement.** Gating is checked at the action, not at login: `BookingService` calls `requireTravelerConsents(loggedInUserId)` before a logged-in booking and `requireLocalConsents` before host booking actions; `ExperienceService.createMyExperience` calls `requireLocalConsents`. A missing current-version consent throws `BadRequestException`. Status is computed by filtering the user's consents to those whose `version` equals `CURRENT_CONSENT_VERSION`, so bumping the constant automatically forces re-acceptance.
- **Gaps.** Enforcement is only wired into booking/experience-create paths found in code; other authenticated flows do not re-check consent. Guest (no-account) bookings have no consent capture in this model. There is no consent withdrawal/revocation endpoint.

GDPR — export and erasure

Self-service controller `AccountGdprController` (`/api/account/gdpr`) and admin controller `AdminGdprController` (`/api/admin/gdpr`).

- **Export (`GdprService.exportMyData`).** Synchronous, read-only aggregation of the subject's account, consents, notification preferences, bookings, payments, reviews, and geo check-ins into `GdprExportResponse`. Returns directly over HTTP (no async job, no signed download, no size guard).
- **Erasure workflow.** `requestDeletion` creates a `DataDeletionRequest` (`status=REQUESTED`, one pending request enforced via `existsByUserIdAndStatus`). An admin then calls `process` (anonymize) or `reject`. `processDeletionRequest` is guarded only by the `/api/admin/**` ROLE_ADMIN URL rule.
- **Anonymization (`GdprService.anonymize`).** In-place mutation of the `users` row: `fullName="Deleted User"`, `email="deleted+<id>@deleted.localbuddy.invalid"`, `phone=null`, `passwordHash=null`, `emailVerified/phoneVerified=false`, `status=DELETED`. Bookings, payments, reviews, and check-ins are intentionally retained (referential integrity / financial records) but **de-linked only by virtue of the user row being scrubbed**.
- **Known erasure gaps (real in code):**
  - **Host data is not erased.** `anonymize` touches only `users`; it does not clear `LocalProfile` PII or any bank/tax/payout fields. A host's personal and financial data survives a deletion request (matches the Red finding in `docs/OPEN_FINDINGS.md`).
  - **No actor recorded.** The processed request stores `processedAt` and an optional `adminNote`, but **not which admin processed it** — there is no audit of who performed the erasure.
  - **PII spread.** `guest_email` /booking PII and consent `ip_address` / `user_agent` rows are not addressed by anonymization; check-in rows have their own 90-day retention job (`app.checkin.retention-days`) but other tables do not.

Webhook trust boundary

`StripeWebhookController` (`/api/public/payments/webhooks/stripe`, public per the `/api/public/**` rule) **requires** a configured `STRIPE_WEBHOOK_SECRET` (throws `IllegalStateException` if absent) and verifies the `Stripe-Signature` header via `Webhook.constructEvent` before processing — i.e. the public route is protected by signature verification rather than auth. Invalid signatures are logged at WARN and rejected.

Public profile / sensitive-field exposure

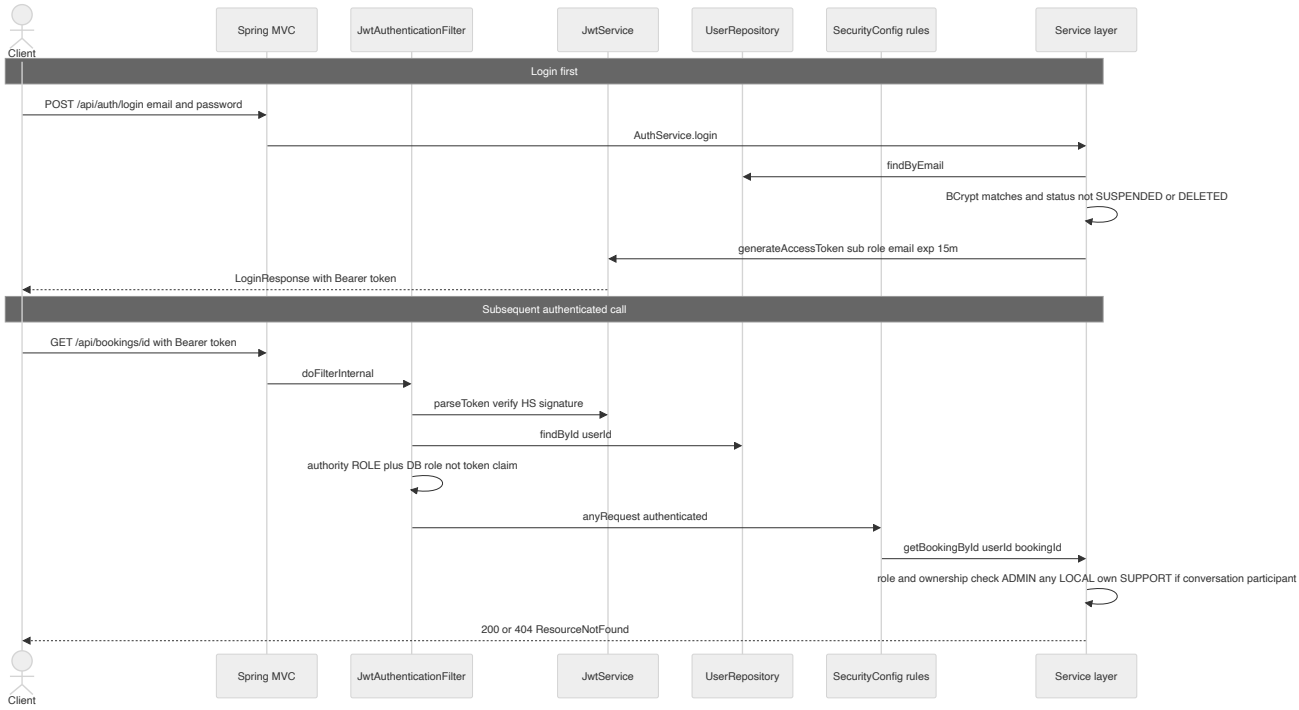
The brief flags "public profile exposing sensitive fields." The `User` entity carries `email`, `phone`, and `passwordHash`. `passwordHash` is never serialized in any response DTO. `CurrentUserResponse` (the `/api/auth/me` payload) does expose `email` and `phone`, but only to the authenticated owner. I did not find a dedicated public user/host profile endpoint that serializes the raw `User` (host data is surfaced via `LocalProfile`); a full audit of every host-facing DTO for over-exposure (e.g. host email/phone on public experience pages) was out of scope here and should be verified field-by-field.

Architectural decisions and trade-offs

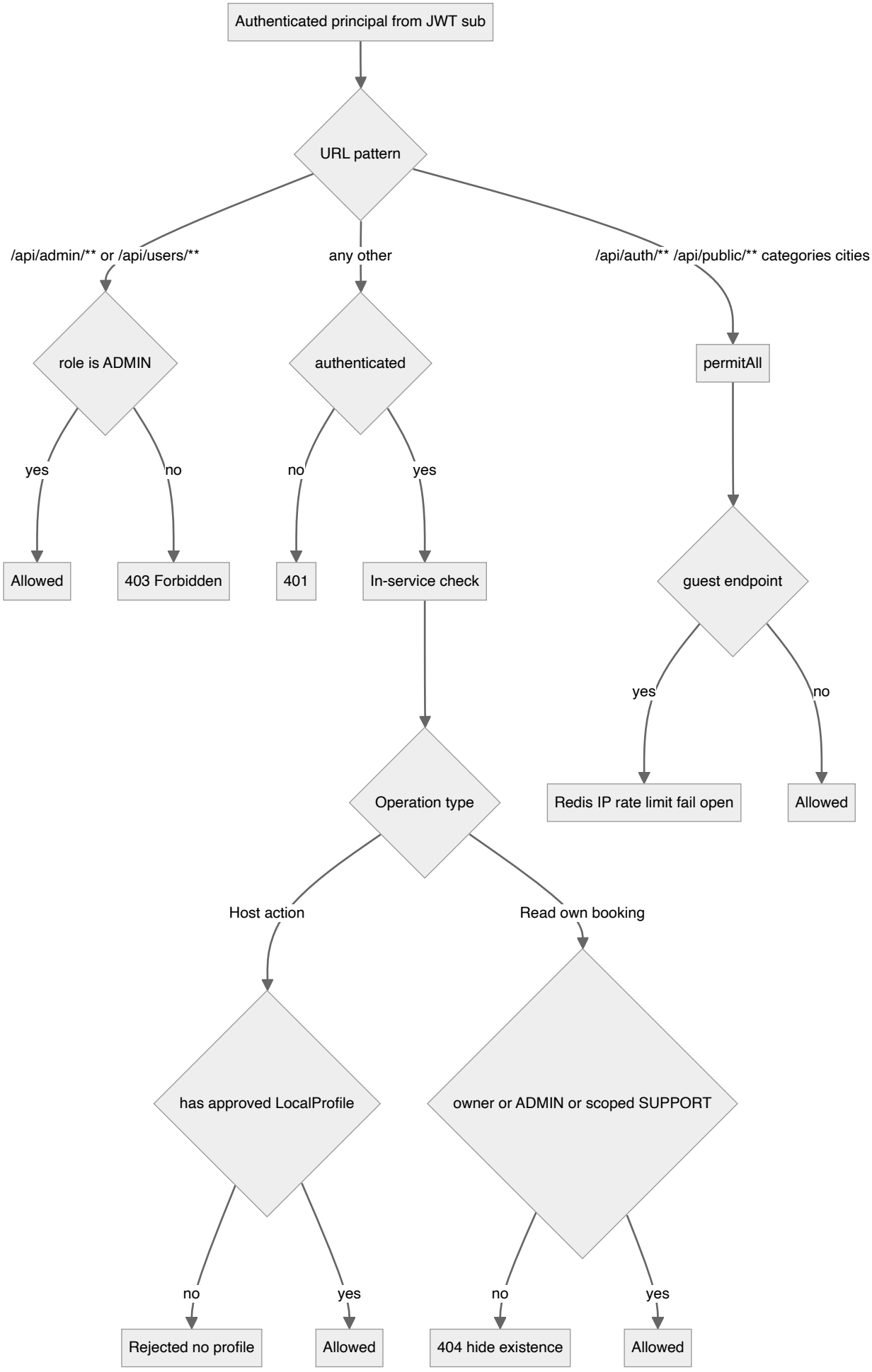
Decision	Trade-off
Stateless JWT, 15-min expiry, **no refresh token / no revocation**	Simple and horizontally scalable; but no logout, suspension doesn't revoke live tokens, and short expiry forces frequent re-login (no refresh UX).
Per-request DB user load in the filter	Always-fresh role/status and instant lockout on next call; costs one DB read per authenticated request.
Authority derived from DB role, not token claim	Token tampering of `role` is moot; but the token still *carries* `role` / `email` (info leak if logged).
Thin URL rules + manual in-service checks; **no method security**	Flexible ownership logic; but correctness hinges on convention, and any endpoint outside `/api/admin` or `/api/users` that omits its check is unguarded. Enabling `@EnableMethodSecurity` would harden this.
Resource-ownership authZ for hosts (require `LocalProfile`)	Avoids hardcoding role-route maps; but the `LOCAL` role itself is unenforced at the URL layer.
Guest identity = enumerable reference + email	Frictionless guest UX; security rests on the email match and IP rate limiting, both weak against a targeted attacker.
Rate limiting fails open + spoofable IP + 400 not 429	Availability over strictness; but Redis outage = no throttling, and `X-Forwarded-For` trust enables evasion.
Auth endpoints unthrottled	No brute-force / credential-stuffing protection on login/social.
Versioned consent via single constant	Trivial to force global re-consent; but enforcement only wired into booking/experience paths, and no withdrawal flow.
In-place anonymization of `users` only	Preserves financial/referential integrity; but leaves host `LocalProfile` + bank/tax data and consent IP/UA, and records no processing actor.
Social audience check optional/absent	Works before client IDs are provisioned; but Google audience is unverified when unset and Facebook app-id is never verified — tokens minted for another app could be accepted.

Diagrams

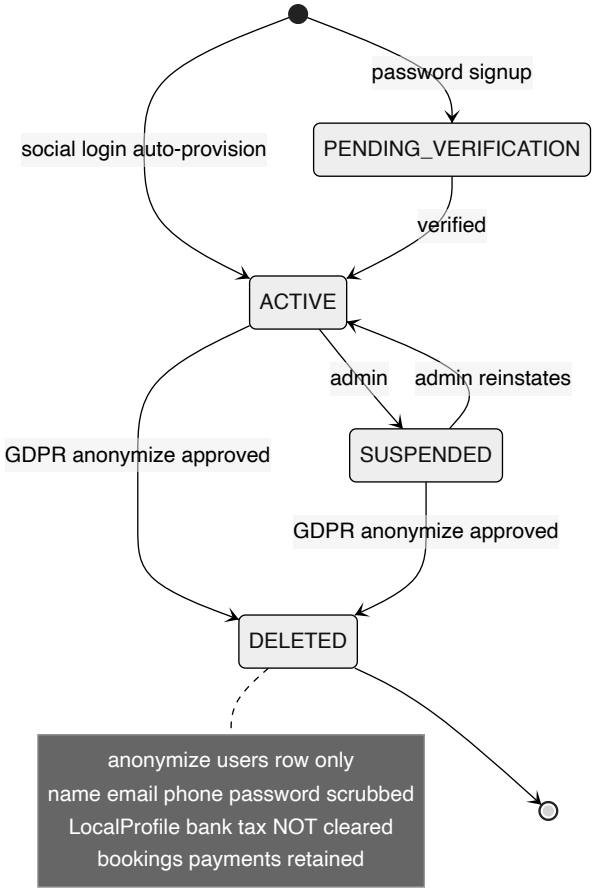
Authentication & request authorization flow



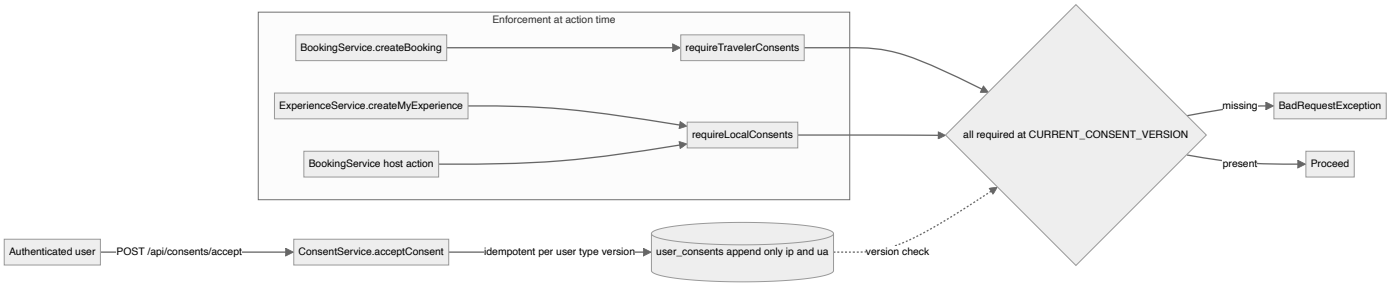
Authorization model — roles, routes and resource ownership



User account & GDPR deletion lifecycle



Consent capture and enforcement



Key architectural decisions & trade-offs

- Stateless, non-revocable access tokens only: HS256 JWT, 15-minute default expiry (app.security.jwt.access-token-expiration-minutes), no refresh token and no server-side session or token blacklist. Logout and revocation are impossible until natural expiry.
- Authorization is split across two layers: coarse URL rules in SecurityConfig (only /api/users/** and /api/admin/** are pinned to ROLE_ADMIN; everything else is merely authenticated), and fine-grained ownership/role checks performed manually inside services (e.g. BookingService.getBookingById, LocalProfile lookups). Method-level security (@EnableMethodSecurity / @PreAuthorize) is NOT enabled anywhere.
- LOCAL (host) authorization is resource-driven, not route-driven: host operations require an approved LocalProfile for the caller rather than a role check on the URL, so the JWT role claim is largely advisory outside the ADMIN routes.
- Guest (no-account) identity = booking reference plus matching guest email, verified on every lookup (BookingService.lookupGuestBooking); guest mutations are protected only by Redis IP rate limiting, not by a secret token.
- Versioned consent enforcement: a single CURRENT_CONSENT_VERSION constant (2026-05-v1) gates traveler bookings (requireTravelerConsents) and host actions (requireLocalConsents); consents are immutable, append-only rows capturing IP and User-Agent for evidentiary value.
- GDPR erasure is a request-then-admin-approve workflow that anonymizes the users row in place (overwrites name/email/phone, nulls password) rather than hard-deleting, preserving booking/payment referential integrity; export is self-service and synchronous.
- Secrets (JWT secret, DB, Stripe, social client IDs, Azure, wallet keys) are externalized entirely to environment variables in application.yaml with no committed defaults for the sensitive ones; the seeded dev admin is confined to the dev Spring profile.
- Rate limiting fails open: any Redis/infrastructure error in RateLimitService is swallowed so the app keeps serving (no throttle), and limit breaches return HTTP 400 rather than 429.

← back to the architecture index

Money — Pricing Engine, Payments, Payouts & Invoicing

LocalBuddy backend architecture · June 2026

Money: pricing engine, payments, payouts, invoicing

> A deterministic BigDecimal pricing engine snapshots a full per-transaction financial breakdown onto each Payment; Stripe hosted checkout + idempotent webhooks confirm bookings; a host earnings ledger with hold windows and proportional clawback drives scheduled Stripe Connect payouts; and an immutable invoice/receipt/statement set closes the VAT loop.

Money: Pricing Engine, Payments, Payouts, Invoicing

This lens covers the end-to-end financial model: how a customer-facing price is decomposed into experience value, platform commission, customer service fee, and per-leg VAT; how Stripe hosted checkout charges the customer and confirms the booking; how host earnings accrue on a ledger and are disbursed via Stripe Connect; and how immutable VAT invoices, receipts, and payout statements are issued. All four concerns live in dedicated packages under `com.localbuddy`: `pricing/`, `payment/`, `payout/`, `invoice/`.

Design Principles (verified in code)

- **Pure calculator + impure resolver split.** `PricingCalculator` (`@Component`) is a dependency-free function from `PricingInput` to `PricingBreakdown`. `PricingEngine` (`@Service`) resolves the inputs (rates, VAT, discounts) from the DB/config and is the only thing that touches the domain. This makes the arithmetic unit-testable in isolation against golden vectors (`PricingCalculatorTest`, 10 adversarial scenarios, all asserting the cash-conservation invariant).
- **BigDecimal everywhere, HALF_UP, 2 decimals.** Money columns are `NUMERIC(10,2)` (payments) / `NUMERIC(12,2)` (ledger/payouts); rates are `NUMERIC(5,4)` decimals (`0.2100` = 21%). `PricingCalculator.round()` applies `setScale(2, HALF_UP)` to every line before summing. No `double` / `float` appears in the money path.
- **Immutable per-transaction snapshot.** The full breakdown is written onto the `payments` row (`commission_*`, `service_fee_*`, `experience_*`, `host_payout_amount`, `place_of_supply_country`) at payment-prepare time via `PricingEngine.applyTo`. Downstream invoices, ledger entries, and payouts read these snapshot columns, so changing a rate rule later never retro-alters an issued document.

1. Pricing Engine

Classes & responsibilities

Class	Type	Responsibility
`PricingEngine`	`@Service`	Resolves all rates/VAT/discounts for a `Booking`, calls the calculator, writes the breakdown onto a `Payment` (`applyTo`) or returns a preview (`priceBooking`).
`PricingCalculator`	`@Component`	Pure arithmetic: gross/net/VAT, commission + commission VAT treatment, service fee + VAT, settlement (host payout, platform keep, VAT remitted).
`PricingInput` / `PricingBreakdown`	`record`	Immutable in/out value objects.
`CommissionResolver`	`@Service`	Resolves commission rate by scope precedence + config default `app.platform.commission-percentage` (20%).
`ServiceFeeResolver`	`@Service`	Resolves service-fee rate, same precedence; config default `app.fees.service-fee-percentage` (2.5%).
`VatService`	`@Service`	Resolves host VAT status, place of supply, experience VAT rate, and fee VAT rate.
`RateAdminService`	`@Service`	Admin CRUD over commission/service-fee/VAT rules with `rate_change_audit` trail and the max-commission cap.

Rate resolution precedence

Both `CommissionResolver` and `ServiceFeeResolver` walk scopes most-specific-first, first match wins. Commission additionally honors per-row override columns:

```
experience.commissionRate column
  -> active EXPERIENCE-scoped rule
  -> host.commissionRate column
  -> active HOST rule
  -> CATEGORY rule -> CITY rule -> PLATFORM rule
  -> config default (app.platform.commission-percentage / 100)
```

`ScopeType` enum encodes the order: `PLATFORM < CITY < CATEGORY < HOST < EXPERIENCE`. Rules are time-boxed (`effective_from` / `effective_to`) and `active`, so time-limited promos are just rules at the right scope (`CommissionRuleRepository.findActive(scope, scopeId, now)`).

VAT model

`VatService.hostVatStatus()` maps `(taxCountry, vatRegistered)` to `HostVatStatus`:

Host situation	`HostVatStatus`	Experience-leg VAT	Commission-leg treatment
Registered, home country (NL)	`NL_REGISTERED`	charged at experience rate	`STANDARD` (VAT charged, host reclaims)
Not registered (KOR/private), in EU	`NL_NOT_REGISTERED`	**none**	`NOT_REGISTERED` (VAT charged, host cannot reclaim)
Registered, other EU country	`EU_OTHER_REGISTERED`	charged at experience rate	`REVERSE_CHARGE` (no VAT, host self-accounts)
Outside EU	`NON_EU`	**none**	`OUT_OF_SCOPE`

`EU_COUNTRIES` is a hard-coded 27-member set. **Place of supply is currently always the platform home country** (`placeOfSupply()` returns `homeCountry` — a documented simplification; per-city ISO mapping is a noted TODO). Experience VAT rate = place-of-supply × category × date (`vat_rates`), falling back to the country `STANDARD` row, then `app.vat.standard-rate-percentage` (21%). Fee VAT always uses the home-country standard rate.

The settlement arithmetic (`PricingCalculator.compute`)

```
 effectiveExpVat = hostCarriesVat ? experienceVatRate : 0      // only NL_REGISTERED / EU_OTHER_REGISTERED carry it
experienceGross = enteredPrice (GROSS mode)                   // or net*(1+vat) in NET mode
experienceNet   = gross / (1 + effectiveExpVat)
experienceVat   = gross - net

hostGross       = hostExperienceGross ?? experienceGross      // higher when platform absorbs a discount
commission      = hostGross * commissionRate
commissionVat   = (NL_*) ? commission * feeVatRate : 0
serviceFee      = experienceGross * serviceFeeRate
serviceFeeVat   = serviceFee * feeVatRate

customerTotal   = experienceGross + serviceFee + serviceFeeVat
hostPayoutCash  = hostGross - commission - commissionVat
platformKeeps   = commission + serviceFee - platformBorneDiscount
platformRemitsVat = commissionVat + serviceFeeVat
```

Customer invariant: `customerTotal = experienceGross + serviceFee + serviceFeeVat`. The service fee is charged *on top of* the post-discount experience gross (confirmed in `PricingEngine` Javadoc and `compute` Step 4).

Cash-conservation invariant: `customerTotal == hostPayoutCash + platformKeeps + platformRemitsVat`. This holds exactly when the host bears the discount. When the platform absorbs a discount (`hostGross > experienceGross`), `hostPayoutCash` is computed on `hostGross` while `customerTotal` is computed on `experienceGross` and `platformKeeps` subtracts `platformBorneDiscount` to compensate — `PricingCalculatorTest` asserts the identity within a **0.02 rounding tolerance** rather than exactly. Architects should treat the invariant as "holds within rounding," not bit-exact.

Discount-bearer handling

`PricingEngine.platformBorneDiscount()` sums, across all `BookingPromoCode`s (multi-code stacking) or the legacy single `promoCode`, the platform's share of each discount via `PromoCode.discountBearer`: `HOST` → 0, `PLATFORM` → full discount, `SPLIT` → `discount × platformSharePercentage / 100`. That sum is added to `experienceGross` to form `hostExperienceGross`, so the host earns on the un-absorbed portion.

Guardrail

`RateAdminService.createCommissionRule` rejects any commission rate above `app.platform.max-commission-rate` (default `0.50`) — explicitly to prevent commission + commission VAT from driving `hostPayoutCash` negative. Rates must be fractions in `[0,1]`; DB `CHECK` constraints (`chk_commission_rate`, `chk_service_fee_rate`, `chk_vat_rate`) enforce the same at the storage layer.

---

2. Payments (Stripe hosted checkout)

Integration model

Stripe is integrated via the official SDK `StripeClient` (`StripeClientConfig` bean, built from `app.payments.stripe.secret-key`). `StripePaymentCheckoutProvider` (implements `PaymentCheckoutProvider`) creates **hosted Checkout Sessions** in `PAYMENT` mode — no card data ever touches the backend (PCI scope minimized). Config record `StripeProperties` binds `secret-key`, `webhook-secret`, `success-url`, `cancel-url`.

Checkout creation flow

`PaymentService.createCheckout` is deliberately **not** `@Transactional` (it must not hold a DB transaction across the Stripe network call). It delegates to:

1. `PaymentTransactionService.preparePaymentForCheckout` (`@Transactional`) — finds/creates a PENDING `Payment`, runs `PricingEngine.applyTo`, optionally reserves a gift card (`GiftCardService.reserveForCheckout`) atomically with the payment, and pre-initializes lazy fields.
2. If a gift card covers the whole amount (or all but a sub-Stripe-minimum remainder), `completeWithoutStripeCharge` marks it PAID and finalizes — bypassing Stripe entirely (`shouldCompleteWithoutStripeCharge` against `app.payments.stripe.minimum-charge`, default €0.50; the platform absorbs the sub-minimum remainder).
3. Otherwise `StripePaymentCheckoutProvider.createCheckout` charges `amount − giftCardAmount` (in cents via `movePointRight(2).longValueExact()`), with `paymentId` / `bookingId` in session metadata and `client_reference_id`.
4. `PaymentTransactionService.attachCheckoutResult` (`@Transactional`) flips the payment to PROCESSING and stores the session id / payment-intent id / checkout URL.

A Spring self-injection (`@Lazy PaymentService self`) routes `completeWithoutStripeCharge` / `releaseGiftCardOnFailure` back through the proxy so their `@Transactional` boundaries actually apply (self-invocation would otherwise bypass them). On any `RuntimeException` the reserved gift card is released.

Status lifecycle (`PaymentStatus`)

`PENDING → PROCESSING → PAID`; terminal/branch states `FAILED`, `CANCELLED`, `REFUNDED`, `PARTIALLY_REFUNDED`, `REFUND_PENDING`, `REFUND_FAILED`. An active payment is blocked if one already exists in `{PENDING, PROCESSING, PAID}` (`existsByBookingIdAndPaymentStatusIn`).

Webhook handling

`StripeWebhookController` (`POST /api/public/payments/webhooks/stripe`) **requires** a configured webhook secret and verifies the `Stripe-Signature` via `Webhook.constructEvent` (rejects on `SignatureVerificationException`). It calls `PaymentService.handleStripeWebhookEvent(Event, rawPayload)`, which:

- **Idempotency:** short-circuits if `(STRIPE, providerEventId)` already exists in `payment_webhook_events`; otherwise persists the raw event with `processed` / `processingError`.
- **`checkout.session.completed`:** if session metadata carries a `giftCardId`, activates the purchased gift card; otherwise `markPaymentPaidFromStripeSession` flips PENDING/PROCESSING → PAID, records the payment-intent id, then finalizes.
- **`checkout.session.expired` / `async_payment_failed`:** `handleFailedOrExpiredCheckoutSession` marks FAILED, releases any gift card, and immediately frees the seat (`BookingExpiryService.releaseBookingSlotAfterFailedPayment`).

`markPaymentPaidFromStripeSession` is **status-guarded** (idempotent against already-PAID; rejects from REFUNDED/CANCELLED) and includes a **late-payment safety net**: if the booking's hold expired and the slot was released before the payment landed, it issues a full refund instead of confirming (`refundFullPayment`).

> **Note (potential gap):** Only the three checkout events are handled. There is no `charge.refunded` / `payment_intent.*` reconciliation, and refund status is set synchronously from the Stripe `Refund.status` response (`succeeded` → REFUNDED/PARTIALLY_REFUNDED, else REFUND_PENDING). Asynchronously-settling refunds are not later reconciled by webhook. A second, older entry point `handleStripeWebhook(StripeWebhookRequest)` (unsigned DTO) also exists but the signed `Event` path is the one wired to the controller.

Post-payment finalization

`finalizePaidBooking` (shared) redeems promo + referral codes, confirms the booking (`PENDING_PAYMENT` / `ACCEPTED` → `CONFIRMED` + sends confirmation), records the host earning on the ledger, records gift-card redemption, and generates invoices — making payment confirmation the single fan-out point into ledger and invoicing.

---

3. Payouts (Stripe Connect + host ledger)

Ledger model (`host_ledger_entries`, V10) — the source of truth

`HostLedgerService` manages an append-style ledger keyed by `localProfileId`. `LedgerEntryType = EARNING | REVERSAL | ADJUSTMENT`; `LedgerEntryStatus = PENDING | AVAILABLE | PAID | REVERSED`.

- **`recordEarning`** posts an EARNING for `payment.hostPayoutAmount`, status `PENDING` if within the hold window else `AVAILABLE`. `availableAt = experience end + app.payout.hold-hours` (default 72h). Idempotency is double-guarded: an app-level `existsByPaymentIdAndEntryType` check **and** a partial unique index `ux_ledger_earning_per_payment` (one EARNING per payment), with `DataIntegrityViolationException` swallowed on concurrent confirm.
- **`releaseHolds`** (`@Scheduled fixedDelay app.payout.ledger-release-delay-ms`, 5 min) flips due PENDING → AVAILABLE in batches of 500.
- **Balances** (`HostLedgerBalances`): onHold (PENDING sum), availableNow (AVAILABLE, unattached), paidOut (PAID sum), lifetime net.

Connect onboarding

`StripeConnectPayoutProvider` creates **Express** accounts (`AccountCreateParams.Type.EXPRESS`) and hosted onboarding links (`AccountLinkCreateParams`, refresh/return URLs from `app.payouts.connect.*` — note these default in code and are **not** present in `application.yaml`). `HostPayoutService.createConnectOnboarding` stores `localProfiles.stripe_connect_account_id`. The host must also have `payouts_enabled = true` for auto-disbursement.

Payout run

`PayoutScheduler` (`@Scheduled fixedDelay app.payout.processor-delay-ms`, 1h) acts only on the cadence's payout day (`app.payout.schedule`: WEEKLY/BIWEEKLY/MONTHLY, `day-of-month` clamped to 1–28). For each host with an available balance ≥ `app.payout.minimum-amount` (€25), it calls `HostPayoutService.createPayoutForHost`, which:

1. Sums the host's AVAILABLE, unattached ledger entries into a `Payout` (PENDING) in `payouts`.
2. **Reserves** those entries by stamping `payout_id` (`attachToPayout`) so they cannot be paid twice.
3. If Connect is configured + host onboarded: `transfer()` (separate-charge-and-transfer model), then PAID + `settlePayout` (entries → PAID) + payout statement. On failure → FAILED with entries **still reserved** (never double-paid); an admin can `mark-paid` (offline) or `retry`.

Clawback (refund-driven)

`PaymentService.handleBookingCancellationPayment` computes the refund via `CancellationRefundPolicyService` (policy by `cancelledBy` actor × hours-before-start, refund% of `booking.totalAmount`). It then **proportionally reverses** the host earning: `refundFraction = refundPercentage / 100`, passed to `HostLedgerService.reverseForPayment(paymentId, fraction, reason)`:

- 0% refund → no reversal (host keeps full earning).

- Still PENDING/AVAILABLE → reduce the earning in place (or mark REVERSED if full).
- Already PAID → post a negative `REVERSAL` entry that nets against future earnings (true clawback).

The refund itself is **split**: gift-card share returns to the card (`giftCardService.returnToCard`); only the cash share goes to Stripe (`refundPayment`), with `REFUND_PENDING → REFUNDED/PARTIALLY_REFUNDED` or `REFUND_FAILED` .

> **Observation:** The V2 `payout_items` table (with `ux_payout_items_payment` uniqueness) predates the V10 ledger and is **not written** by the current `HostPayoutService` — disbursement is tracked entirely via `payout_id` on ledger entries. `payout_items` appears to be legacy/dead schema.

4. Invoicing

`InvoiceService` issues three immutable document types (`InvoiceType`):

Type	Recipient	Trigger	Content
`COMMISSION`	HOST	`generateForConfirmedPayment`	Platform commission + commission VAT, with a `vatNote` per `commission_vat_treatment` (reverse-charge / KOR / out-of-scope).
`SERVICE_FEE_RECEIPT`	CUSTOMER	`generateForConfirmedPayment`	Service fee + service-fee VAT.
`PAYOUT_STATEMENT`	HOST	`generatePayoutStatement` (on payout settle)	Lines per settled ledger entry; remittance only (VAT detail is on commission invoices).

- **Immutability:** issuer legal/BTW details are snapshotted from the single active `company_settings` row (`legalName` , `vatNumber` , address, country) onto each `Invoice` at issue time.
- **Numbering:** `InvoiceNumberService.next(prefix, year)` allocates gap-free sequential numbers per year under `SELECT ... FOR UPDATE` (`invoice_sequence`), formatted `PREFIX-YYYY-0001` .
- **Idempotency:** guarded by `existsByBookingIdAndInvoiceType` / `existsByPayoutIdAndInvoiceType` .
- **Access control:** `renderPdf` enforces that a non-admin requester is either the invoice's host or the booking's logged-in user (`InvoicePdfService` renders the PDF). Notifications fire via `NotificationType.INVOICE_ISSUED` .

Configuration keys (money path)

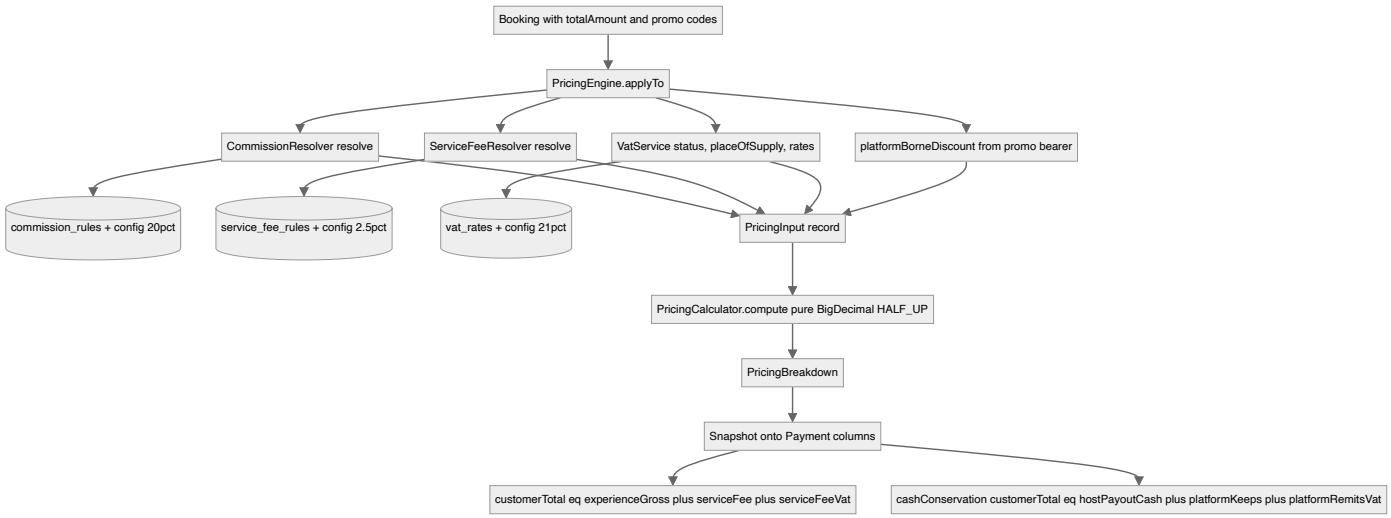
Key	Default	Effect
`app.platform.commission-percentage`	20	Fallback commission when no PLATFORM rule exists.
`app.platform.max-commission-rate`	0.50	Hard cap admins can assign.
`app.fees.service-fee-percentage`	2.5	Fallback service fee.
`app.vat.default-country` / `standard-rate-percentage`	NL / 21	Home country + fallback standard VAT.
`app.vat.merchant-of-record`	INTERMEDIARY	MoR posture (INTERMEDIARY | DEEMED_SUPPLIER) — config only; not yet branching logic in the calculator.
`app.payout.schedule` / `hold-hours` / `minimum-amount` / `day-of-month`	MONTHLY / 72 / 25 / 1	Payout cadence, ledger hold, threshold.
`app.payments.stripe.{secret-key,webhook-secret,success-url,cancel-url}`	—	Stripe hosted checkout + webhook.
`app.payments.stripe.minimum-charge`	0.50	Sub-minimum cash remainder the platform absorbs.
`app.payouts.connect.{refresh,return}-url`	localhost defaults (code only)	Connect onboarding redirects (not in `application.yaml`).

Trade-offs & risks for an architect

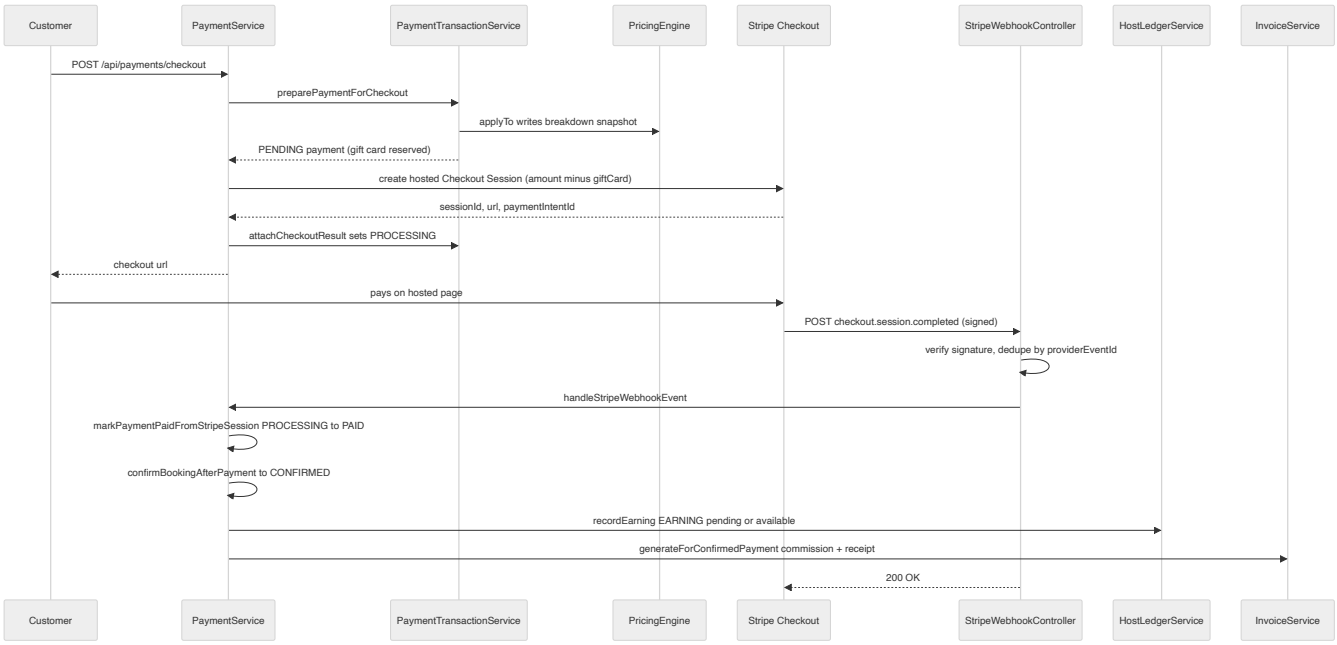
- **Place of supply hard-coded to home country** — incorrect for cross-border experiences; a documented TODO.
- **Refund reconciliation is synchronous-only** — no `charge.refunded` webhook; async refund settlement and disputes/chargebacks are not reconciled back into `PaymentStatus` .
- **Cash-conservation holds only within rounding** under platform-borne discounts (0.02 test tolerance), because `customerTotal` and `hostPayoutCash` use different gross bases.
- **Dead `payout_items` schema** coexists with the live ledger — a cleanup/clarity risk.
- **MoR config (`merchant-of-record`) is inert** — present in config but not yet driving deemed-supplier VAT logic.

Diagrams

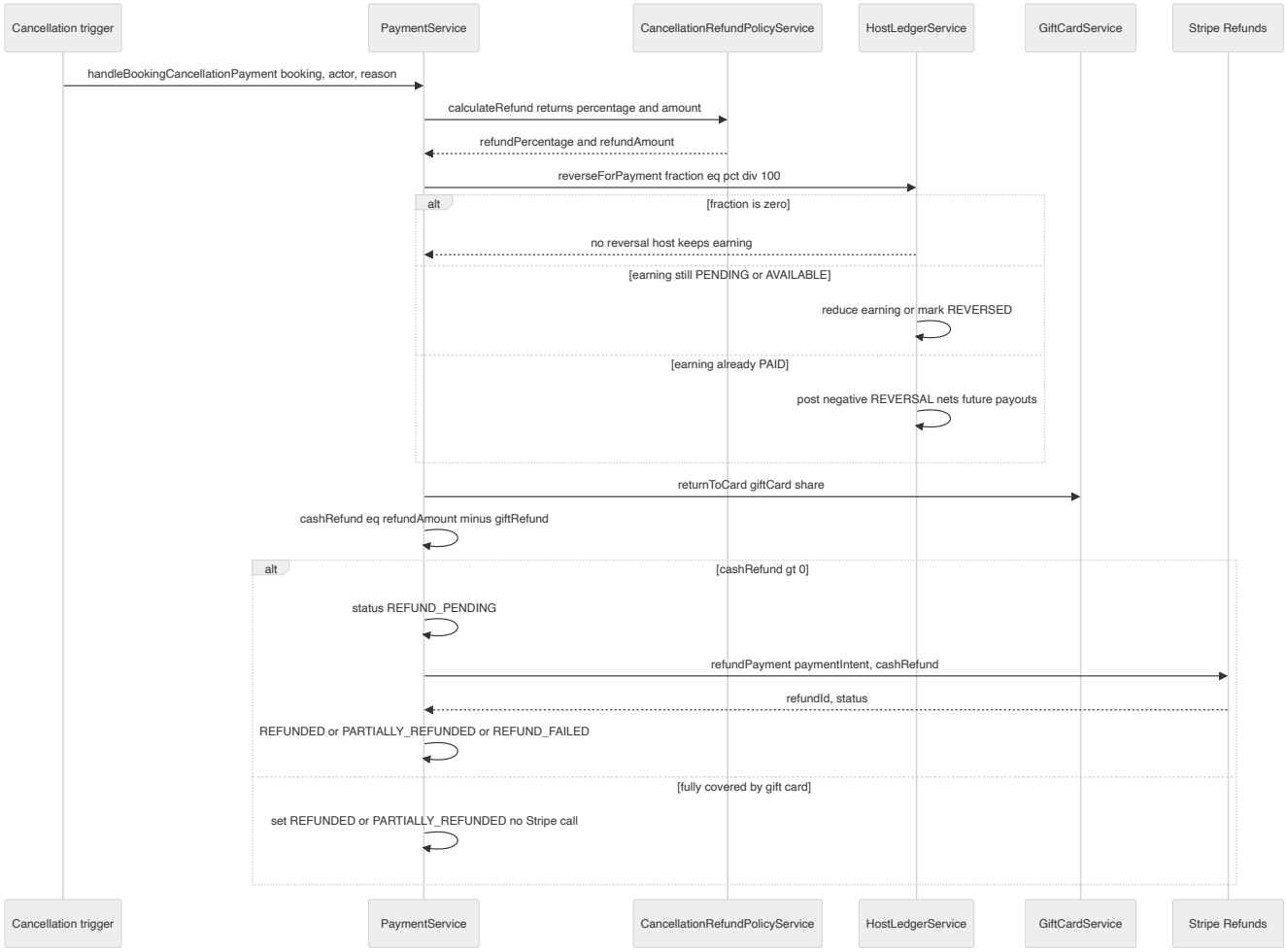
Pricing engine flow and components



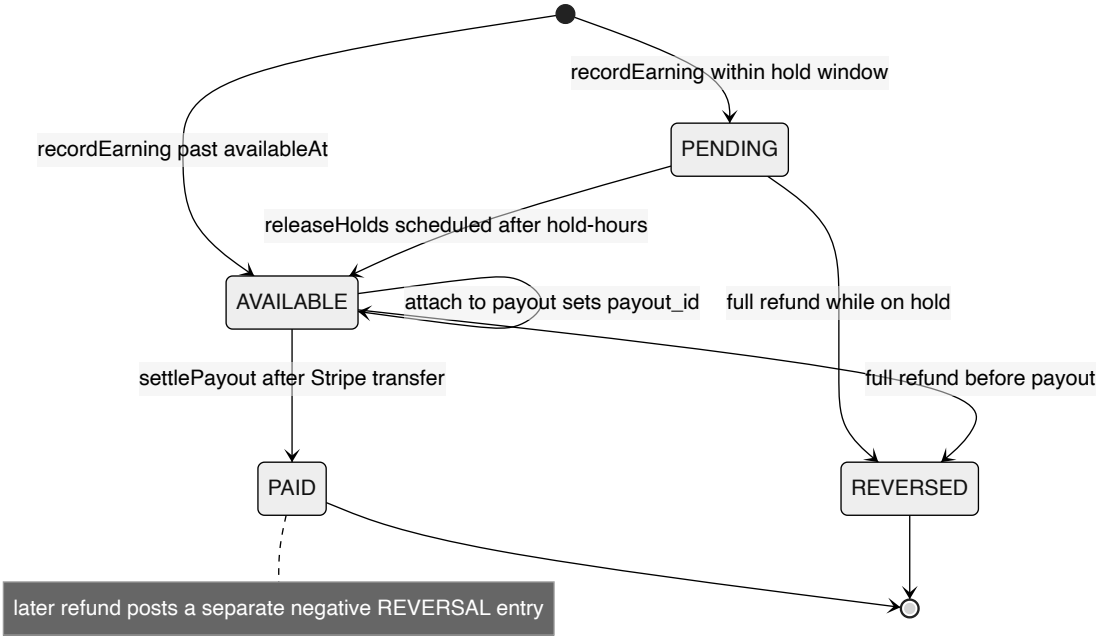
Booking to checkout to webhook to confirm



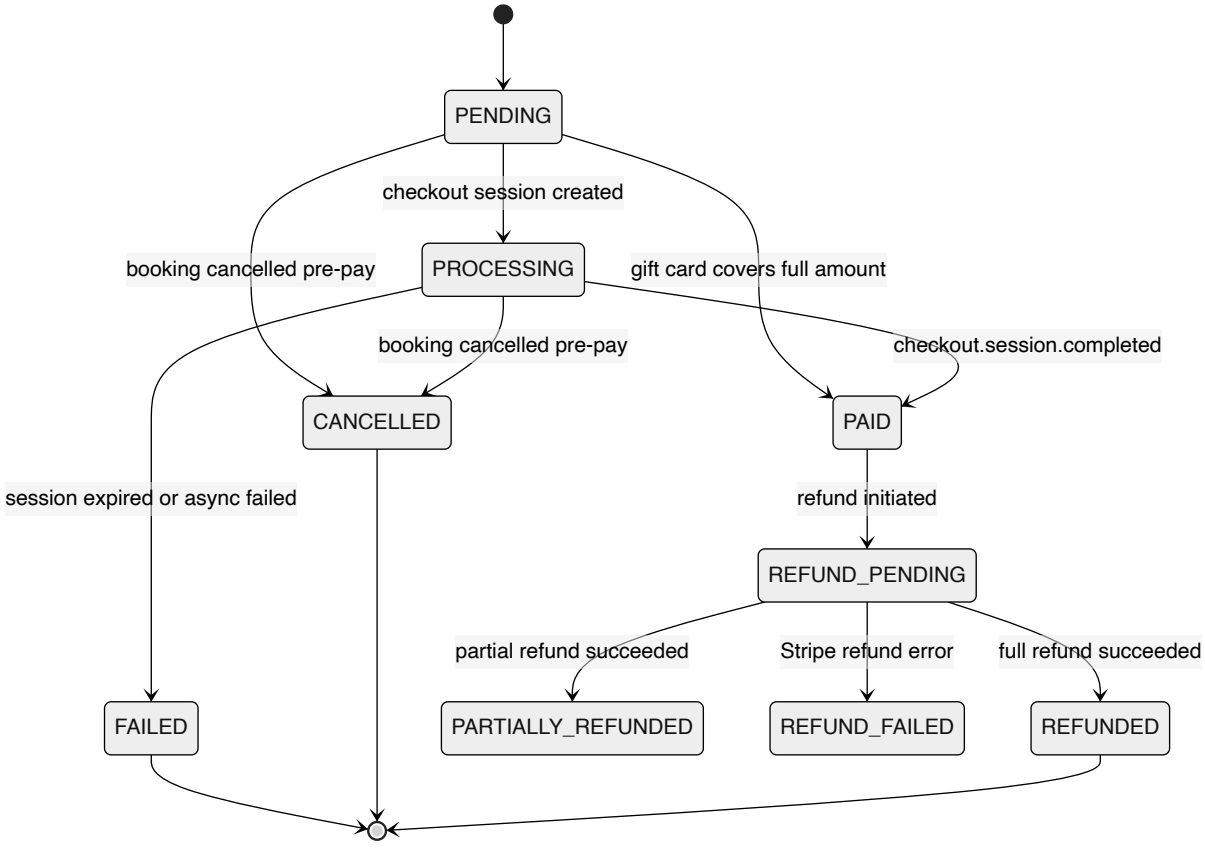
Refund and proportional clawback



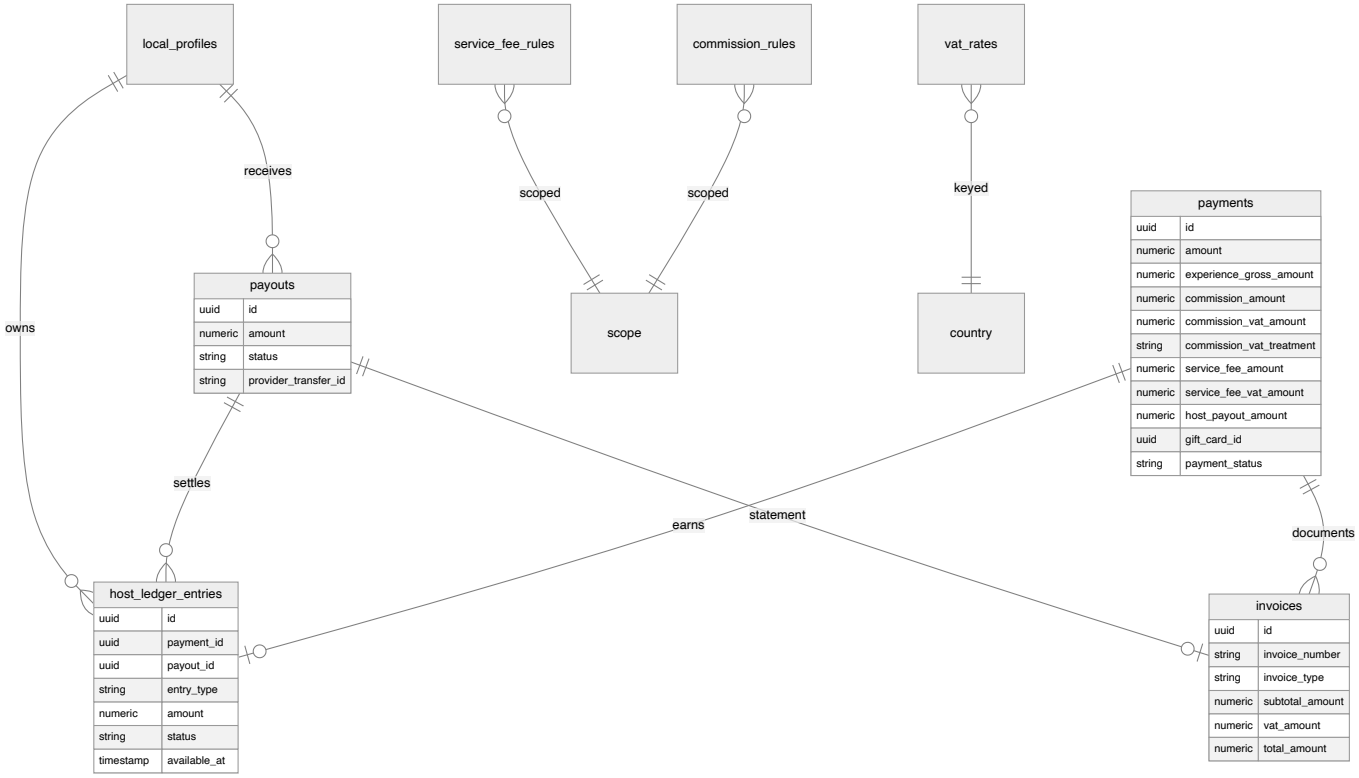
Host ledger entry lifecycle



Payment status lifecycle



Money data model



Key architectural decisions & trade-offs

- Pricing is a pure, dependency-free calculator (PricingCalculator) fed by resolved inputs (PricingEngine + resolvers), verified against golden vectors and a cash-conservation invariant in PricingCalculatorTest — separating rate resolution (DB/config) from arithmetic.
- Every monetary value is BigDecimal NUMERIC(_,2) rounded HALF_UP per line before summing; rates are NUMERIC(5,4) decimals (0.2100 = 21%). No floating point anywhere in the money path.
- The full breakdown is snapshotted onto the Payment row at checkout-prep time (commission/service-fee/VAT/payout columns) so issued invoices and payouts are immutable even if rate rules change later.
- Customer pricing invariant: customerTotal = experienceGross + serviceFee + serviceFeeVat — the service fee is added ON TOP of the (post-discount) experience gross, not carved out of it.
- Commission base is hostGross (which can exceed customerGross when the platform absorbs a promo discount), letting the host still earn on platform-borne discounts while the platform's margin absorbs them.
- Host VAT status (NL_REGISTERED / NL_NOT_REGISTERED / EU_OTHER_REGISTERED / NON_EU) drives both experience-leg VAT and commission-leg VAT treatment (STANDARD / NOT_REGISTERED / REVERSE_CHARGE / OUT_OF_SCOPE).
- Stripe is integrated as hosted Checkout Sessions (no card data touches the backend); the platform is merchant-of-record (INTERMEDIARY) and pays hosts separately via Stripe Connect Express transfers — a separate-charge-and-transfer model, not destination charges.
- Webhook idempotency is enforced by a unique (provider, providerEventId) on payment_webhook_events; signature verification is mandatory in StripeWebhookController.
- Payouts run off a double-entry-ish host ledger (host_ledger_entries) with a hold window (experience end + app.payout.hold-hours, default 72h); earnings are PENDING then AVAILABLE then PAID, with a unique index guaranteeing one EARNING per payment.
- Refund triggers a PROPORTIONAL clawback: the host earning is reversed only by the same fraction the customer is actually refunded (0% refund leaves the host whole), splitting gift-card share back to the card and cash share to Stripe.
- Commission rate is hard-capped by app.platform.max-commission-rate (default 0.50) in RateAdminService to guard against driving host payout negative once commission VAT is added.
- Invoice numbers are gap-free per-year sequences allocated under a SELECT ... FOR UPDATE row lock (InvoiceNumberService), and issuer company/BTW details are snapshotted onto each invoice.

← back to the architecture index

Booking, Availability, Concurrency & Lifecycle

LocalBuddy backend architecture · June 2026

Booking, availability, concurrency & lifecycle

> Seat reservation is guarded by pessimistic row locks on availability_slots plus DB CHECK/partial-unique backstops, and bookings move through a payment-driven lifecycle (PENDING_PAYMENT to CONFIRMED to COMPLETED) reconciled by four scheduled sweepers.

Scope & key types

This lens covers the `com.localbuddy.availability` and `com.localbuddy.booking` packages plus the satellite services that drive booking state transitions: `com.localbuddy.noshow`, `com.localbuddy.attendance`, `com.localbuddy.safety`, and the relevant slice of `com.localbuddy.payment.PaymentService`.

Concern	Real artifact
Slot entity	`AvailabilitySlot` (table `availability_slots`): `capacity`, `bookedCount`, `status`
Slot status enum	`AvailabilityStatus` = `AVAILABLE`, `BLOCKED`, `CANCELLED`
Booking entity	`Booking` (table `bookings`): `seatsBlocked`, `guestsCount` + age bands, `status`, `attendanceOutcome`, `guestShowStatus`
Booking status enum	`BookingStatus` = `REQUESTED`, `ACCEPTED`, `PENDING_PAYMENT`, `CONFIRMED`, `DECLINED`, `CANCELLED_BY_LOGGED_IN_USER`, `CANCELLED_BY_LOCAL`, `CANCELLED_BY_ADMIN`, `CANCELLED_MINIMUM_NOT_MET`, `COMPLETED`, `EXPIRED`
Booking mode enum	`BookingMode` (on `Experience`) = `SHARED`, `PRIVATE_ALLOWED`, `PRIVATE_ONLY`
Channel enum	`BookingSource` = `LOGGED_IN_USER`, `GUEST_USER`, `ADMIN`
No-show flag	`AttendanceOutcome` = `NONE`, `HOST_NO_SHOW`, `CUSTOMER_NO_SHOW`
In-person mark	`GuestShowStatus` = `PENDING`, `SHOWED`, `NO_SHOW`
Core service	`BookingService` (creation, accept/decline, cancel, complete, reschedule)
Concurrency primitive	`AvailabilitySlotRepository.findByIdForUpdate` (`@Lock(PESSIMISTIC_WRITE)`)

Booking channels (three creation paths)

All three converge on the same seat-reservation logic but differ in entry status and validation:

1. **Logged-in traveller** — `BookingService.createBooking(loggedInUserId, CreateBookingRequest)`. Requires `UserRole.LOGGED_IN_USER`, runs `trustSafetyService.requireUserCanBook` / `requireUserCanHost`, `consentService.requireTravelerConsents`, and an application-level duplicate check (`existsByLoggedInUserIdAndAvailabilitySlotIdAndStatusIn`). Creates the booking at `PENDING_PAYMENT`. Usually invoked via `BookingCheckoutService.createBookingAndCheckout`, which immediately calls `PaymentService.createCheckout` to mint a Stripe session.
2. **Guest (no account)** — `BookingService.createGuestBooking(CreateGuestBookingRequest, ip, userAgent)`. Validates terms + a current `ConsentService.CURRENT_CONSENT_VERSION`, normalises/duplicate-checks on `LOWER(guest_email)`, captures consent IP/user-agent. Also starts at `PENDING_PAYMENT`. Looked up later by reference + email via `lookupGuestBooking`.
3. **Admin offline** — `BookingService.createBookingByAdmin(AdminCreateBookingRequest)`. Skips online payment entirely: seats are blocked, status is set directly to `CONFIRMED` (`acceptedAt = now`), `discountAmount = 0`, and `BookingConfirmationNotifier.sendConfirmation` fires immediately.

> Note: although `BookingStatus.REQUESTED` is the JPA default on the `Booking` entity and `acceptBooking` / `declineBooking` exist, **no live creation path sets `REQUESTED`**. `createBooking` overwrites the default with `PENDING_PAYMENT`, and `acceptBooking` explicitly rejects `PENDING_PAYMENT` ("already ready for payment"). The accept/decline host-handshake is therefore dormant scaffolding kept for defensive completeness, not part of the active flow.

Booking modes & private (whole-slot) buyout

`requirePrivateBookingAllowed` and `computePricing` (in `BookingService`) implement the buyout rules described on `BookingMode`:

- **SHARED**: per-head pricing `pricePerGuest × billableUnits` (age-band weighted); `seatsBlocked = head count`.
- **PRIVATE_ALLOWED**: may book shared OR private. A private booking is only offered while `slot.bookedCount == 0` (enforced in `requirePrivateBookingAllowed` and surfaced by `AvailabilitySlotService.toResponse.privateBookingAvailable`). A private booking sets `seatsBlocked = slot.capacity`, charges the flat `experience.privatePrice`, and immediately drives the slot to `BLOCKED`.
- **PRIVATE_ONLY**: every booking is a whole-slot buyout; a non-private request is rejected.
- Private booking is also rejected when `experience.isListedOnExternalPlatform()`.

This `seatsBlocked` field is the linchpin of capacity accounting: `seatsConsumed(booking)` returns `seatsBlocked` (falling back to `guestsCount`), and every release path decrements `bookedCount` by exactly that amount.

Age-band pricing & age gate

`AgeBandPricing` (config-driven via `app.pricing.age-band.*`) resolves `adults/teens/children/infants` into `AgeBands`. **Seats count every person** (`totalGuests`), but **price is weighted**: adults & teens at `1.0`, children `0.5`, infants `0.0` by default. `validateAgeGate` rejects too-young bands against `experience.minimumAge` (TEEN_MAX 17, CHILD_MAX 12, INFANT_MAX 2). When no band counts are supplied it falls back to treating `guestsCount` as all adults (backward compatible).

Concurrency control — pessimistic, multi-layered

There is **no optimistic locking** (no `@Version` on `AvailabilitySlot` or `Booking`) and **no explicit transaction isolation override** — services rely on the default isolation plus a pessimistic row lock. The defence is layered:

Layer	Mechanism	Where
1. Row lock	`SELECT ... FOR UPDATE` via `findByIdForUpdate` (`@Lock(PESSIMISTIC_WRITE)`) serialises all writers on a single slot	Every create/cancel/decline/reschedule/expiry path loads the slot through this method
2. In-tx invariant	`validateSlot` re-checks status, future start, and `guestsCount > capacity − bookedCount` while holding the lock	`BookingService.validateSlot`
3. DB CHECK	`chk_availability_booked_count_capacity` (`booked_count <= capacity`), `booked_count >= 0`, `capacity > 0`	`V1__baseline_schema.sql`
4. Partial-unique	`ux_bookings_active_traveler_slot` and `ux_bookings_active_guest_slot` prevent a second active booking by the same traveller/guest-email per slot; create paths catch `DataIntegrityViolationException` and translate to a friendly "already have an active booking"	`V1` indexes; `BookingService` try/catch

The ordering inside `createBooking` matters: the **lock is acquired first** (`findByIdForUpdate`), then the in-memory mutation (`bookedCount += seatsToBook`, flip to `BLOCKED` at capacity), then `save`. Because the lock is held until commit, two concurrent requests for the last seat are serialised — the second one re-reads the now-updated `bookedCount` and fails `validateSlot`. The DB CHECK constraint is a last-resort backstop that would only fire if the lock were somehow bypassed.

> Trade-off: pessimistic locking is simple and correct here but serialises **all** bookings against a hot slot. The `LOGGED_IN_BOOKING_TIMING` / `GUEST_BOOKING_TIMING` instrumentation peppered through `createBooking` exists precisely because promo/referral/notification work happens *inside* the lock window, lengthening the critical section. A latency risk under contention, though acceptable at this scale.

State machine — transitions and triggers

From	To	Trigger / Actor	Capacity effect
(new)	`PENDING_PAYMENT`	`createBooking` / `createGuestBooking` (traveller or guest)	seats reserved
(new)	`CONFIRMED`	`createBookingByAdmin` (offline)	seats reserved
`PENDING_PAYMENT`	`CONFIRMED`	Stripe paid → `PaymentService.confirmBookingAfterPayment` / `finalizePaidBooking`	seats retained
`PENDING_PAYMENT`	`EXPIRED`	`BookingExpiryService.expirePendingPaymentBookings` (no paid payment after `pending-payment-expiration-minutes`, default 15) OR `releaseBookingSlotAfterFailedPayment` on Stripe failure/session-expiry	seats released
`PENDING_PAYMENT`	`CONFIRMED`	expiry sweep finds a late PAID payment → confirms instead of expiring	seats retained
`PENDING_PAYMENT` / `CONFIRMED`	`CANCELLED_BY_LOGGED_IN_USER`	`cancelBookingByLoggedInUser`	seats released + refund policy
`PENDING_PAYMENT` / `CONFIRMED`	`CANCELLED_BY_LOCAL`	`cancelBookingByLocal` — **blocked within 24h of start** (`HOST_CANCEL_MIN_HOURS`)	seats released + refund
`PENDING_PAYMENT` / `CONFIRMED`	`CANCELLED_BY_ADMIN`	`cancelBookingByAdmin`; also `NoShowService.approve` for a verified HOST no-show (full refund)	seats released + refund
active set	`CANCELLED_MINIMUM_NOT_MET`	`UnderbookedSlotService.cancelUnderbookedSlot` (host-initiated guaranteed-departure cancel)	full refund; slot → `CANCELLED`, `bookedCount = 0`
`CONFIRMED`	`COMPLETED`	`completeBooking` (host, gated by safety checklist) OR `BookingAutoCompletionService` (48h after start, no pending no-show) OR `NoShowService.approve` of a CUSTOMER no-show	none
`REQUESTED`	`ACCEPTED` / `DECLINED`	`acceptBooking` / `declineBooking` (dormant host handshake)	decline releases seats

Reschedule (`rescheduleBooking`, host or admin) is an in-state move for `PENDING_PAYMENT` / `CONFIRMED`: it locks **both** old and new slots via `findByIdForUpdate`, validates capacity on the new slot, moves `seatsConsumed` between them, and notifies the waitlist on the freed old slot.

Payment-driven confirmation & the expiry race

`PaymentService.confirmBookingAfterPayment` flips `PENDING_PAYMENT` → `CONFIRMED` and fires the wallet/calendar confirmation. The system explicitly handles the race between the 15-minute hold and a slow payment:

- If a payment lands **after** the seat was already released (booking no longer in an honorable state), `finalizePaidBooking`'s safety net refunds in full rather than confirming (PaymentService ~line 644).
- Conversely, `expireBookingIfStillUnpaid` re-checks for a PAID payment before expiring; if found, it **confirms** instead. `cancelOpenPaymentsForExpiredBooking` proactively calls `paymentCheckoutProvider.expireCheckout(...)` so a late Stripe payment cannot sneak in after the seat is freed.

Capacity release & `BLOCKED` ↔ `AVAILABLE` toggling

Five methods release capacity, all structurally identical (`Math.max(0, bookedCount - seatsConsumed)`) then un-block if below capacity, then `waitlistService.notifyOpenedSpots`): `BookingService.releaseAvailabilityCapacity`, `releaseAvailabilityCapacityFromSlot` (reschedule), `declineBooking`, and `BookingExpiryService.releaseAvailabilityCapacity`. A slot auto-flips to `BLOCKED` the moment `bookedCount >= capacity` and back to `AVAILABLE` on release. Hosts can manually `block` / `unblock` / `delete` slots via `AvailabilitySlotService`, but `requireNoActiveBookings` forbids blocking/deleting a slot that still holds active bookings — meaning a booked slot inside 24h is effectively frozen.

Completion safety gate

`completeBooking` calls `validateSafetyChecklistCompleted`, which requires a completed `BookingSafetyChecklist` row (`existsByBookingIdAndUserIdAndCompletedTrue`) for **both** the traveller and the host before a `CONFIRMED` booking may become `COMPLETED`. The automatic completer (`BookingAutoCompletionService`) bypasses this gate by design — it only runs 48h post-start and skips bookings with a pending no-show report.

Guaranteed-departure / minimum-not-met

`UnderbookedSlotService` implements optional guaranteed-departure. The platform **never auto-cancels**; instead a scheduled sweep (`notifyUnderbookedSlots`) finds slots in the notice window (between `underbooked-cancel-deadline-hours` and `underbooked-notice-hours` before start) below `minimum-guests-to-confirm` (default 3) and notifies the host once. The host may then call `cancelUnderbookedSlot`, which (while below minimum and before the deadline) fully refunds every active booking, sets each to `CANCELLED_MINIMUM_NOT_MET`, and marks the slot `CANCELLED`.

No-show & attendance

- `NoShowService`: either party (or an anonymous guest by reference+email) files within a 48h window on a `CONFIRMED` booking. Admin `approve` of a HOST report → full refund + `CANCELLED_BY_ADMIN` + `attendanceOutcome = HOST_NO_SHOW`; approve of a CUSTOMER report → `COMPLETED` + `CUSTOMER_NO_SHOW` (host keeps payment). One open report per subject is enforced.
- `AttendanceService`: geo check-in. **Guests are hard-gated** — must be `CONFIRMED`, inside the check-in time window (`app.checkin.before/after-minutes`), supply trustworthy accuracy (`max-accuracy-meters`), and be inside the error-adjusted geofence. **Hosts are never distance-rejected** (recorded with a warning). Hosts mark `guestShowStatus` (`SHOWED` / `NO_SHOW`) operationally. Concurrent double-tap check-ins are handled idempotently via `saveHandlingConcurrentInsert` (separate-transaction insert + re-read on unique violation against the V23 partial indexes).

Scheduled jobs (lifecycle reconcilers)

Job	Class	Default cadence	Purpose
Pending-payment expiry	`BookingExpiryService.expirePendingPaymentBookings`	60s (`expiry-processor-delay-ms`)	expire unpaid holds after 15min, release seats, or confirm if late-paid
Underbooked notice	`UnderbookedSlotService.notifyUnderbookedSlots`	300s	warn hosts of sub-minimum slots
Auto-completion	`BookingAutoCompletionService.autoCompletePastBookings`	300s	complete `CONFIRMED` bookings 48h post-start with no pending no-show
Check-in retention purge	`AttendanceService.purgeExpiredCheckIns`	daily	delete check-ins past `retention-days` (90)

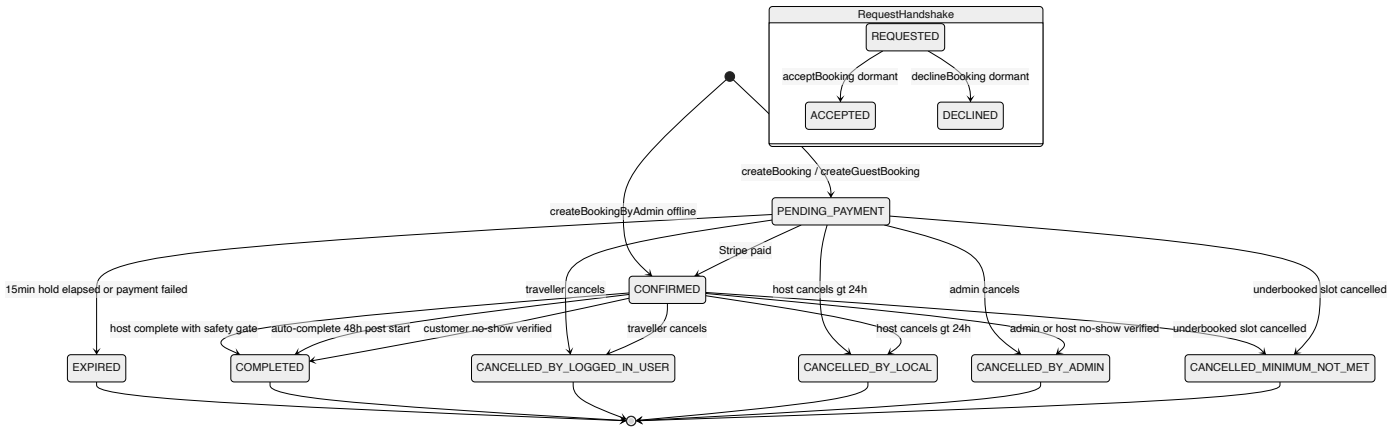
All run `@Transactional` and page with `findTop100.../findTop200...` bounded queries to cap per-tick work.

Key architectural decisions & trade-offs

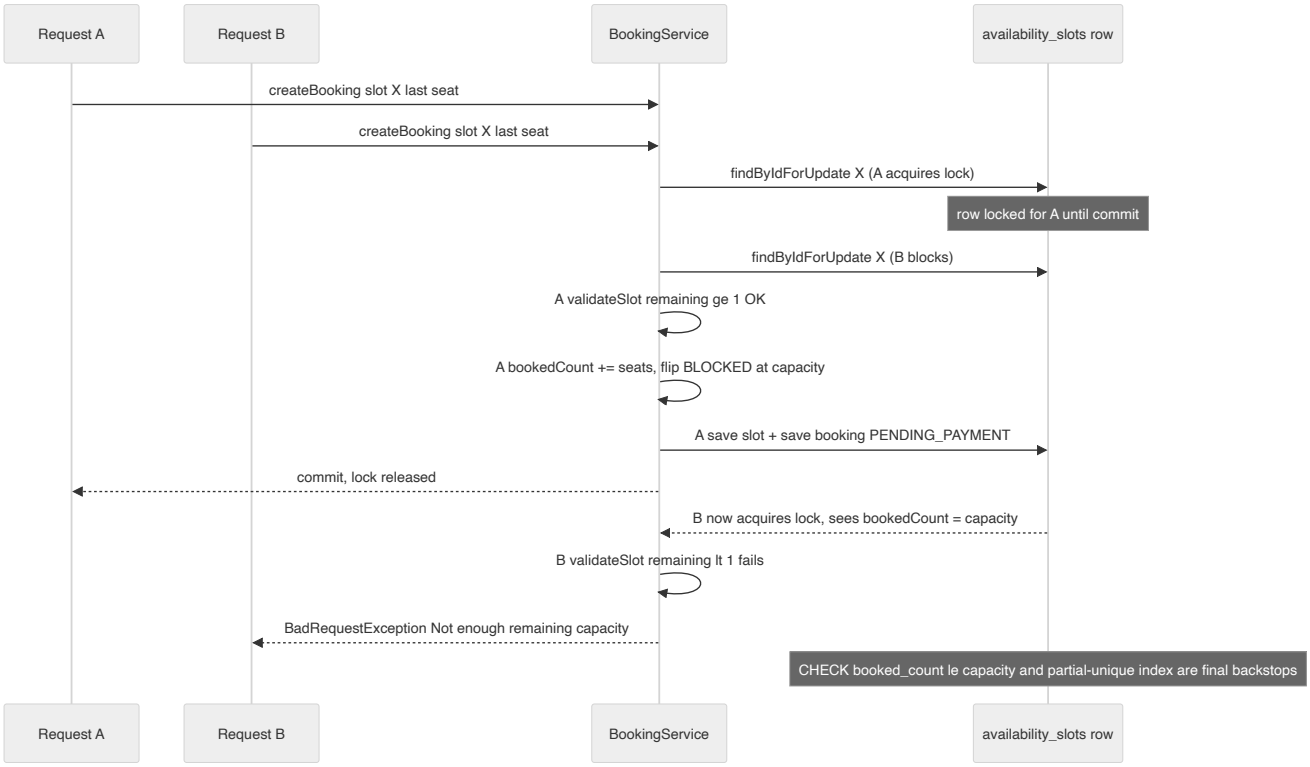
- **Pessimistic over optimistic locking.** `findByIdForUpdate` serialises slot writers. Simple and race-free, but lengthens the critical section because promo/referral/notification work happens inside the lock; the timing logs reflect awareness of this cost.
- **Defence in depth on capacity.** App-level `validateSlot` + DB CHECK (`booked_count <= capacity`) + partial-unique active-booking indexes mean overbooking and double-booking are blocked even if one layer is bypassed.
- **`seatsBlocked` decouples seats from heads.** Cleanly supports private buyout (seats = capacity, flat price) alongside age-band head counts without special-casing release logic.
- **Payment is the source of truth for confirmation.** PENDING_PAYMENT/CONFIRMED reconciliation is bidirectional and idempotent, explicitly handling the hold-expiry-vs-late-payment race in both directions (refund a too-late payment; confirm a just-in-time one).
- **Hosts never get auto-cancelled or distance-blocked.** Guaranteed-departure and host check-in are advisory/host-driven, reflecting a marketplace trust posture; only guests are hard-gated.
- **Dormant request/accept handshake.** `REQUESTED` / `ACCEPTED` and `acceptBooking` / `declineBooking` are retained but unreachable from live flows — worth flagging as either future re-enablement or dead code to prune.

Diagrams

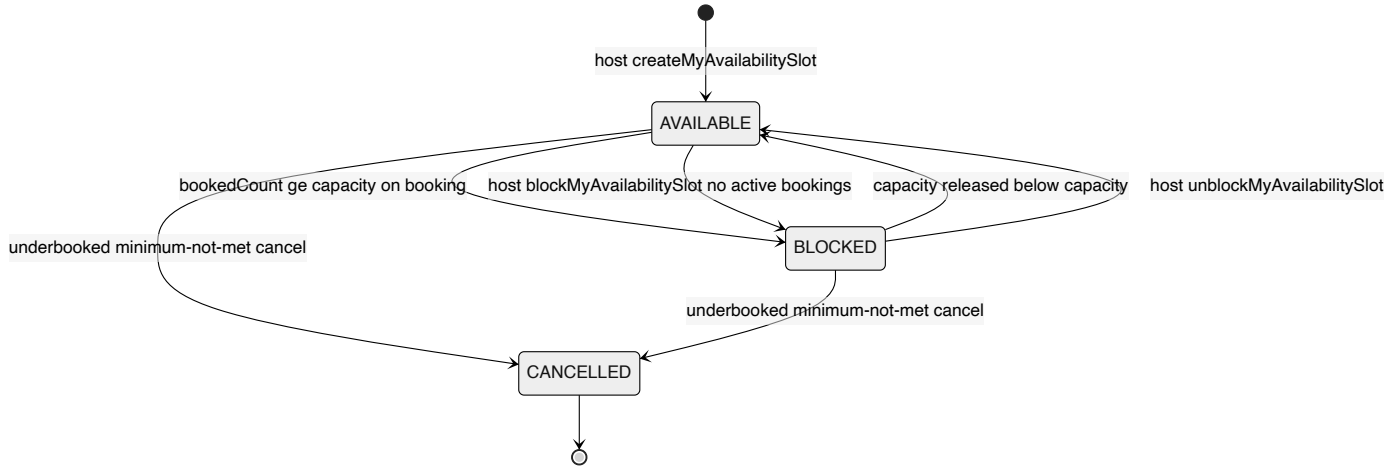
Booking lifecycle state machine



Concurrent seat reservation with pessimistic lock



Availability slot status transitions



Key architectural decisions & trade-offs

- Seat concurrency is controlled by pessimistic row locks (SELECT FOR UPDATE via AvailabilitySlotRepository.findByIdForUpdate, @Lock PESSIMISTIC_WRITE); there is no @Version optimistic locking and no explicit isolation override.
- Capacity correctness is layered: in-transaction validateSlot check + DB CHECK constraint booked_count <= capacity + partial-unique active-booking indexes (ux_bookings_active_traveler_slot / _guest_slot), with DataIntegrityViolationException translated to a friendly error.
- Bookings start at PENDING_PAYMENT (admin offline path starts at CONFIRMED); the REQUESTED/ACCEPTED host-handshake (acceptBooking/declineBooking) is dormant scaffolding not reachable from live creation flows.

- The seatsBlocked field decouples reserved seats from head count, enabling private whole-slot buyout (seats = capacity, flat privatePrice) to share all capacity-release logic with shared bookings.
- Payment is the source of truth for confirmation; the hold-expiry vs late-payment race is handled bidirectionally — expiry confirms a late-PAID booking, and a too-late payment is refunded by finalizePaidBooking's safety net.
- Four scheduled @Transactional sweepers reconcile lifecycle: pending-payment expiry (60s), underbooked notice (300s), auto-completion 48h post-start (300s), and check-in retention purge (daily), all bounded with findTop100/200 queries.
- Completion requires a two-sided safety checklist (host + traveller) via validateSafetyChecklistCompleted, except auto-completion which intentionally bypasses the gate 48h after start.
- Guaranteed-departure is host-driven, never automatic: UnderbookedSlotService notifies hosts of sub-minimum (default 3) slots and lets them cancel with full refunds (CANCELLED_MINIMUM_NOT_MET) before a deadline.

← back to the architecture index

External Integrations, Adapters & Resilience

LocalBuddy backend architecture · June 2026

External integrations, adapters & resilience

> Every outbound dependency sits behind a thin port-and-adapter abstraction that is config-guarded and dormant-by-default, degrading gracefully (stale-cache, console fallback, link-only modes) rather than failing the app — but there is no formal retry/circuit-breaker layer.

Overview

LocalBuddy talks to eight categories of external system. Each is isolated behind a thin Spring `@Component` / `@Service` adapter, and almost every one is **config-guarded and dormant-by-default**: with no credentials the bean still loads, reports `isConfigured() == false` (or has an empty `@Value` default), and the app boots cleanly for dev/test. Resilience is **bespoke per integration** — there is no shared retry, timeout, or circuit-breaker layer (a `pom.xml` / source scan finds no `resilience4j`, `spring-retry`, or `@Retryable`). The `RestClient`-based adapters use Spring's default connect/read timeouts.

Outbound integration inventory

Integration	Adapter class(es)	Abstraction (port)	SDK / transport	Config prefix	Configured-when	Degradation / fallback
Stripe Checkout + refunds	`StripePaymentCheckoutProvider`	`PaymentCheckoutProvider`	`stripe-java` 32.1.0 (`StripeClient`)	`app.payments.stripe`	`secret-key` set	**Fails fast** — `StripeClientConfig` throws startup if key absent
Stripe Connect payouts	`StripeConnectPayoutProvider`	`ConnectPayoutProvider`	`stripe-java` (`accounts`, `accountLinks`, `transfers`)	`app.payments.stripe` + `app.payouts.connect`	`secret-key` set	`isConfigured()` gate; throws `BadRequestException` if calle unconfigured
Stripe webhooks (inbound)	`StripeWebhookController` → `PaymentService`	n/a	`com.stripe.net.Webhook`	`app.payments.stripe.webhook-secret`	secret set	Throws `IllegalStateExcepti` if secret missing; rejects bad signatures
Google social login	`GoogleTokenVerifier`	`SocialTokenVerifier`	`RestClient` → Google `tokeninfo`	`app.social.google.client-id`	optional (audience check skipped if blank)	Verification failure → `BadRequestException("Inva Google token")`
Facebook social login	`FacebookTokenVerifier`	`SocialTokenVerifier`	`RestClient` → Graph `/me`	—	always present	Failure → `BadRequestException("Inva Facebook token")`
Apple social login	*(none)*	`SocialTokenVerifier`	—	—	**never**	`SocialAuthService` returns "Sign-in with APPLE is not availab yet"
Claude AI	`ClaudeClient` → `AiService`	concrete (no interface)	`RestClient` → Anthropic Messages API	`app.ai.anthropic`	`api-key` set	`isConfigured()` gate; moderation **fails open**; listing cop falls back to raw text
WhatsApp	`WhatsAppService`	concrete	`RestClient` → Meta Cloud API v21.0	`app.whatsapp`	`access-token` + `phone-number-id`	Falls back to credential-free `wa.` click-to-chat links
Azure Blob media	`AzureBlobStorageProvider`	`MediaStorageProvider`	`azure-storage-blob` 12.29.1	`app.media.azure`	`connection-string` set	Falls back to "register external UF endpoint; delete is best-effort no-throw
Frankfurter FX rates	`FrankfurterExchangeRateProvider`	`ExchangeRateProvider`	`RestClient` → `api.frankfurter.dev`	`app.currency`	always (no API key)	`CurrencyConversionService` serves **stale cache**; else `BadRequestException`
Email	`AzureEmailProviderService` / `ConsoleEmailProviderService`	`EmailProviderService`	`azure-communication-email` 1.0.23	`app.email.provider`	`azure` vs `console` (default)	`@ConditionalOnProperty` selects console logger when not `azure`
Apple Wallet	`AppleWalletService`	concrete	BouncyCastle `bcpkix-jdk18on` 1.78.1 (PKCS#7)	`app.wallet.apple`	passTypeId+teamId+cert+wwdr	`isConfigured()`; `WalletService` omits Apple li unconfigured
Google Wallet	`GoogleWalletService`	concrete	`jjwt` 0.12.6 (RS256 JWT)	`app.wallet.google`	issuerId+classId+SA email+key	`isConfigured()`; save-URL omitted if unconfigured
Calendar (.ics)	`CalendarService`	concrete	none (string-built RFC 5545)	—	always (offline)	n/a — pure generation, no outbou call

Ports-and-adapters style

The codebase applies hexagonal isolation **selectively**, where a vendor swap is genuinely plausible:

- **`PaymentCheckoutProvider`** (`payment/`) — defines `getProvider()`, `createCheckout`, `createGiftCardCheckout`, `refundPayment`, and a best-effort `expireCheckout`. The enum `PaymentProvider {STRIPE, PAYPAL, ADYEN, MANUAL}` signals intended multi-PSP support; only `STRIPE` is implemented.
- **`ConnectPayoutProvider`** (`payout/`) — `isConfigured`, `ensureConnectAccount`, `createOnboardingLink`, `transfer`. Single impl `StripeConnectPayoutProvider` (Express accounts, hosted onboarding, destination transfers).
- **`MediaStorageProvider`** (`media/`) — `isConfigured`, `upload`, `delete`, returning `StoredObject(storageKey, url)`. Single impl over Azure Blob.
- **`ExchangeRateProvider`** (`currency/`) — `fetchLatestRates(base, symbols)`. Single impl over Frankfurter; the *caching/fallback* concern lives one layer up in `CurrencyConversionService`, keeping the adapter a pure fetcher.
- **`EmailProviderService`** (`notification/email/`) — selected at wiring time by `@ConditionalOnProperty(name = "app.email.provider", havingValue = "azure" | "console")`, so there is exactly one bean and consumers (`NotificationProcessingService`) are oblivious.
- **`SocialTokenVerifier`** (`auth/`) — each verifier declares its `provider()`; `SocialAuthService` injects `List<SocialTokenVerifier>` and reduces it to a `Map<SocialProvider, SocialTokenVerifier>`, the cleanest extensibility seam in the integration layer.

By contrast, **AI, WhatsApp, both wallet services, and the calendar** are concrete services with no interface. This is a reasonable trade-off (single vendor each, no current swap pressure) but means a future second provider for any of them requires extracting an interface first.

Resilience & graceful-degradation patterns

Each integration handles failure in its own way — there is no unifying policy:

- **Currency — stale-cache read-through (the richest pattern).** `CurrencyConversionService` keeps a `ConcurrentHashMap<String, CachedRates>` with a configurable TTL (`app.currency.cache-ttl-seconds`, default 3600). On a fresh hit it returns cached rates; on miss/expiry it calls Frankfurter; if that **throws and a stale entry exists**, it logs a warning and serves the stale rates; only with no cache at all does it raise `BadRequestException("Currency rates unavailable")`. This is the only integration with an explicit stale-fallback.
- **Email — provider substitution.** When `app.email.provider != azure`, the `ConsoleEmailProviderService` logs the message and returns a synthetic success (`console-<ts>`), so notification flows work end-to-end without a mail vendor. `AzureEmailProviderService` catches all exceptions and returns `EmailSendResult(success=false, …)` rather than throwing.

- **WhatsApp — dual-mode with a credential-free default.** `buildClickToChatLink()` produces `wa.me` deep links needing no API and no cost; `sendMessage()` (Meta Cloud API) is gated by `isConfigured()`. A code comment flags the real-world constraint that business-initiated messages outside the 24h window need approved templates.
- **Media — alternate ingestion path.** If Blob storage is unconfigured, `AzureBlobStorageProvider.upload` throws a guidance error pointing to the `registerPhotoUrl` endpoint (`ExperiencePhotoService`), letting hosts attach externally hosted image URLs. `delete()` is best-effort and never throws.
- **AI — fail-open + tolerant parsing.** `AiService.moderate()` returns `flagged=false` when the model output can't be parsed as a verdict (does **not** block content), and `generateListing()` falls back to using raw model text as the description. `tryParseJson` strips Markdown code fences and extracts the first `{…}`.
- **Best-effort no-throw operations.** `StripePaymentCheckoutProvider.expireCheckout` (called from `BookingExpiryService` line ~123) and `AzureBlobStorageProvider.delete` swallow all exceptions by contract — the session/blob may already be gone.
- **Async decoupling for notifications.** Email/WhatsApp sends never run on the request thread. `NotificationProcessor` polls `PENDING` rows every `app.notifications.processor-delay-ms` (10s) in batches of 20; `NotificationProcessingService` transitions each row `PENDING → PROCESSING → SENT/FAILED/SKIPPED` (`findByIdForUpdate` row-lock), persisting `failureReason`. There is no automatic re-enqueue of `FAILED` rows.

> **Resilience gap worth flagging to the architect:** outbound calls have no explicit timeouts beyond `RestClient` /SDK defaults, no retry/backoff, and no circuit breaker. A slow Anthropic, Frankfurter, Graph, or Stripe endpoint will tie up the calling thread (notification scheduler or request thread) for the default read timeout. The notification poller mitigates this for email/WhatsApp; synchronous paths (AI endpoints, currency, social login, checkout creation) do not benefit.

Stripe deep-dive: signature, idempotency, money safety

- **Inbound integrity.** `StripeWebhookController` reads the raw body + `Stripe-Signature` header and calls `Webhook.constructEvent(payload, sig, webhookSecret)`; a `SignatureVerificationException` becomes a 400. Missing webhook secret → `IllegalStateException`.
- **Idempotency.** `PaymentService.handleStripeWebhookEvent` short-circuits on `existsByProviderAndProviderEventId(STRIPE, eventId)` and returns "Duplicate webhook ignored", persisting every event into `payment_webhook_events` with `raw_payload`, `processed`, and `processing_error`. Handler exceptions are caught and stored as `processing_error` (the event is recorded, not retried).
- **Handled event types.** `checkout.session.completed` (branches on `giftCardId` metadata → `GiftCardService.activatePurchasedCard`, else `markPaymentPaidFromStripeSession`), plus `checkout.session.expired` / `checkout.session.async_payment_failed` → release the held seat. Sessions carry `paymentId` / `bookingId` / `giftCardId` in `metadata` and a `clientReferenceId` for correlation.
- **Money-handling specifics.** Amounts are converted to integer cents via `movePointRight(2).longValueExact()`; the customer is charged only the non-gift-card remainder; refund reasons are mapped to Stripe's enum; checkout sessions get `expiresAt = now + 31min` (Stripe's 30-min floor) while the booking expiry job proactively expires them earlier.

Configuration toggles (env-driven)

All integrations are wired from `application.yaml` via `${ENV:default}` placeholders, so production is configured purely through environment variables (Azure App Service):

Capability	Key env var(s)	Effect when unset
Stripe	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`	**App fails to start** (StripeClientConfig)
Email provider	`EMAIL_PROVIDER` (`console` / `azure`), `AZURE_COMMUNICATION_CONNECTION_STRING`	console logger (default)
Media	`AZURE_BLOB_CONNECTION_STRING`, `AZURE_BLOB_CONTAINER`, `AZURE_BLOB_PUBLIC_BASE_URL`	upload disabled, external-URL path only
WhatsApp	`WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`	wa.me links only
AI	`ANTHROPIC_API_KEY`, `ANTHROPIC_MODEL`, `ANTHROPIC_BASE_URL`, `ANTHROPIC_MAX_TOKENS`	AI endpoints return "not available yet"
Currency	`app.currency.provider-base-url`, `app.currency.supported`, `app.currency.cache-ttl-seconds`	live (no key needed)
Social	`GOOGLE_CLIENT_ID`	Google audience check skipped (still works); Apple always disabled
Apple Wallet	`APPLE_WALLET_PASS_TYPE_ID/TEAM_ID/CERT_BASE64/CERT_PASSWORD/WWDR_BASE64`	Apple pass disabled
Google Wallet	`GOOGLE_WALLET_ISSUER_ID/CLASS_ID/SA_EMAIL/SA_PRIVATE_KEY/ORIGIN`	Google save-link disabled

Wallet & calendar: in-process generation (no issue-time round-trip)

- **Apple (`AppleWalletService`)** builds an event-ticket `pass.json` + generated icons, a SHA-1 `manifest.json`, and a **detached PKCS#7 signature** via BouncyCastle (`CMSSignedDataGenerator`, `SHA256withRSA`), loading the Pass Type ID cert from a base64 `.p12` and chaining the Apple WWDR cert, then zips a `.pkpass`. No Apple network call.
- **Google (`GoogleWalletService`)** builds a generic-pass object and signs a `savetowallet` JWT (RS256 via `jjwt`, service-account private key) returning a `https://pay.google.com/gp/v/save/<jwt>` link.
- **Calendar (`CalendarService`)** emits RFC-5545 `.ics` (with proper escaping) and builds Google/Outlook "add to calendar" deep links entirely offline; `googleCalendarLink(booking)` is reused inside booking-confirmation messages.
- `WalletService` / `CalendarService` both enforce participant authorization (traveler or host) and return 404 on mismatch to avoid booking-ID enumeration.

Deployment context

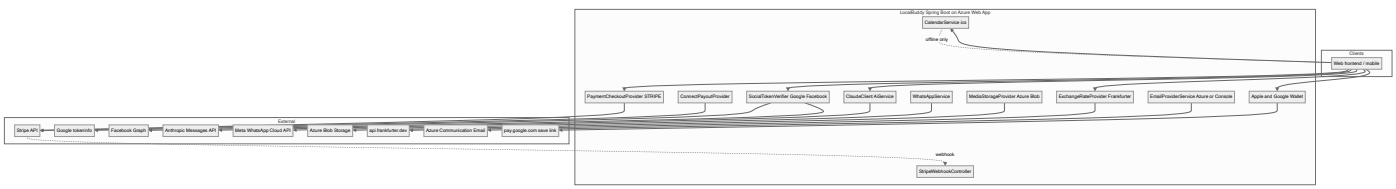
Outbound calls originate from a single Spring Boot service deployed to **Azure Web App** (`.github/workflows/azure-webapp.yml`, JDK 21, `azure/webapps-deploy@v3`, `workflow_dispatch` only). Inbound Stripe webhooks land on `POST /api/public/payments/webhooks/stripe`. Health is exposed via `management.endpoints … health,info`, but the **Redis health indicator is disabled** and no health checks exist for the external dependencies themselves.

Notable gaps / risks for the architect

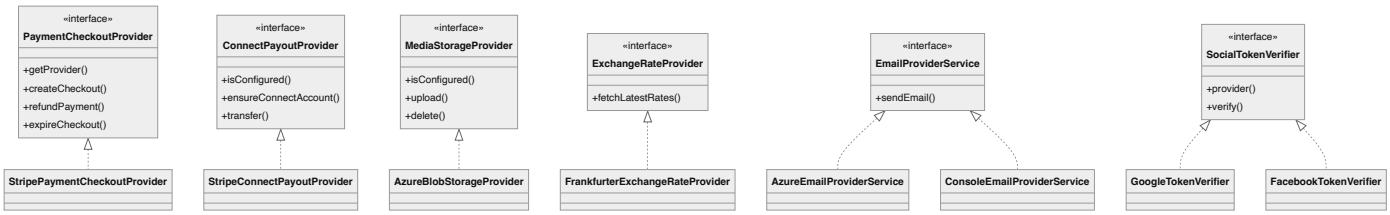
- No timeouts/retries/circuit-breakers on any outbound call (default `RestClient` /SDK timeouts only).
- Failed notifications (`FAILED` status) are not automatically retried; they require manual/operator intervention.
- Failed Stripe webhook handlers are stored with `processing_error` but not re-driven; reconciliation relies on Stripe's own redelivery hitting the idempotency guard.
- AI client lacks Anthropic prompt-caching, streaming, or token accounting; a single blocking call per request with a fixed `max-tokens`.
- `AppleWalletService` generates a flat-colour placeholder icon and includes only `icon` / `icon@2x` (no logo/strip), which Apple may reject in production — verify against Wallet requirements before launch.

Diagrams

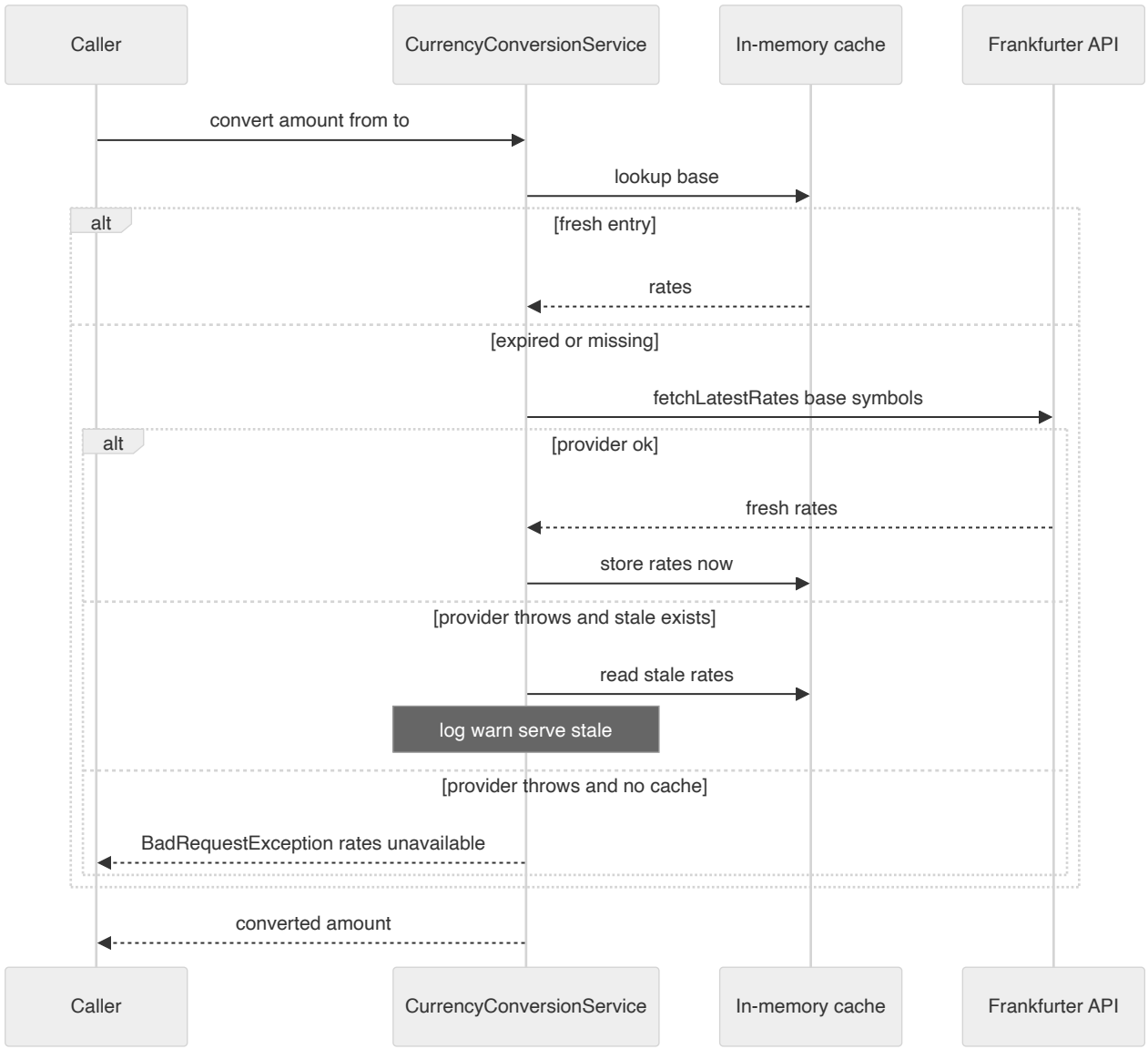
Integration context diagram



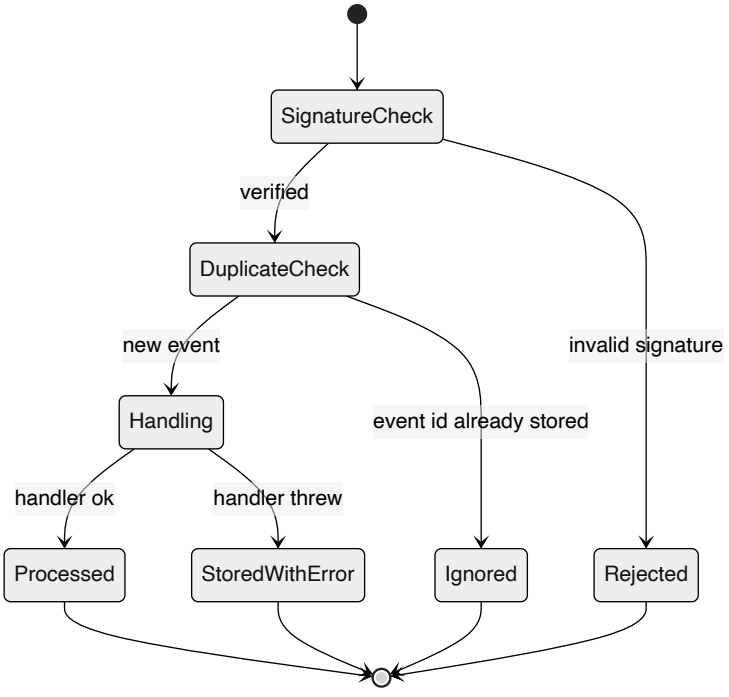
Adapter ports and config-guard structure



Currency stale-cache fallback



Stripe webhook idempotent processing



Key architectural decisions & trade-offs

- Ports-and-adapters only where a swap is plausible: payments (PaymentCheckoutProvider), payouts (ConnectPayoutProvider), media (MediaStorageProvider), exchange rates (ExchangeRateProvider), email (EmailProviderService), social auth (SocialTokenVerifier). AI, WhatsApp, wallet, and calendar are concrete services with no interface — accepted because there is currently a single vendor each.
- Config-guarded, dormant-by-default integrations: every paid/credentialed adapter exposes isConfigured() (or an empty @Value default) so the app boots and runs in dev/test with zero external credentials. Stripe is the one exception — StripeClientConfig fails fast at startup if the secret key is absent.
- Graceful degradation is bespoke per integration rather than a shared resilience policy: currency serves a stale in-memory cache on provider failure; email falls back to a console logger; WhatsApp falls back to credential-free wa.me click-to-chat links; media falls back to a register-external-URL endpoint; AI moderation fails-open. No resilience4j / spring-retry / circuit breaker is present.
- Stripe webhook integrity + idempotency: signatures are verified with the Stripe SDK (Webhook.constructEvent) and replays are de-duplicated by persisting every event in payment_webhook_events keyed on (provider, provider_event_id). Webhook processing swallows handler exceptions, storing processing_error so a failed event is recorded rather than retried automatically.
- Outbound adapter failures are uniformly wrapped in the domain BadRequestException (HTTP 400) instead of surfacing as 5xx, and best-effort operations (expireCheckout, blob delete) deliberately never throw.
- Social-login extensibility via Spring collection injection: SocialAuthService builds a Map<SocialProvider, SocialTokenVerifier> from all beans, so adding a provider is just adding a bean. APPLE is enumerated but has no verifier, so it returns a graceful 'not available yet' error.
- Notifications are processed asynchronously out-of-band by a DB-polling scheduler (NotificationProcessor every 10s), decoupling the request thread from email/WhatsApp provider latency; unsupported channels (SMS) are marked SKIPPED, not failed.
- Wallet passes are generated in-process (no vendor round-trip at issue time): Apple .pkpass is PKCS#7-signed locally via BouncyCastle; Google Wallet is an RS256 JWT save-link via jjwt — both dormant until certs/keys are configured.

← back to the architecture index

End-to-End Flows, Eventing & Observability

LocalBuddy backend architecture · June 2026

End-to-end flows, eventing, async & observability

> A single-instance Spring Boot monolith stitches cross-domain journeys together with a transactional-outbox notification table drained by a polling worker, an in-process AFTER_COMMIT event for booking creation, and ~11 fixed-delay @Scheduled sweeps — all idempotent via DB dedupe keys and partial unique indexes, but with no distributed locking, retries, dead-lettering, or tracing.

Scope and System Shape

This lens covers the reliability backbone that ties LocalBuddy's domains together: the **notification outbox + polling delivery worker**, the **scheduled-job fleet**, the **in-process eventing**, the **geo check-in / attendance** subsystem, the **no-show report → verification → refund** flow, the **host onboarding → submission → approval → listing** journey, and the system's **error/retry and observability posture**.

The architecture is a **single-deployment Spring Boot 3 / Java 21 monolith** on Azure Web App (`.github/workflows/azure-webapp.yml`) backed by one Postgres (HikariCP, `maximum-pool-size: 5`). There is **no message broker, no queue, and no distributed coordination**. All asynchrony is achieved with two primitives only:

1. **An outbox table** (`notifications`) written transactionally and drained by a poller.
2. **Spring `@Scheduled` fixed-delay sweeps** plus a single in-process `@TransactionalEventListener` .

`@EnableScheduling` and `@EnableConfigurationProperties` live on `LocalbuddyBackendApplication` ; `@EnableAsync` lives on `config/AsyncConfig` (an empty marker — **no custom `TaskExecutor` bean is defined**, so `@Async` uses Spring's default `SimpleAsyncTaskExecutor` , which spawns an unbounded new thread per call rather than a bounded pool).

The Notification Outbox

The notification subsystem is a textbook **transactional outbox**, and it is the single most important reliability mechanism in the codebase.

Write side — `NotificationService`

Domain code never calls an email/SMS provider directly. Instead it calls one of the typed emitters on `notification/NotificationService` :

Method	Channel	Recipient
`createEmailNotificationForUser`	`EMAIL`	logged-in `User`
`createEmailNotificationForGuest`	`EMAIL`	anonymous guest email
`createInAppNotificationForUser`	`IN_APP`	logged-in `User`
`createEmailAndInAppNotificationForUser`	both	fan-out, suffixes dedupe key with `:EMAIL` / `:INAPP`
`createWhatsAppNotificationForUser` / `...ForGuest`	`WHATSAPP`	phone-bearing recipient

Each call inserts a `Notification` row in state `PENDING` **inside the caller's transaction**, so the message is committed atomically with the business state change (e.g. a booking, an approval). Channels are `notification/NotificationChannel` and types are the 30-value `notification/NotificationType` enum (`BOOKING_CREATED` , `LOCAL_PROFILE_APPROVED` , `HOST_ARRIVED` , `BOOKING_REMINDER` , etc.).

Idempotency is the load-bearing design choice. Every emission carries a deterministic `dedupeKey` persisted under a `UNIQUE` column (`Notification.dedupeKey` , `length=255`). `createNotification` guards twice:

```
 if (notificationRepository.existsByDedupeKey(dedupeKey)) return;   // fast path
...
try { notificationRepository.save(notification); }
catch (DataIntegrityViolationException ex) { /* race: another tx won, ignore */ }
```

Dedupe keys are constructed from the entity and a stage/role discriminator, e.g.:

- `BOOKING_CREATED:LOCAL:<bookingId>` and `BOOKING_CREATED:TRAVELER:<bookingId>` (`BookingNotificationService`)
- `BOOKING_CONFIRMED:<bookingId>` / `...:WHATSAPP` (`BookingConfirmationNotifier`)
- `HOST_ARRIVED:<bookingId>` (`AttendanceService.notifyGuestsHostArrived`)
- `LOCAL_PROFILE_APPROVED:<profileId>` , `LOCAL_PROFILE_REJECTED:<profileId>:<reviewedAt>` (`LocalProfileService`)
- `booking-reminder:<bookingId>` (`BookingReminderService`), `wishlist-reminder:<itemId>:<stage>h:<channel>` , `abandoned-booking:<bookingId>:<stage>h:<channel>` (`ReminderService`)

This is what lets the schedulers run frequently and overlap without spamming — emission is **at-least-once**, but the row is **exactly-once**. No per-row "sent" flag is needed.

Read/drain side — `NotificationProcessor` + `NotificationProcessingService`

`NotificationProcessor.processPendingNotifications()` runs `@Scheduled(fixedDelay 10s, key app.notifications.processor-delay-ms)` . Each tick:

1. `findPendingNotificationIds(PENDING)` (JPQL, ordered `createdAt asc`), capped to `BATCH_SIZE = 20` in memory.
2. For each id, delegates to `NotificationProcessingService.processOneNotification(id)` — **one transaction per notification** so a single failure never poisons the batch.

`processOneNotification` selects the row `FOR UPDATE` via `findByIdForUpdate` (`@Lock(PESSIMISTIC_WRITE)`), re-checks it is still `PENDING` (guards against a concurrent picker), flips it to `PROCESSING` , then dispatches by channel:

- `IN_APP` → marked `SENT` immediately (delivered by being persisted; read through the feed).
- `EMAIL` → `EmailProviderService.sendEmail(...)` ; `SENT` + `providerMessageId` on success, `FAILED` + `failureReason` on failure, `SKIPPED` if recipient email missing.
- `WHATSAPP` → `SKIPPED` if `WhatsAppService.isConfigured()` is false or phone missing, else best-effort send (note: free-form WhatsApp only delivers inside a 24h session; templates are a documented TODO).
- any other channel → `SKIPPED` .

`NotificationStatus` is `PENDING | PROCESSING | SENT | FAILED | SKIPPED` .

Provider abstraction

`EmailProviderService` has two `@ConditionalOnProperty(app.email.provider=...)` implementations: `AzureEmailProviderService` (Azure Communication Services, synchronous `SyncPoller.waitForCompletion()`) and `ConsoleEmailProviderService` (dev default, `app.email.provider=console`). The Azure send is **blocking inside the worker transaction**, so a slow provider holds the row lock and a worker thread for the duration.

Reliability gaps (outbox)

- **No retry / backoff / max-attempts.** A `FAILED` row is terminal; the poller only ever picks `PENDING` . A transient provider blip permanently drops the message.
- **No `PROCESSING` reclaim / lease.** If the app dies after flipping to `PROCESSING` but before the provider returns, the row is stranded — no visibility timeout sweeps it back to `PENDING` .
- **No dead-letter queue or alerting.** Failures are discoverable only by querying `notifications WHERE status='FAILED'` .
- **Batch is fetch-all-then- `limit(20)` :** `findPendingNotificationIds` returns *all* pending ids and trims in Java, so a large backlog loads every id each tick.

Eventing

There is exactly **one application event**: `booking/BookingCreatedEvent(UUID bookingId)` , published by `BookingService` (lines ~245 logged-in, ~812 guest) via `ApplicationEventPublisher` . `BookingNotificationEventListener` consumes it with:

```
@Async
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onBookingCreated(BookingCreatedEvent event) { ... }
```

`AFTER_COMMIT` ensures the booking row is durable before fan-out; `@Async` moves fan-out off the request thread. `BookingNotificationService.createBookingCreatedNotifications` then writes the outbox rows (host "new booking", traveler/guest "complete payment").

Everything else is direct synchronous invocation, not eventing: `LocalProfileService` calls `NotificationService` inline on approve/reject; `AttendanceService` calls it inline on host arrival; `BookingConfirmationNotifier.sendConfirmation` is called directly by `BookingExpiryService` when a payment lands. So the system is effectively **one event + an outbox**, not an event-driven architecture. A consequence: because `@Async` AFTER_COMMIT runs in a *new* transaction with no request context, a failure there is logged-and-lost — there is no compensation for a dropped fan-out (though the outbox rows it writes are themselves durable once committed).

Scheduled Jobs (the async backbone)

Eleven `@Scheduled` fixed-delay methods carry all time-driven behavior. All share the **default single-threaded scheduler** (no `TaskScheduler` /pool bean), so they execute serially; a long job delays the next.

Job (class.method)	Default delay	Config key	Responsibility / idempotency
`NotificationProcessor.processPendingNotifications`	10s	`app.notifications.processor-delay-ms`	Drain outbox; idempotent via row status + `FOR UPDATE`
`BookingExpiryService.expirePendingPaymentBookings`	60s	`app.booking.expiry-processor-delay-ms`	Expire unpaid `PENDING_PAYMENT`, release seats, cancel Stripe session; confirms if paid
`BookingAutoCompletionService.autoCompletePastBookings`	5m	`app.booking.auto-complete-processor-delay-ms`	Complete confirmed bookings 48h post-start; **skips any with a pending no-show report**
`UnderbookedSlotService` (sweep)	5m	`app.booking.underbooked-processor-delay-ms`	Min-guests-to-confirm / underbooked notices
`HostLedgerService.releaseHolds`	5m	`app.payout.ledger-release-delay-ms`	Move ledger entries `PENDING` → `AVAILABLE` past `available_at`
`PayoutScheduler.runScheduledPayouts`	1h	`app.payout.processor-delay-ms`	Pay hosts on cadence (`WEEKLY/BIWEEKLY/MONTHLY`); per-host try/catch
`BookingReminderService.sendUpcomingBookingReminders`	15m	`app.notifications.reminder-processor-delay-ms`	One reminder per confirmed booking in lead window (24h); dedupe-keyed
`ReminderService.sendWishlistReminders`	30m	`app.reminders.processor-delay-ms`	Wishlist nudges at 2/24/48h offsets, suppressed if engaged
`ReminderService.sendAbandonedBookingReminders`	30m	`app.reminders.processor-delay-ms`	Nudge `EXPIRED` bookings to re-book
`AttendanceService.purgeExpiredCheckIns`	24h	`app.checkin.retention-processor-delay-ms`	GDPR-style purge of check-ins > 90 days

Notable patterns:

- **Bounded batches** (`findTop100By...`, `findTop200By...`, `findTop500By...`) cap each sweep and rely on the next tick to make progress — a deliberate throughput-vs-fairness trade.
- **Polling-on-a-cadence** for payouts: the job runs hourly but `isPayoutDay()` no-ops except on the cadence day; once paid, balances drop below `minimum-amount` so repeat runs are inert.
- **No distributed lock (no ShedLock).** Safe under one instance only. The booking/payout sweeps lean on **row-level `SELECT ... FOR UPDATE`** (`AvailabilitySlotRepository.findByIdForUpdate`) for correctness, not job-level locking — but two instances would still double-fire reminders and payouts.

Host Onboarding → Submission → Approval → Listing

State lives on `localprofile/LocalProfile.approvalStatus` (`LocalApprovalStatus`): `DRAFT → SUBMITTED → {APPROVED | CHANGES_REQUESTED | REJECTED} → ...`, plus a terminal `BLOCKED`. `LocalProfileService` enforces all transitions; each emits exactly one outbox notification.

Action	Method	From → To	Notification type / dedupe
Create profile	`createMyLocalProfile`	(none) → `DRAFT`	— (`verificationStatus=NOT_STARTED`)
Submit	`submitMyLocalProfile`	`DRAFT/CHANGES_REQUESTED/REJECTED` → `SUBMITTED`	`LOCAL_PROFILE_SUBMITTED:<id>`
Edit while approved	`updateMyLocalProfile`	`APPROVED` → `SUBMITTED` (re-review, sets `resubmittedAt`)	—
Admin approve	`approveLocalProfile`	`SUBMITTED` → `APPROVED`	`LOCAL_PROFILE_APPROVED:<id>`
Admin request changes	`requestChangesForLocalProfile`	`SUBMITTED` → `CHANGES_REQUESTED`	`LOCAL_PROFILE_CHANGES_REQUESTED:<id>:<reviewedAt>`
Admin reject	`rejectLocalProfile`	`SUBMITTED` → `REJECTED`	`LOCAL_PROFILE_REJECTED:<id>:<reviewedAt>`

Only `APPROVED` unlocks `canCreateExperience` (surfaced by `getMyOnboardingStatus` → `LocalOnboardingStatusResponse`). Re-submitting after `CHANGES_REQUESTED` / `REJECTED` clears the prior `reviewedAt` /reason fields and resets review state. The dedupe keys for changes/reject include `reviewedAt`, so a *second* review cycle correctly produces a *new* notification (unlike submit/approve which are once-per-profile).

Geo Check-in / Attendance

`attendance/AttendanceService` implements an **asymmetric trust geofence**, configured by `CheckInProperties` (`app.checkin.*`): 15-min before/after window, 300m geofence radius, 500m worst-trusted GPS accuracy, 90-day retention.

- **Guest check-in is hard-gated.** Booking must be `CONFIRMED`; `enforceWindow` checks the ±15-min slot window; `accuracyMeters` must be present and ≤ `maxAccuracyMeters`; `evaluateGeofence` requires `(distance − accuracySlack) ≤ radius`. Failures throw `BadRequestException` with human-readable distance/accuracy guidance. Two entry points: authenticated (`guestCheckIn`, `POST /api/bookings/{id}/check-in`) and anonymous-by-reference+email (`guestCheckInAnonymous`, `POST /api/public/check-in`).
- **Host check-in is soft.** `hostCheckIn` (`POST /api/host/slots/{slotId}/check-in`, multipart) is **never rejected for distance** — it records a warning + measured distance, optionally stores an arrival photo via `MediaStorageProvider`, then calls `notifyGuestsHostArrived` to emit `HOST_ARRIVED` notifications to every confirmed booking.
- **Host roster + show/no-show.** `getSlotAttendance` builds a `SlotAttendanceResponse`; `markGuestAttendance` sets `Booking.guestShowStatus` (`GuestShowStatus`, distinct from the admin `attendanceOutcome`).

Concurrency hardening (the most sophisticated bit of the codebase): the V23 **partial unique indexes** `uq_attendance_host_per_slot` (`WHERE role='HOST'`) and `uq_attendance_guest_per_booking` (`WHERE role='GUEST' AND booking_id IS NOT NULL`) guarantee one row per slot/booking. A racing double-submit is handled by `saveHandlingConcurrentInsert`: existing rows are updated in-place (no insert, no index trip); brand-new rows are inserted by `AttendanceCheckInWriter.insertNew` under `@Transactional(propagation = REQUIRES_NEW)` + `saveAndFlush`, so a unique-violation rolls back **only the nested tx** and the caller recovers idempotently by re-reading the winning row (avoiding Spring's `UnexpectedRollbackException` poisoning).

Check-ins are explicitly **operational signal, not refund proof**, and `purgeExpiredCheckIns` deletes them after `retentionDays`.

No-Show: Report → Verification → Refund

`noshow/NoShowService` runs a **human-in-the-loop** dispute flow; `NoShowReportStatus` is `REQUESTED | APPROVED | REJECTED`, subject is `NoShowSubject` (`HOST | CUSTOMER`).

1. **File** (`reportNoShow` for logged-in party at `POST /api/no-show/bookings/{id}/report`; `reportHostNoShowAsGuest` for anonymous guests at `POST /api/public/guest-no-show/report`). `validateReportable` requires `CONFIRMED` status, experience already started, within the **48h** **`REPORT_WINDOW_HOURS`**, and no existing open/approved report for the same subject. Non-parties get a `ResourceNotFoundException` (existence-hiding IDOR guard). A customer report targets `HOST`; a host report targets `CUSTOMER`.
2. **Admin verify** (`GET/POST /api/admin/no-show/reports...`). `approve`:

- `HOST` no-show → `PaymentService.fullyRefundBookingPayment(...)`, `attendanceOutcome=HOST_NO_SHOW`, status → `CANCELLED_BY_ADMIN`. **This is the only async-adjacent path that moves money automatically.**
- `CUSTOMER` no-show → informational; booking → `COMPLETED` (host keeps payment). `reject` simply closes the report. `removeFlag` clears the flag without reversing a refund.

3. **Auto-completion interplay.** `BookingAutoCompletionService` completes confirmed bookings 48h post-start but **defers any booking with a `REQUESTED` report** (`existsByBookingIdAndStatus`) so the admin decision wins. This coupling between two independently-scheduled jobs is the key correctness invariant of the no-show flow.

Observability, Error Handling & Retry Posture

Observability is minimal:

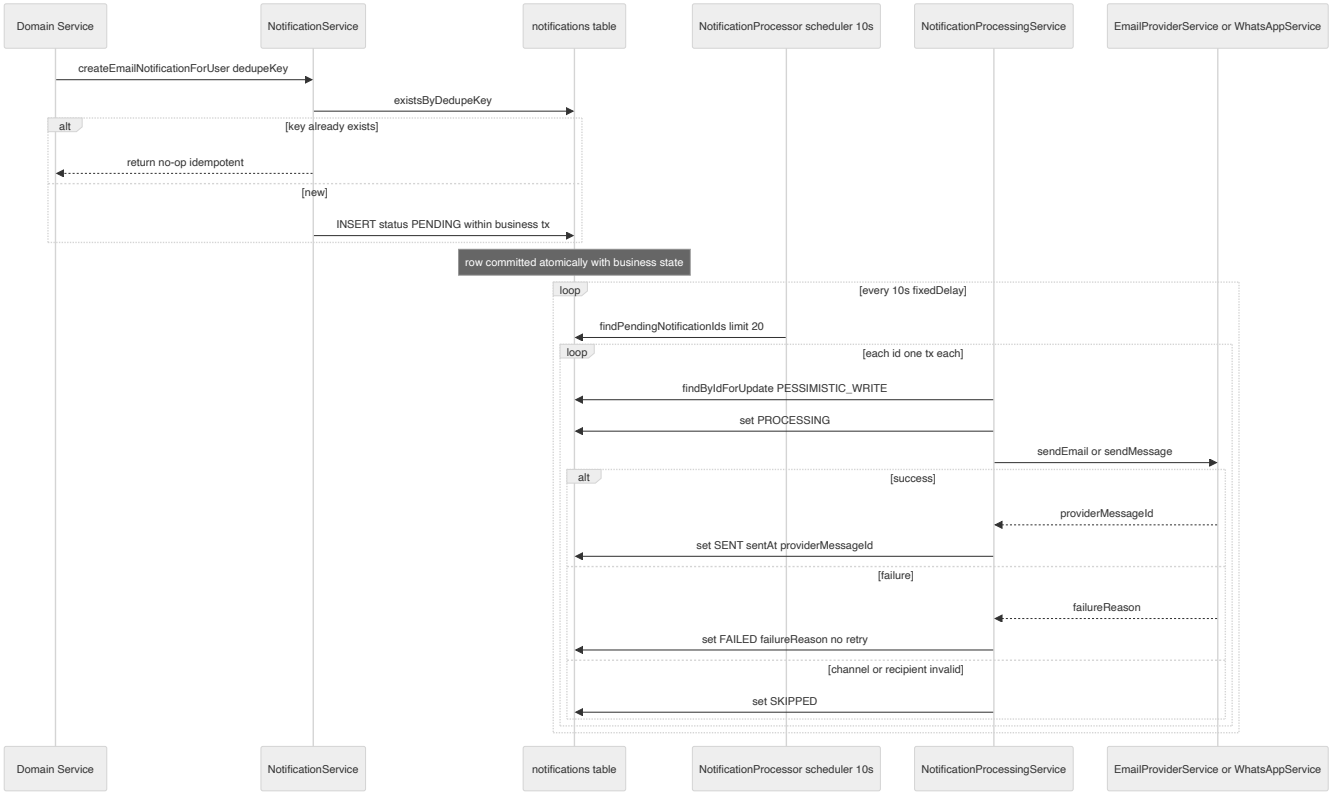
- **Actuator** exposes only `health` and `info` (`management.endpoints.web.exposure.include: health,info`), with `health.show-details: never` and the Redis health indicator disabled. **No Prometheus/metrics endpoint, no Micrometer registry, no custom health indicators** for the outbox/queue depth.
- **Logging** is SLF4J/Logback to stdout: `root: INFO`, `com.localbuddy: DEBUG`, Hibernate SQL `WARN`. There is **no correlation/request ID, no MDC enrichment, and no distributed tracing** (`JwtAuthenticationFilter` is the only `OncePerRequestFilter`; it does not set MDC). Cross-async correlation (request → AFTER_COMMIT listener → outbox row → 10s-later worker) is therefore manual.
- **Ad-hoc timing logs** exist on hot booking paths (`LOGGED_IN_BOOKING_TIMING publishEventMs=...`, `GUEST_BOOKING_TIMING ...`) and scheduler info logs (`purgeExpiredCheckIns`, payout warnings) — useful but unstructured.

Error/retry posture is fail-soft:

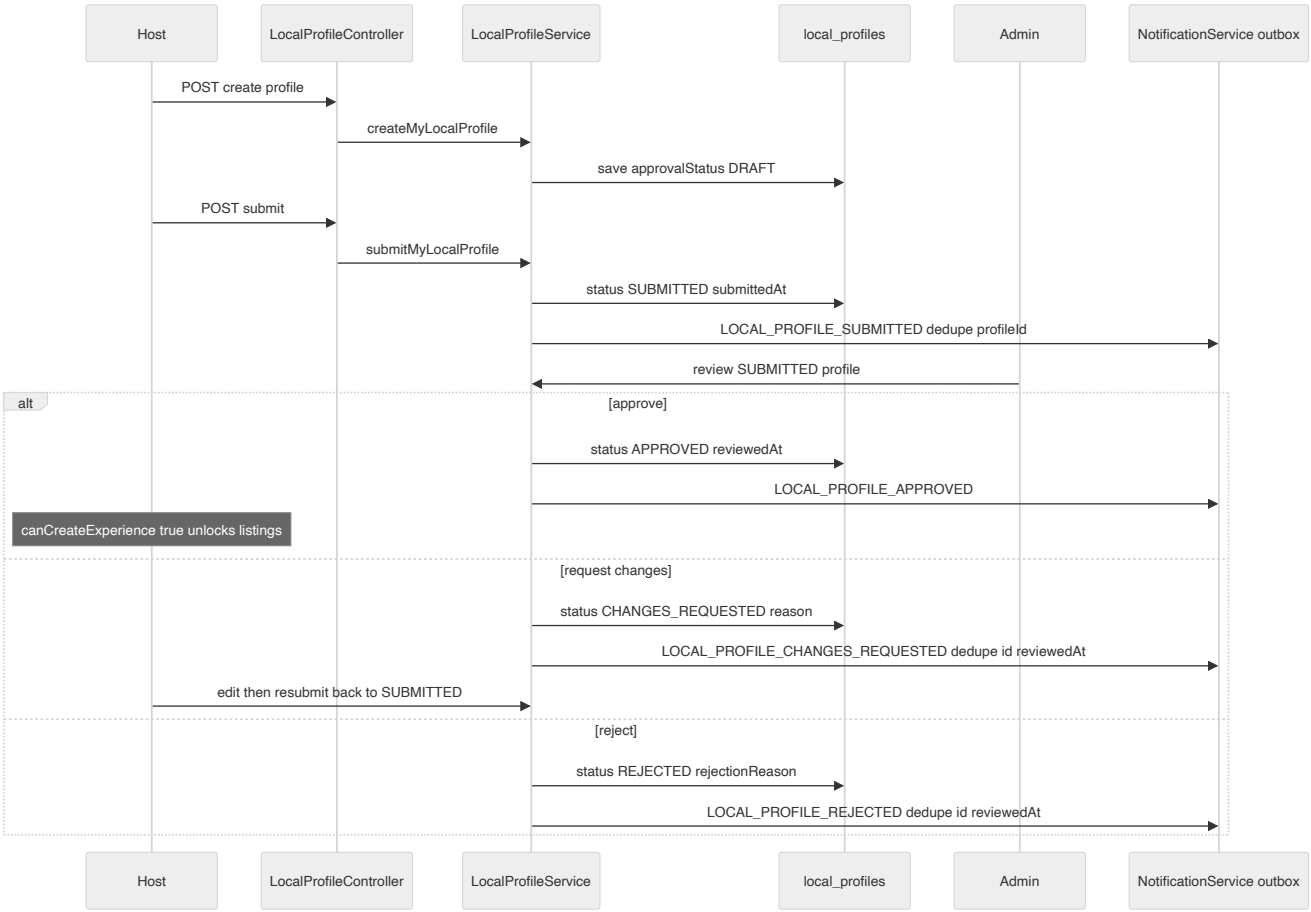
- `GlobalExceptionHandler` (`@RestControllerAdvice`) maps domain exceptions to a uniform `ErrorResponse` (timestamp/status/error/message/path). The catch-all `Exception` handler returns a generic 500 with **no stack trace in the response** (good) but **also does not log the exception** in the handler itself (observability gap — the stack only surfaces via Spring's default logging).
- **No retry anywhere in the async layer.** The outbox worker never re-attempts `FAILED`; schedulers do not retry failed rows; `PayoutScheduler` swallows per-host failures with a `log.warn` and moves on. Resilience comes purely from **next-tick re-polling of still-eligible rows** plus DB idempotency, not from explicit retry/backoff machinery.
- **Single-instance assumption** is the overarching trade-off: no ShedLock, no leader election, no queue. It is operationally simple and cheap but blocks horizontal scaling without first adding distributed locking to the scheduler fleet.

Diagrams

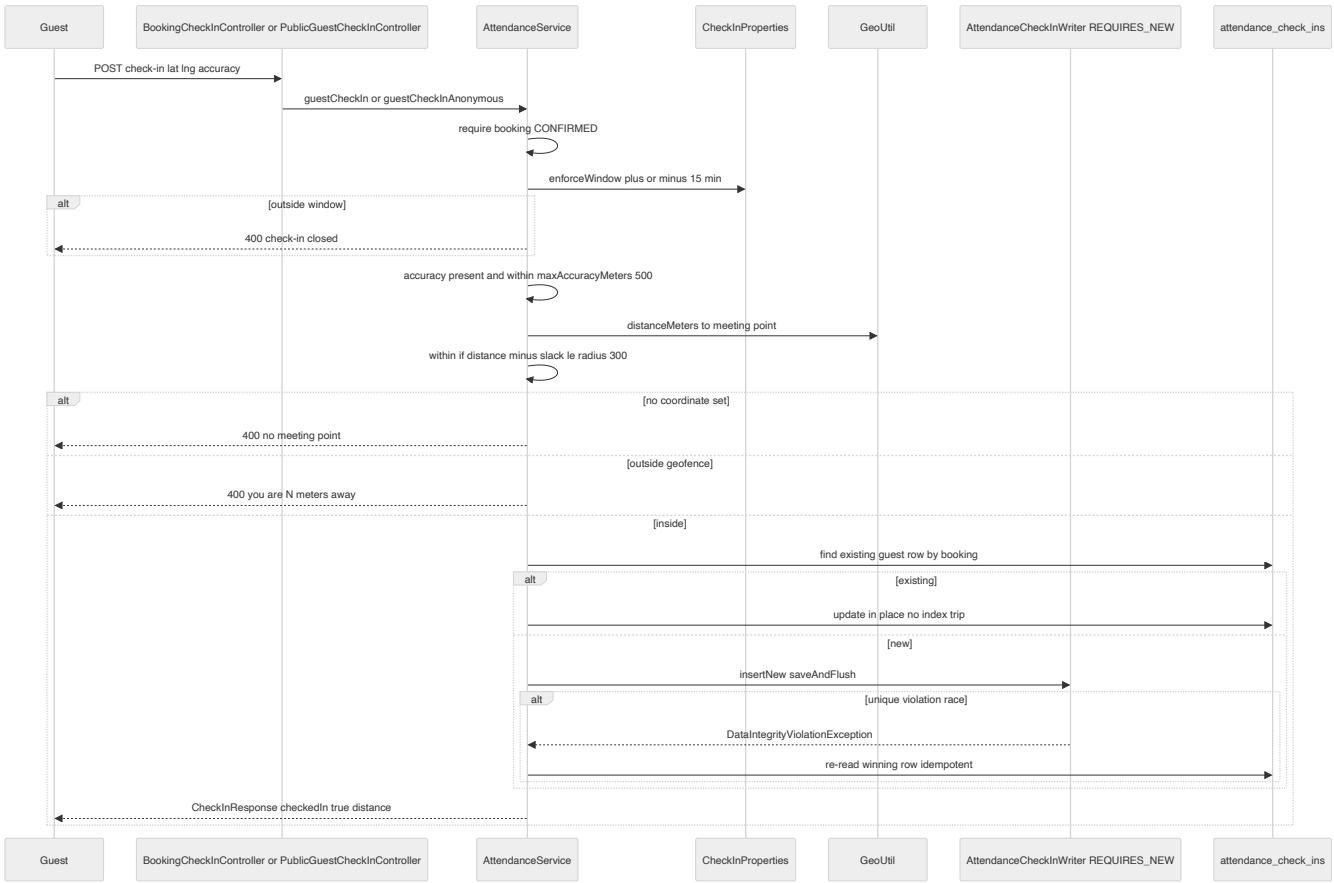
Notification Outbox Worker Lifecycle



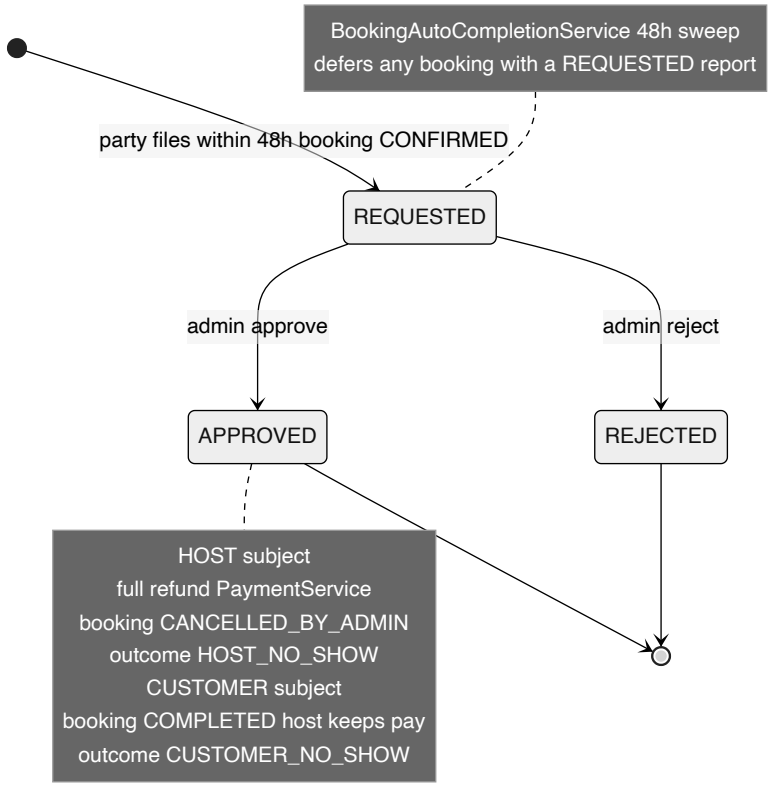
Host Onboarding to Admin Approval



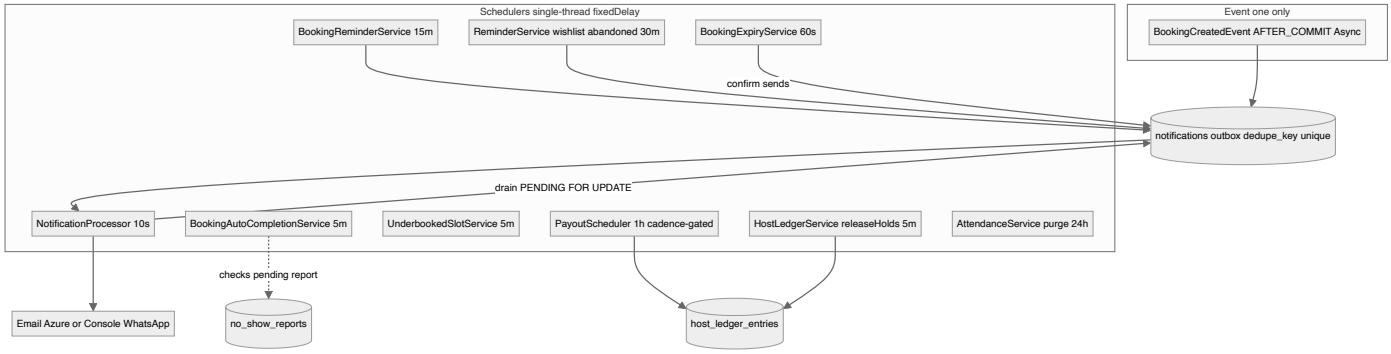
Guest Geofenced Arrival Check-in



No-Show Report to Verification to Refund



Scheduled Job Fleet and Outbox Backbone



Key architectural decisions & trade-offs

- Transactional outbox for delivery: notifications are persisted as PENDING rows in the notifications table inside the business transaction, then drained asynchronously by a polling NotificationProcessor (fixedDelay 10s). This decouples delivery from request latency and survives provider outages, at the cost of up to ~10s delivery delay.
- Idempotency is pushed entirely to the database. Every emission carries a deterministic dedupeKey with a UNIQUE constraint (notifications.dedupe_key); duplicates are swallowed via existsByDedupeKey plus a DataIntegrityViolationException catch. Schedulers can therefore run as often as they like (at-least-once emission, exactly-once row).
- Eventing is minimal and in-process: only BookingCreatedEvent exists, delivered via @TransactionalEventListener(AFTER_COMMIT) + @Async so notification fan-out runs after the booking commits and off the request thread. All other 'events' are direct synchronous service calls or scheduler polls — there is no message broker.
- Scheduling is built on plain @Scheduled fixedDelay with the default single-threaded scheduler; ~11 jobs share one thread. There is NO ShedLock / distributed lock, so the design implicitly assumes a single app instance. Horizontal scaling would cause duplicate scheduler runs (mostly safe due to dedupe keys, but payouts/expiry rely on row-level pessimistic locks, not job-level locks).
- Concurrency on attendance check-ins is handled with V23 partial unique indexes (uq_attendance_host_per_slot, uq_attendance_guest_per_booking) plus a REQUIRES_NEW nested insert (AttendanceCheckInWriter) so a racing double-submit fails only the nested tx and the caller recovers by re-reading the winning row.
- Geofence trust model is asymmetric: guests are hard-gated (must be inside the error-adjusted geofence with trustworthy GPS accuracy <= 500m), hosts are never rejected for distance (soft check-in with a warning). Check-ins are explicitly operational signal, not refund proof, and are purged after 90 days.
- No-show resolution is human-in-the-loop: either party files within 48h, an admin verifies; HOST no-show triggers a full automated refund via PaymentService and CANCELLED_BY_ADMIN, CUSTOMER no-show is informational. A separate BookingAutoCompletionService auto-completes confirmed bookings 48h after start, but defers any booking with a still-pending report.
- Error/retry posture is fail-soft, not retrying. A failed email/WhatsApp send marks the row FAILED with a failure_reason and is never retried by the worker (no backoff, no max-attempts, no dead-letter); PROCESSING rows that crash mid-flight are also never reclaimed. Operability depends on querying the notifications table by status.
- Observability is thin: Actuator exposes only health and info, logging is SLF4J/Logback to stdout at DEBUG for com.localbuddy with no correlation/trace IDs, no Micrometer metrics registry, and no distributed tracing. Some hot paths emit ad-hoc timing logs (LOGGED_IN_BOOKING_TIMING, GUEST_BOOKING_TIMING).

← back to the architecture index